> **NOTE**
>
> Join the Astra RPG Discord server!
> There is now a dedicated **Discord server** for Astra RPG Framework and its extensions. Join to **receive notifications** about new extension releases and important updates, to **ask for new features**, **report bugs**, **share ideas**, and **showcase your Astra creations** with other developers.
> **Join the Discord Server: https://discord.gg/nJVRMkGrZg**

Introduction

Astra RPG Framework is a lightweight, modular framework that takes a different approach compared to other RPG solutions. While many existing frameworks offer comprehensive, pre-built systems, they often come with steep learning curves and rigid structures that can limit your creative freedom. Astra RPG Framework instead focuses on providing a clean, flexible foundation that you can shape to your exact needs.

Key advantages:

- **Modular Architecture**: Pay just for what you need. Astra RPG Framework serves as the foundation, with additional packages building upon it to extend functionality in a modular way.
- **True Flexibility**: Unlike more rigid frameworks, Astra RPG Framework defines only essential concepts, letting you model any game system without being constrained by pre-made assumptions. The limitations are minimal.
- **Gentle Learning Curve**: Start creating immediately with intuitive, inspector-driven workflows, avoiding the complexity of larger frameworks.
- **100% Inspector-Driven**: Make changes and balance your game without touching code. Game designers can tweak values through ScriptableObjects in real-time, even during play mode, without needing recompilation. This also enables rapid testing and debugging by allowing instant value adjustments.
- **Minimal Lock-in**: The framework's lightweight nature means you're never locked into specific game design patterns.

Whether you're creating a traditional RPG, a roguelike/roguelite, an MMO, or even a game with unique mechanics, Astra RPG Framework adapts to your needs without forcing you into predetermined patterns. It provides essential building blocks for managing attributes, statistics, levels, and classes, along with powerful systems for controlling stat growth through customizable formulas, handling game events, and implementing scaling calculations. This lets you focus on the creative aspects of game development while having precise control over how your game elements evolve and interact.

Vocabulary of Astra RPG Framework

The package is developed around the concept of *entity*, so let's clarify what we mean by this term in the context of Astra RPG Framework. In its most minimal version, an entity is a `GameObject` that has a set of statistics. Optionally, an entity can have attributes, can level up, and can have a class. Let's clarify what we mean by each mentioned term.

Statistics (Stat)

A statistic is a value that quantifies an aspect of the entity. The meaning of this aspect is solely due to the concept it refers to.

Examples

In an RPG, a statistic can be `physical damage`. The concept of physical damage refers the player to the amount of damage inflicted by physical attacks, whether with weapons or without. Other statistics can be `ability power`, `defense`, `speed`, `armor penetration`, `range`, etc.

Attributes

An attribute is a value that can influence the value of one or more statistics. The weight of its influence on the statistics can be variable.

Examples

In an RPG, attributes can be: `strength`, `dexterity`, `intelligence`, `constitution`, etc. Considering the previous example of statistics, `strength` could influence `physical damage`, `dexterity` would increase `speed`, `intelligence` would increase `ability power`, and `constitution` would increase `defense`.

Experience and Level

The entity can gain experience and level up. This functionality is used by the class to express how attributes and statistics grow with levels, for that particular class.

Class

The class is associated with a set of statistics and optionally a set of attributes. The class describes how statistics and attributes vary with levels.

Examples

In RPGs most common classes are: `warrior`, `rogue`, `mage`, `paladin`, and so on. These classes have different attribute values. For example, a warrior will have more `strength` and `constitution` than a mage. The `rogue` might have the highest `dexterity`, etc.

How is Astra RPG Framework organized and how does it work?

Entity

A `GameObject` becomes an entity once the `EntityCore` and `EntityStats` `MonoBehaviour`s (Mono) are added to it. `EntityCore` comes with a built-in `EntityLevel` (plain C# `class`) that manages the experience and the level of the entity.

Stat

A `Stat` is a class that derives from `ScriptableObject` (SO) and represents a statistic in the game. Each statistic has a name (the name given to the SO instance of the created `Stat`), and we can choose whether to provide it with a maximum and/or minimum value. Additionally, we can define how that statistic grows or is reduced, depending on certain `Attributes`.

StatSet

A `StatSet` is a class derived from ScriptableObject that defines a collection of `Stat`s. Stat sets can be composed by combining other sub-stat sets, enabling hierarchical organization and easy reuse of statistics among different entities or classes.

EntityStats

`EntityStats` allows us to configure:

- the base statistics
- the flat modifiers
- the *StatToStat* modifiers
- the percentage modifiers We will see what these modifiers are in the section Understanding Stat Modifier Types.

The base statistics can be *fixed*, or instead derive from a class if the entity has one assigned. If we use the fixed ones, we must also provide a `StatSet`, while if we use those of a class, the class's `StatSet` will be used. If the entity levels up and we want its statistics to grow with levels, we are forced to use a class, as the *fixed* statistics are immutable.

Class

`Class` derives from SO and represents a game class. Each class has a name, a `GrowthFormula` that defines how the base Max HP grows with levels, a `StatSet`, optionally an `AttributeSet`, and associates each `Stat` of the provided StatSet with a `GrowthFormula` that describes how the statistic varies with levels. Similarly, if an `AttributeSet` is provided, it will be possible to associate a `GrowthFormula` for each `Attribute` present in the set, to describe how the attributes vary with levels.

EntityClass

`EntityClass` derives from Mono and allows us to assign a `Class` to our entity.

Attribute

An `Attribute` is a class that derives from SO and represents an attribute in the game. Each attribute has a name and, like statistics, can have a maximum and minimum value.

AttributeSet

An `AttributeSet` is a class that derives from SO and defines a set of `Attribute`s.

EntityAttributes

Optionally, we can add the Mono `EntityAttributes` to our entity if we want to give it attributes. `EntityAttributes` allows us to specify how many attribute points to provide at each new level. These points can be spent on various attributes to increase their value. For `EntityAttributes` we can configure:

- the base attributes
- the flat modifiers
- the percentage modifiers Similarly to `EntityStats`, we can decide whether the base attributes are *fixed* or if they instead derive from the class associated with `EntityClass`.

Growth Formula

To express how `Stats`, `Attributes`, Max HP, and the experience required to level up vary at each level, we can use instances of `GrowthFormula`. This is a class that derives from SO and allows us to define a mathematical function, or a system of functions, that describe how a value changes as levels increase. We will see in more detail how to define a `GrowthFormula` in the Growth Formulas section.

Scaling Formulas

Scaling formulas provide a flexible way to define how values such as damage, healing, or other effects are calculated based on one or more attributes or stats. They allow you to combine base values (which can be constant or level-dependent) with contributions from various stats and attributes, each weighted by customizable scaling components.

Scaling components

Specify how much a particular stat or attribute influences the final value of the scaling formula, enabling complex and dynamic calculations for abilities, equipment, or other game mechanics. This modular approach lets you easily adjust and extend scaling logic to fit your game's needs.

Game Events

Game events are ScriptableObjects that allow you to implement the Observer pattern in your game. They provide a way to decouple systems by broadcasting notifications when something happens (such as a player jumping, leveling up, or taking damage). Listeners can subscribe to these events and react accordingly, all through inspector-driven workflows. Game events can carry context parameters, making them flexible for a wide range of use cases.

 Game Event Generators

Game Event Generators are ScriptableObjects that let you define custom game events with up to four context parameters. They automate the creation of event and listener classes, making it easy to extend your event system for complex gameplay scenarios. You can specify parameter types and documentation, and generate code and assets directly from the inspector.

Game Actions

Game Actions are ScriptableObjects that encapsulate reusable pieces of logic that can be executed in response to game events or other triggers. They promote modularity and code reuse by allowing you to define specific actions (like teleport player to base, enable a GO, resurrect an entity, etc.) that can be easily invoked from various parts of your game without duplicating code. They are asynchronous and can be also used as responses to Game Events.

 Game Tags

`GameTag`s are lightweight identifiers that let you classify Astra assets and components for gameplay logic and editor organization. They appear as colored pills in tag-aware inspectors, making it easy to scan related assets at a glance and to build tag-based queries or conditions. For the full authoring workflow, see Game Tags.

 Conditions

`Condition`s are reusable predicates evaluated against runtime context such as the current holder, performer, or event payload. They are authored directly in the Inspector through polymorphic condition fields and can be used to gate game actions and other reactive behaviors without writing bespoke code for every rule. For the complete model and editor workflow, see Conditions.

How is Astra RPG Framework implemented?

The package is developed around a Scriptable Objects architecture inspired by the GDC talk of Ryan Hipple. In a nutshell, the main benefits provided by this architecture are:

- **encapsulation**: separation of game logic from data. Game logic code shouldn't mix with data. All data is nicely wrapped withing SO instances
- **game designers friendly**: game designers can make changes and balancements from the inspector without touching the code

- **greater reusability**: most features are `ScriptableObject`s that can be reused by many components
- **greater testability**: being data separated from code, is easier to isolate and fix bugs. Moreover, SO events can be raised with ease at the press of a button from the inspector interface, easing and speeding up debugging even further.

Flexibility of Astra RPG Framework

Although the package is specifically designed for RPG games or games with progression systems, its flexibility allows it to be used in almost any game. As it allows creating attributes like `strength`, `dexterity`, `agility`, etc., and statistics such as `physical attack`, `magic power`, `physical defense`, etc., in RPG, Roguelike, MMO games, etc., nothing prevents it from being used, for example, to implement a firearm. The attributes could be `weight`, `size`, `ergonomics`, etc., and the statistics `recoil`, `handling`, `stability`, `intimidation`, etc. Attributes can influence statistics. A heavier weapon could reduce `handling` but increase `stability`. A larger weapon could reduce `handling` but increase `intimidation`. A more ergonomic weapon could reduce `recoil` and increase `handling`. And so on... The weapon's levels, if present, influence the attributes and statistics, progressively improving them. Classes could represent weapon types (assault rifles, snipers, shotguns, etc.), and each class could have its own set of dedicated attributes and statistics. For example, shotguns could have, in addition to the aforementioned ones, the `barrel length` attribute that influences the `pellet spread` statistic.

Workflows

Creating instances of the objects

All the scriptable objects provided by the framework can be created through the Unity Editor by either right-clicking in the hierarchy and selecting `Create > Astra RPG` or navigating to the `Assets` menu at the top of the window and choosing `Create > Astra RPG`.

Mandatory, re-play, and read-only fields

Fields marked with a red asterisk (*) are mandatory and must be filled out to ensure proper functionality of the framework.

Fields marked with an orange **R** are re-play fields. Any changes made to these fields during playtime will require a restart to ensure the changes take effect.

Fields marked with a teal **RO** are read-only fields. The framework manages these fields internally, and they cannot and should not be modified directly by the user.

Some utilities

Almost every class provided by this package uses events or variables in the form of `ScriptableObject`. Therefore, let's quickly introduce these concepts so that we are clear about what we are talking about when we encounter them in the following paragraphs.

Game events as `ScriptableObjects`

The Scriptable Objects based architecture allows us to implement the Observer pattern through scriptable objects. In the simplest case, with events without context, we can define various game events as `GameEvent` instances: a class that derives from `ScriptableObject`. For example, we can create an instance called `PlayerJumped` that represents the event "The player has jumped". This event will notify all listening systems when it occurs. Systems subscribe to this event using the `MonoBehaviour` `GameEventListener`. We can assign a `GameEvent` to this component, and it will handle the subscription and invoke a callback when the event is triggered. The callback is a UnityEvent, so we can select a callback to invoke in response to our event directly from the inspector.

For more details, see the Game Events section.

Int and Long Vars

Another common use of `ScriptableObject` in the SO based architecture is to define variables. The main advantage of these variables in the form of SO is that they can be easily shared between various objects that may decide to share the same value. A common example is the player's game score. There could be a game manager that adds or removes points from this variable, while the UI HUD uses it to display its

value on the screen. This way, we can keep the game manager and UI completely decoupled, passing shared values (like variables) through the inspector.

From the code, we can access the values held by these variables using the `Value` getter and setter:

```
// intVar is an instance of IntVar
int value = intVar.Value; // Get the value

// Thanks to implicit conversion, we can also use it as an int directly
int intValue = intVar; // Implicit conversion to int

intVar.Value = 10; // Set the value
```

Int and Long Refs

`IntRef` and `LongRef` allow choosing whether to use a native value (`int` or `long`) or an `IntVar`/`LongVar`. As mentioned in the previous paragraph, `IntVar` and `LongVar` have the advantage of being shareable between different components/game objects, while native values are more immediate to use and require less setup (no need to instantiate an `IntVar`/`LongVar` and assign it in the inspector).

Thanks to a custom property drawer, it will be possible, from the inspector, to check a checkbox named `Use constant` to use a native value instead of a `Ref`, and vice versa.

`IntRef` and `LongRef` are widely used in the package's `MonoBehaviour`.

From the code, we can access the values held by these references using the `Value` getter and setter. It is worth mentioning that when we use the setter, we need to provide a native `int` or `long` value. If `Use constant` is unchecked, this value will be assigned to the `Value` property of the referenced `IntVar` or `LongVar` instance; if `Use constant` is checked, the assignment only updates the local constant value and does not affect any referenced variable.

```
// intRef is an instance of IntRef
int value = intRef.Value; // Get the value
// Thanks to implicit conversion, we can also use it as an int directly
int intValue = intRef; // Implicit conversion to int
intRef.Value = 10; // Set the value

// Assigns the value of intVar to intRef.Value using implicit conversion.
// Note: This does not change the IntVar reference held by intRef, only its value.
intRef.Value = intVar;
```

Game events

The package also supports game events with up to 4 context parameters. They are generics, but in Unity, it is not possible to instantiate classes that derive from `ScriptableObject` if they are generics with unspecified type parameters. To use them, we must explicitly declare classes that derive from the generic GameEvent and fix the type parameters with concrete types. To simplify the definition of new event types, with specific types as context parameters, the package provides `GameEventGenerator`. These generators, which derive from SO, allow generating the concrete classes of `GameEvent`. We will see these generators in more detail in the Game Event Generators section. Some game events are already defined and made available by the package (see the Samples page).

Growth Formulas

Relative path: `Growth Formula`

As already mentioned in Introduction, `GrowthFormula` allows defining how a certain value varies as levels increase. A `GrowthFormula` can be instantiated through the hierarchy context menu by going to `Astra RPG Framework -> Growth Formula`. The package provides a custom property drawer for `GrowthFormula`.

Max level for the values

In the inspector of a `GrowthFormula`, we can pass an `IntVar` to define up to which level to grow the values.

Use constant at level one

If the checkbox named `Use constant value at level 1` is checked, the respective constant value will be used.

Growth expressions

The various values of the `GrowthFormula` are defined by a function where values, the y-axis, are expressed as a function of the levels, the x-axis. Such a function is defined as a composite function. Each segment of the function is represented by a string that specifies a mathematical expression for a range of levels. The string can be defined by using the Unity ExpressionEvaluator syntax. On top of it, the following terms can be used:

- `LVL`: the level at each iteration
- `PRV`: the previous value of the `GrowthFormula` (value evaluated at the previous level)
- `SPRV`: the second previous value of the `GrowthFormula` (value evaluated 2 levels ago)
- `PRV[N]`: (🏷️ *v1.3.0+*) the N-th previous value of the `GrowthFormula` (value evaluated N levels ago). N must be a valid number with respect to the level range being used. For example, N cannot be 5 if we are evaluating level an expression in the level range 3 --> 10, as PRV[5] at level 3, 4, and 5 will be undefined.
- `AT[N]`: (🏷️ *v1.3.0+*) the value of the `GrowthFormula` at level N. N cannot be a value equal or grater than the current level being evaluated.

- `SUM`: the sum of the values of the `GrowthFormula` from level 1 up to the previous level

Example of a `GrowthFormula`

Let's see an example of how to define a `GrowthFormula` for defining the Physical Attack of a warrior class. First of all, let's create a new `GrowthFormula` instance and name it `Warrior Physical Attack GF`. In the

inspector, it should look like this:

Warrior Physical Attack GF (Growth Formula)

Open

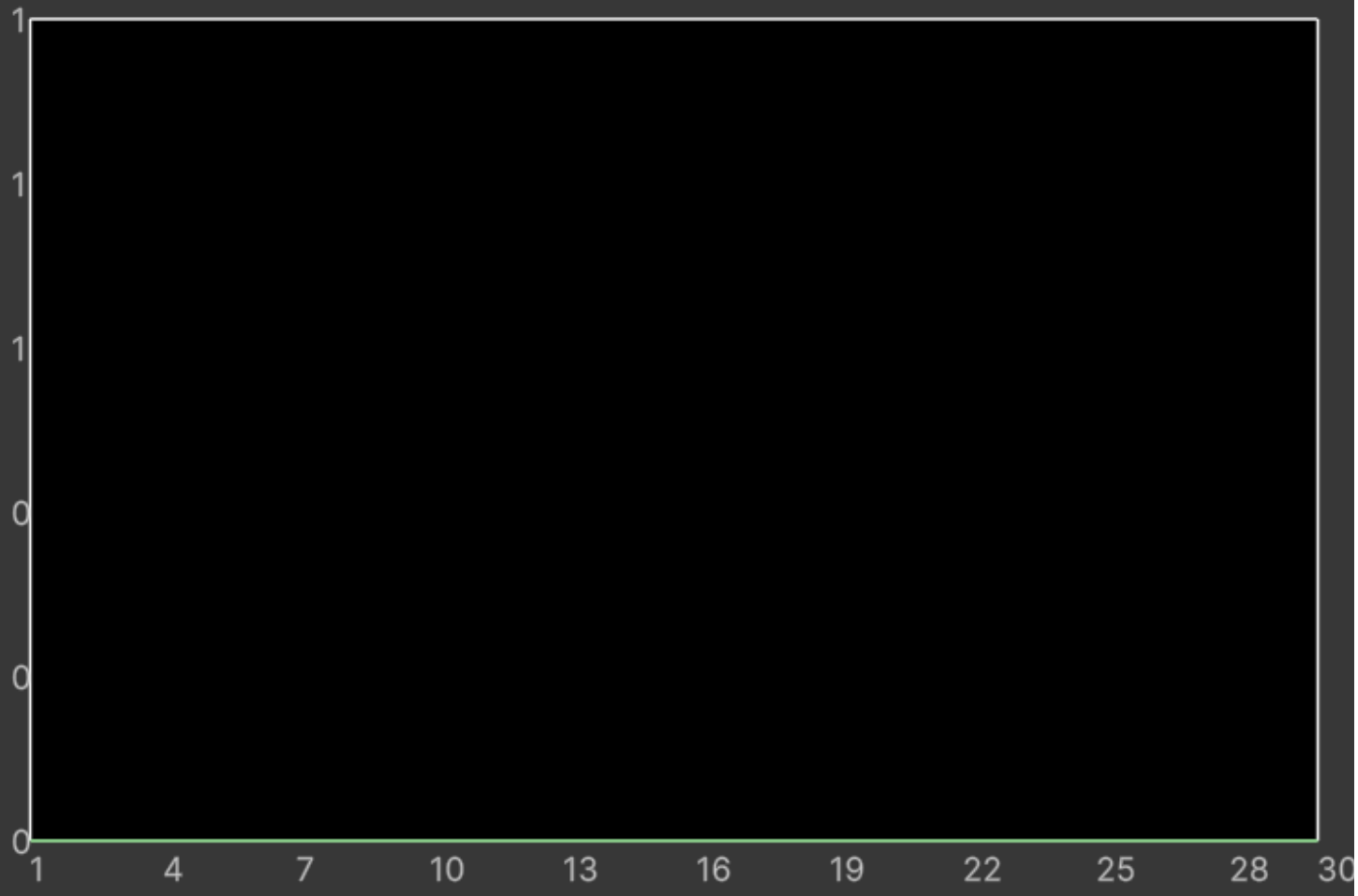
* Max Level: Max Level (Int Var)

Use Constant At Lvl 1: ✓

Constant At Lvl 1: 0

Add growth expression

Growth Values for each level - graph

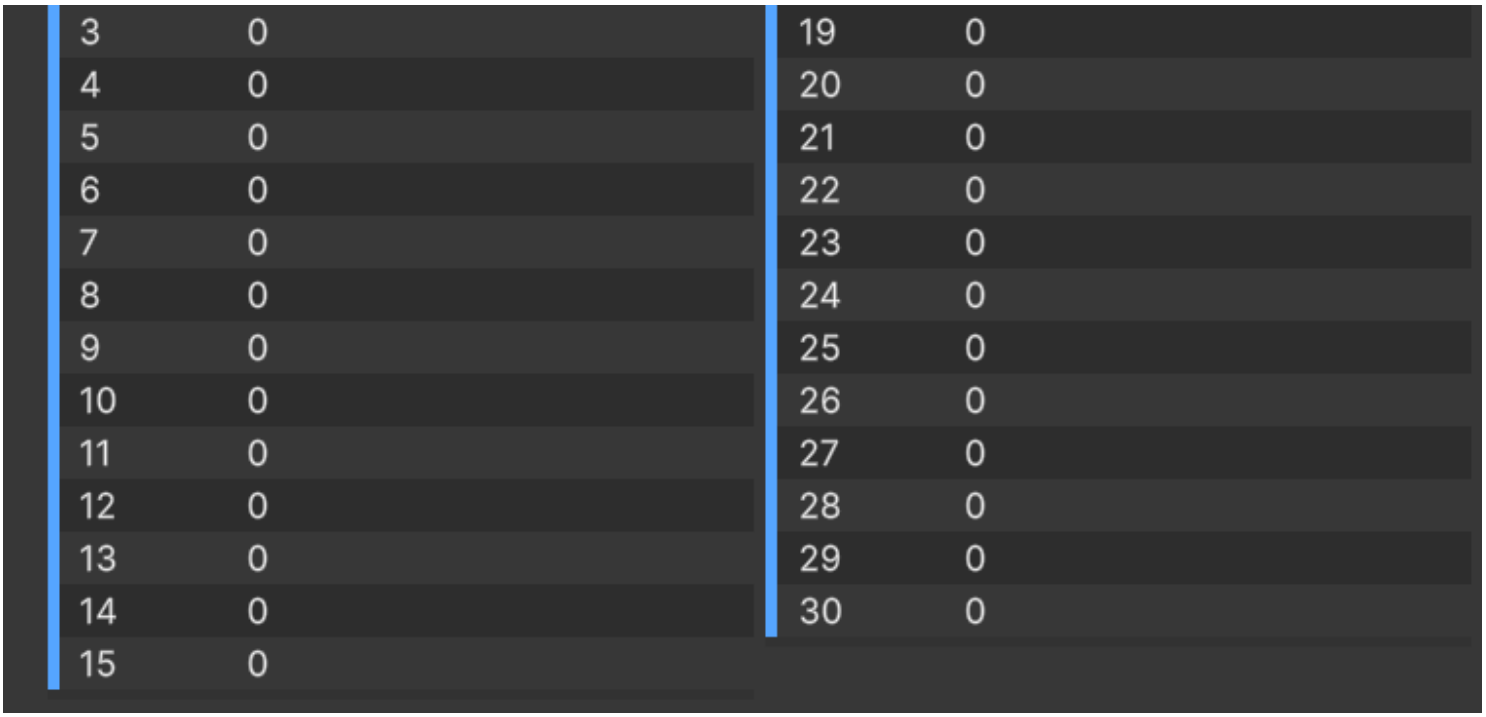


Hold the mouse for a moment on top of the graph to see the value at a certain level

Growth Values per Level

Level	Value
Lvl 1 – 30	Constant
1	0
2	0

Level	Value
16	0
17	0
18	0



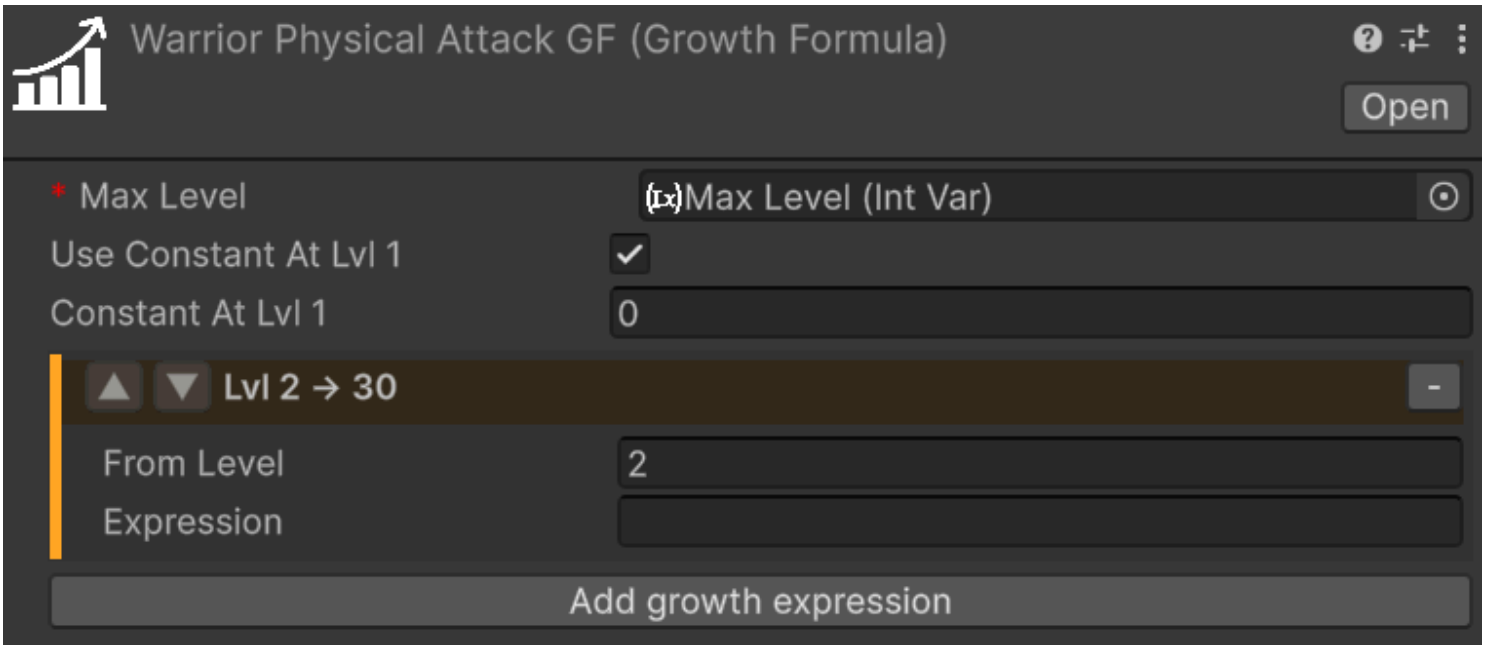
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
19	0
20	0
21	0
22	0
23	0
24	0
25	0
26	0
27	0
28	0
29	0
30	0

The `Max Level`, a mandatory field, is set with an `IntVar` assigned by default. We can edit that variable to change the maximum level that will be computed for our growth formula.

> **WARNING**
>
> When modifying the value of a variable referenced in growth formulas, such as Max Level, the growth formulas are not directly updated unless you select them in the inspector. To update all growth formulas simultaneously after changing the maximum level, a command is available in the menu: `Tools > Astra RPG Framework > Validate All Growth Formulas`.
> Validation occurs automatically during script compilation, upon entering play mode, and when instantiating a prefab. This is achieved through the `OnValidate` callback, which ensures that formulas are updated accordingly.

The `Use constant value at level 1` checkbox lets us decide whether to use a constant value at level 1 or not. If checked, the `Constant Value` field will be enabled, and we can set a value for it. In this case, we set it to 10.

The `Add new growth expression` button lets us add a growth expression for a certain range of levels of our choice. If we press it, we will see the following:



The new section includes two fields: `From Level` and `Growth Expression`.

- **From Level**: Specifies the starting level at which the corresponding `Growth Expression` becomes effective. It is pre-filled with the next level after the last defined growth expression, but it can be edited to any level. If no other growth expression is defined, it will be pre-filled with level 2 if `Use constant value at level 1` is checked, or level 1 if it is not checked.
- **Growth Expression**: Defines how the value evolves starting from the specified level.

If the `Growth Expression` overlaps with the `Constant At Lvl 1` option, a warning will appear. To resolve this, set the `From Level` field to `2` or higher, and the warning will disappear.

We want to model the Physical Attack of a warrior as follows:

- Level 1: 10
- From level 2 to level 5: +2 per level
- At level 11: flat +30 (like a bonus due to other game mechanics, such as an awakening)
- From level 12 and onward: grows by 7% each level

To achieve this, set the `Constant At Lvl 1` field to `10`. For the first growth expression, use `PRV + 2` as the formula. `PRV`, as we saw before, represents the value of the growth formula at the previous level (in this case, `10` at level 1).

This formula ensures that the value grows by `2` times the level at each subsequent level.

Next we want to press the `Add new growth expression` button to add the next growth expression for the levels. For the second growth expression, set `From Level` to `11` and use the formula `PRV + 30`. This ensures that at level 11, a flat bonus of 30 is added to the previous value.

Finally, for the third growth expression, set `From Level` to `12` and use the formula `PRV * 1.07`. This ensures that from level 12 onward, the value increases by 7% each level.

After adding these growth expressions, the `GrowthFormula` for the `Warrior Physical Attack GF` should look like this:

Warrior Physical Attack GF (Growth Formula)

Open

* Max Level: Max Level (Int Var)

Use Constant At Lvl 1: ✓

Constant At Lvl 1: 10

Lvl 2 → 10

From Level: 2

Expression: PRV + 2

Lvl 11 → 11

From Level: 11

Expression: PRV + 30

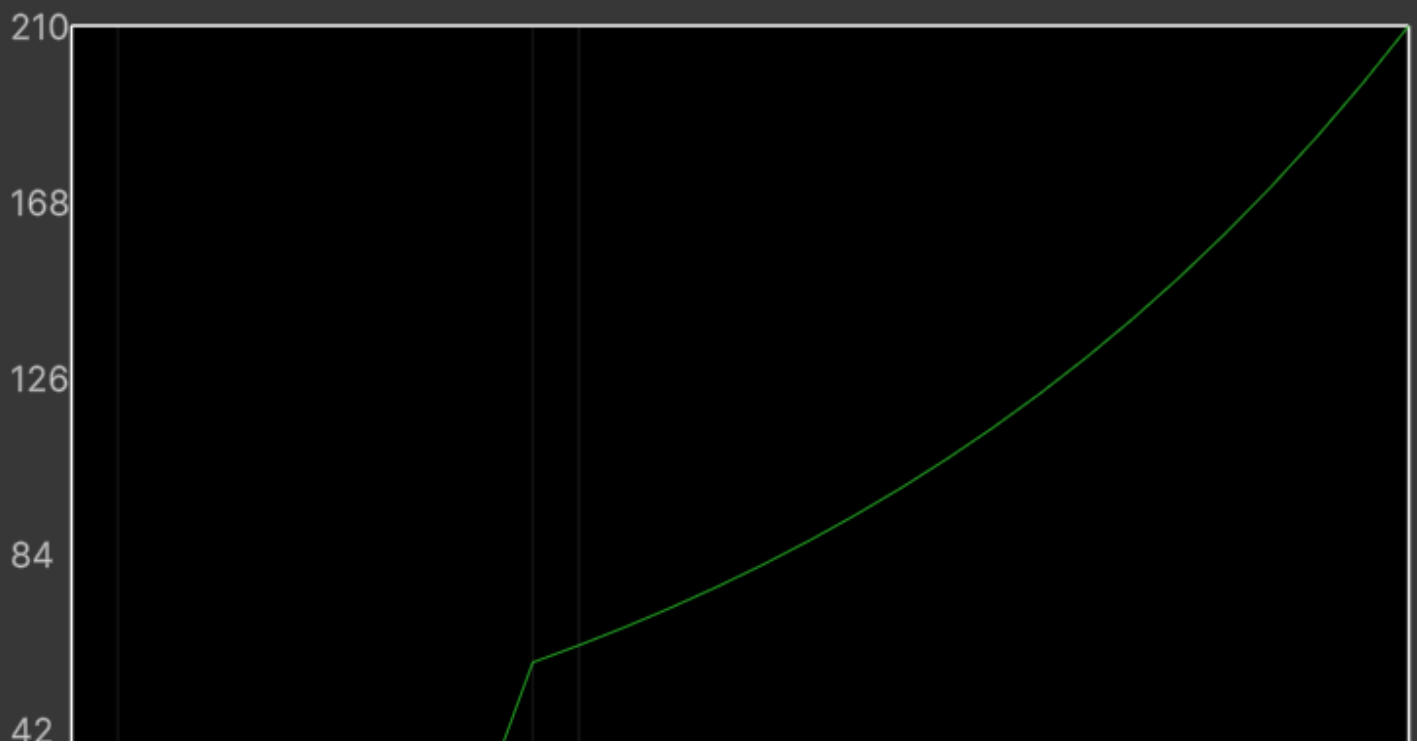
Lvl 12 → 30

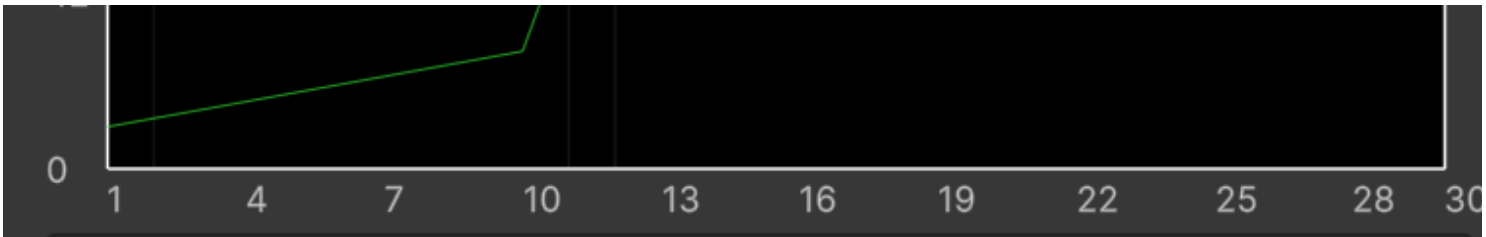
From Level: 12

Expression: PRV * 1.07

Add growth expression

Growth Values for each level - graph





Hold the mouse for a moment on top of the graph to see the value at a certain level

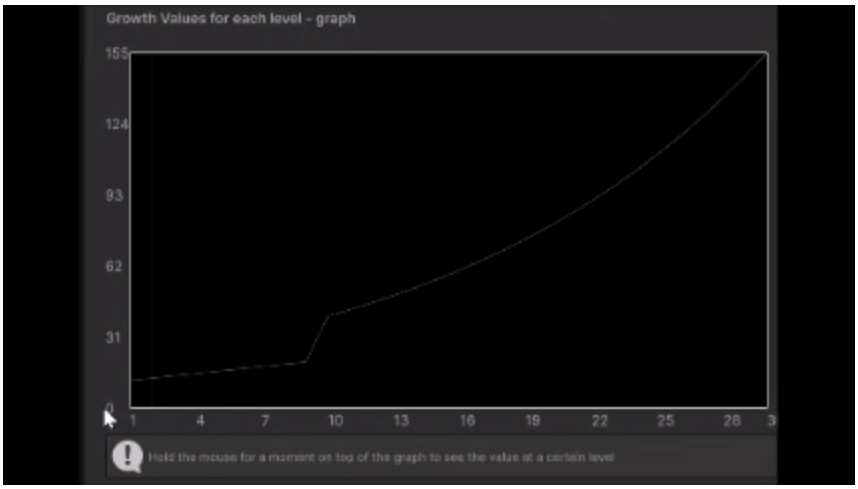
Growth Values per Level

Level	Value	Level	Value
Lvl 1 – 1 | Constant		14	71
1	10	15	76
Lvl 2 – 10 | PRV + 2		16	81
2	12	17	87
3	14	18	93
4	16	19	99
5	18	20	106
6	20	21	114
7	22	22	122
8	24	23	130
9	26	24	139
10	28	25	149
Lvl 11 – 11 | PRV + 30		26	160
11	58	27	171
Lvl 12 – 30 | PRV * 1.07		28	183
12	62	29	196
13	66	30	209

With this setup, the `GrowthFormula` will correctly calculate the Physical Attack values for the warrior class based on the specified rules.

Interactive Chart

If you hold your mouse for a moment onto the chart, a label will show up, showing the exact value of the growth formula at the pointed level:



Retrieving growth values from code

To retrieve the values of a `GrowthFormula` from code, you can use the `GetGrowthValue(int level)` method. For example, to get the Physical Attack value at level 5, you can do:

```
// warriorPhysicalAttackGF is a reference to the Warrior Physical Attack Growth Formula
int physicalAttackLevel5 = warriorPhysicalAttackGF.GetGrowthValue(5);
```

Game Actions

Relative path: `Game Actions`

A `GameAction` (🏷️ *v1.4.0+*) is a `ScriptableObject` that encapsulates reusable logic which can be executed by different components or systems in the game. Game actions make it easy to assign behavior to GameObjects via the Inspector, promoting code reuse and separation of concerns.

`GameAction`s accept a generic context parameter via `GameAction<TContext>`. The recommended context type for built-in actions is `IHasEntity`: because most `GameEventListener` payloads in the framework implement this interface, `IHasEntity`-based actions connect directly to any `GameEventListener` response without needing to extract a `Component` reference first. `Component`-based variants remain available for cases where a raw `Component` context is passed, but the `IHasEntity` variants should be the first choice when wiring actions to events.

At authoring time, polymorphic action references are usually exposed through the non-generic `GameActionBase`, which exists so Inspector fields can accept actions with different concrete context types. At runtime, the action still executes through its concrete `GameAction<TContext>` implementation, so context compatibility continues to matter.

The `IHasEntity`-based variants are listed under `Astra RPG Framework/Game Actions/Context: Entity/` in the asset creation menu.

The framework includes several built-in `GameAction` implementations for `IHasEntity` contexts:

- **Toggle Entity Active GO** (`Context: Entity/Toggle Entity Active GO`): Sets the active state of the entity's `GameObject` to a configured value.
- **Toggle Entity Renderers** (`Context: Entity/Toggle Entity Renderers`): Enables or disables all renderers on the entity's `GameObject` and its children.
- **Toggle Entity Colliders** (`Context: Entity/Toggle Entity Colliders`): Enables or disables all colliders on the entity's `GameObject` and its children.
- **Destroy Entity Game Object** (`Context: Entity/Destroy Entity Game Object`): Destroys the `GameObject` of the entity exposed by the context.
- **Composite** (`Context: Entity/Composite`): Executes a sequence of `GameAction<IHasEntity>` actions in order, passing the same context to each.
- **Delayed** (`Context: Entity/Delayed`): Executes a `GameAction<IHasEntity>` after a specified delay (in seconds).
- **Conditional** (`Context: Entity/Conditional`): Executes a `GameAction<IHasEntity>` only when a configured condition evaluates to true against the entity.
- **Do Nothing** (`Context: Entity/Do Nothing`): Does nothing. Useful as a placeholder, for testing workflows, or to satisfy required fields that must reference a `GameAction` (for example, default on-death and on-resurrection actions in Astra RPG Health).
- **Increase Counter** (`Context: Entity/Increase Counter`): Increases the value of a `LongRef` by a specified amount. Negative values decrease the counter. Useful for tracking statistics like enemies defeated, entities spawned, or interactions performed.
- → **Component Projection** (`Context: Entity/Projections/→ Component Projection`): Bridges an `IHasEntity` context to an inner `GameAction<Component>`, by passing `context.Entity` as the component. Use this to reuse existing `Component`-based actions on entity event listeners. Note that rich payload data (damage amounts, level deltas, etc.) is discarded — use this only for structural actions that only need to know *which* entity was involved.

Equivalent `Component`-based variants exist for all of the above (found under `Context: Component/`). Use them when a raw `Component` is the only context available and no `IHasEntity` payload is in scope.

`GameAction` assets are also taggable, so larger projects can organize action libraries with the same pill-based workflow used by other Astra assets. This becomes especially useful once you start pairing actions with tag-based filtering rules. For the complete authoring flow, see Game Tags and Conditions.

`GameAction`s may target infrastructure/flow control or actual gameplay mechanics. Most of the examples above are infrastructure-oriented. Examples of gameplay-oriented `GameAction`s include:

- **Grant Experience Game Action**: Grants a specified amount of experience to an entity implementing `IEntityLevel`.
- **Drop Loot Game Action**: Spawns collectible loot.
- **Teleport To Base Game Action**: Teleports a character back to a base location.

For invoking `GameAction`s call the `ExecuteAsync` method, passing the required context parameter. For example, let's say we have a reference to a `DelayedEntityContextGameAction` called `delayedAction` that invokes another action after some time, and `entityContext` is an `IHasEntity` payload:

```
// entityContext is an IHasEntity payload (e.g. from a GameEventListener response)
Awaitable delayedActionAwaitable = delayedAction.ExecuteAsync(entityContext,
destroyCancellationToken); // Async execution initiated
DoWorkHere(); // Perform other work while the action is executing
// Await the completion of the action
await delayedActionAwaitable;
Debug.Log("Delayed action completed."); // Will be printed only after the game
action completes
```

Notice the `destroyCancellationToken` parameter. This is an optional `CancellationToken` that can be used to cancel the execution of the action if the GameObject owning the invoker component is destroyed before the action completes. If not provided, the action will run to completion regardless of the owning component's lifecycle.

If your game action should complete independently of the invoker component's lifecycle, consider using the RunFireAndForget method instead. This method uses the `GameActionRunner MonoBehaviour` (adding it to the specified GameObject if missing) to execute the action. For example:

```
// Assume 'context' is the required context parameter for onDeathGameAction
// 'persistentGameObject' is a reference to a GameObject that will own the GameActionRunner
and execute the action
onDeathGameAction.RunFireAndForget(context, persistentGameObject);
```

With this approach, even if the invoker component is destroyed (e.g., because the entity died), the action will still run to completion on the `persistentGameObject`. This is useful for actions such as playing particle effects or sounds that should persist even after the original entity is destroyed.

When a `GameAction` is invoked from a `GameEventListener`, prefer the listener's owner-aware action list over binding `ExecuteAsync` directly through the UnityEvent when your logic depends on `Holder`-aware conditions or owner propagation. The owner-aware path dispatches the selected `GameActionBase` assets with the listener as runtime owner, and wrapper actions such as composite, delayed, conditional, and projection actions preserve that owner as they forward execution. For the full condition model, see Conditions.

For more details, refer to the API documentation for game actions:

- GameAction base constructs
- Concrete implementations of GameAction with open generic types

- Concrete implementations of GameAction with IHasEntity context (primary instantiable Scriptable Objects)
- Concrete implementations of GameAction with Component context (legacy/specialized use)

If you wish to create custom `GameAction`s, review the implementation of existing actions to understand best practices.

> **NOTE**
>
> `GameAction`s are a recent addition to the framework. At the time of writing, only a small set of generic actions is provided; more will be added in future releases. In the meantime, users are encouraged to implement custom `GameAction`s to meet their specific needs and to share useful patterns with the developer and the community.

Game Actions — under the hood

Is a common use case to have `GameAction`s that perform asynchronous operations, such as playing an animation, or waiting for a fixed delay. To support this use case, all `GameAction`s in the framework are designed to be asynchronous.
Obviously, not all actions need to be asynchronous; any synchronous action can be implemented as well.

`GameAction`s are implemented on top of Unity's Awaitable. This allows actions to run asynchronously without blocking the main game thread. Executing a `GameAction` returns an `Awaitable` that can be awaited to determine when the action completes.

Awaitables are a modern alternative to coroutines. They are more efficient because they use an internal pooling system to minimize allocation overhead, which improves performance in scenarios with many asynchronous actions.

> **WARNING**
>
> Awaitables have an important limitation: you must not await the same `Awaitable` instance more than once. Due to pooling, awaiting the same instance multiple times can cause undefined behavior, exceptions, or difficult-to-debug deadlocks. To avoid this, do not reuse the `Awaitable` returned by a `GameAction`'s `ExecuteAsync` method across multiple awaits.

For more information on Awaitables, see Unity's documentation: Awaitable Documentation.

Game Actions with Unity Events/Game Events Listeners

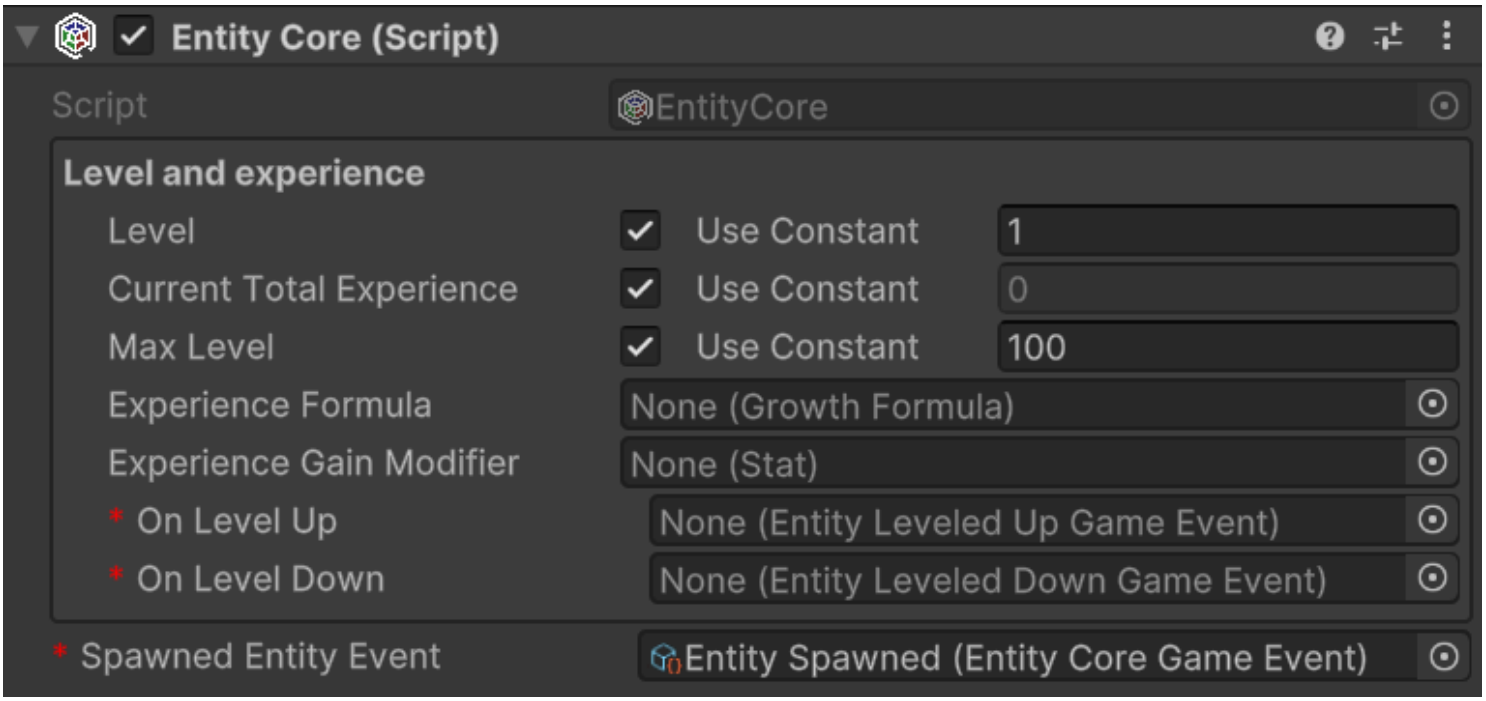
Unity Events accept Awaitables, so a GameAction can be invoked directly from a UnityEvent response. Because GameEventListeners use UnityEvents for their responses, you can wire a GameAction to a GameEventListener provided the action's context type matches the event's context.

`GameEventListener`s also expose an owner-aware action list that accepts `GameActionBase` references. This is usually the better inspector workflow for event-driven actions because it preserves the listener component as runtime owner, which allows nested actions and `Holder`-based conditions to resolve consistently.

> **NOTE**
>
> UnityEvents do not support asynchronous return values. Invoking a GameAction from a UnityEvent will execute the action fire-and-forget — the event invoker will not await its completion.
>
> If you need to run follow-up logic after the action finishes, prefer one of these patterns:
>
> - Use a Composite Game Action that first executes the desired GameAction and then runs the follow-up action as the next step.
> - Use a custom MonoBehaviour that executes the GameAction and handles the continuation (awaiting the action and performing follow-up). Attach that MonoBehaviour via the UnityEvent or subscribe to the GameEvent in code.

Making a `GameObject` an entity

To make a `GameObject` an entity, we need to add the `MonoBehaviour` `EntityCore` to it. Select your object from the hierarchy and click, in the inspector, on "Add component". Then search for and select `EntityCore`.



From the inspector, we can configure several values. Let's analyze them one by one.

`Level`: defines the entity's level. By changing its value, we can assign a different level to the entity directly from the inspector. This can be useful for testing purposes. You'll notice the `Use Constant` checkbox. If checked, you can pass an `IntVar` instead of using a constant.

`Current Total Experience`: Represents the total experience possessed by the entity.

> **WARNING**
>
> If you've passed a `LongRef` for the current total experience, the value contained in this variable should not be modified manually. If `Use constant` is checked instead, the value is readonly.

`Max Level`: The maximum level the entity can reach

`Experience Formula`: `GrowthFormula` that describes how the total experience required to reach the next level grows at each level.

Built-in framework events such as `Spawned`, `Level Up`, `Level Down`, `Stat Changed`, and `Attribute Changed` are dispatched through the active Astra RPG Framework configuration. Use `Edit -> Project Settings -> Astra RPG Framework` to choose which configuration asset is active, then open that configuration asset and assign the shared event assets inside it. If no active Project Settings configuration is assigned, the runtime falls back to a `Resources/Astra Rpg Framework Config` asset.

EntityLevel code APIs

It is honorable to mention some code APIs that can be used to interact with the `EntityLevel` component.

EntityLevel exposes a `Action<EntityCore, int> OnLevelUp` property that can be used to subscribe to level-up events from code.

If we want to grant experience to the entity, we can use the `AddExp(long amount)` method. This method will automatically raise the `OnLevelUp` event if the entity levels up. Alternatively, it is available also the `SetTotalCurrentExp(long totalCurrentExperience)` method, which allows setting the total current experience of the entity. This method will also raise the `OnLevelUp` event if the entity levels up, and the `OnLevelDown` event if the entity levels down.

(🏷️ *v1.2.0*+) Similarly, the `RemoveExp(long amount)` method allows deducting experience from the entity. This method will raise the `OnLevelDown` event if the entity levels down. If you want to *respec* an entity, the `ResetToLevelOne()` method resets the entity's level and experience to level 1. This method will raise the `OnLevelDown` event if the entity levels down. Clearly, all spent attribute points will be reset as well.

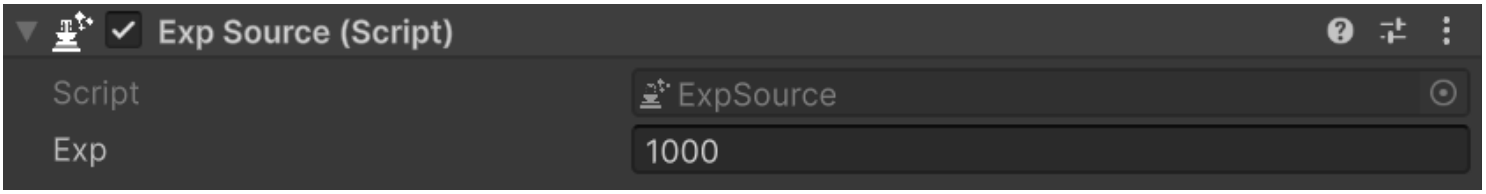
Finally, there are the `CurrentLevelTotalExperience()` and the `NextLevelTotalExperience()` methods. These methods return the total experience required to reach the current level and the next level, respectively. They are useful, for example, for checking how much experience is needed to level up.

Exp Source

The `ExpSource` `MonoBehaviour` component marks a `GameObject` as a source of experience points. When such an entity dies, collection systems (such as those provided by Astra RPG Health) can harvest its experience and award it to eligible recipients.

To add an `ExpSource` to an entity, select the entity's `GameObject` in the hierarchy and use "Add Component" from the inspector. Search for and select `ExpSource`.

In the inspector, you can configure the amount of experience this entity provides:



Exp: The base amount of experience points this entity grants when collected. This value can be modified by collection strategies of Astra RPG Health.

The Harvested Flag

Each `ExpSource` exposes a `Harvested` property. When `Harvested` is `true`, the source is considered already collected and collection strategies will skip it, preventing experience from being granted more than once per entity.

By default, `Harvested` is `false`. Collection strategies that check this flag — such as `DirectKillExpStrategySO` in Astra RPG Health — set it to `true` once the experience has been awarded, ensuring each source can only be harvested a single time.

> **NOTE**
> Not all collection strategies enforce the `Harvested` flag, and neither all collections strategies respect it. Refer to the documentation of the specific strategy you are using to understand its behavior.

Resetting the Harvested Flag

You may encounter scenarios where an entity should become harvestable again — for example, when a defeated enemy is respawned or resurrected. For these cases, the framework provides the `ToggleHarvestedExpSourceEntityGameAction`.

Relative path: `Astra RPG Framework/Game Actions/Context: Entity/Toggle Harvested Exp Source`

This action reads a `Harvested` boolean field configured in the inspector and applies it to the `IExpSource` found on the entity's `GameObject` (`context.Entity`). Set `Harvested` to `false` to make the source collectable again, or to `true` to mark it as already harvested and prevent further collection.

Because the context type is `IHasEntity`, this action can be wired directly to any `GameEventListener` whose payload exposes an entity — for example, an `EntityResurrectedGameEventListener` — without any additional bridging.

> **NOTE**
> A `Component`-based variant (`ToggleHarvestedExpSourceComponentGameAction`, relative path `Context: Component/Toggle Harvested Exp Source`) is also available for cases where only a raw `Component` context is in scope.

Custom Exp Sources

If you need finer control over how experience is provided — for example, if the amount should scale with the entity's level, or the source should conditionally refuse collection — you can implement the `IExpSource` interface directly on a `MonoBehaviour`:

```
public interface IExpSource {
    bool Harvested { get; set; }
```

```
    long Exp { get; }
}
```

Any `MonoBehaviour` implementing `IExpSource` is fully compatible with collection strategies that target this interface, including those provided by Astra RPG Health.

Creating Astra RPG Framework assets

All the instances of the various assets that derive from `ScriptableObject`s can be created in the following ways:

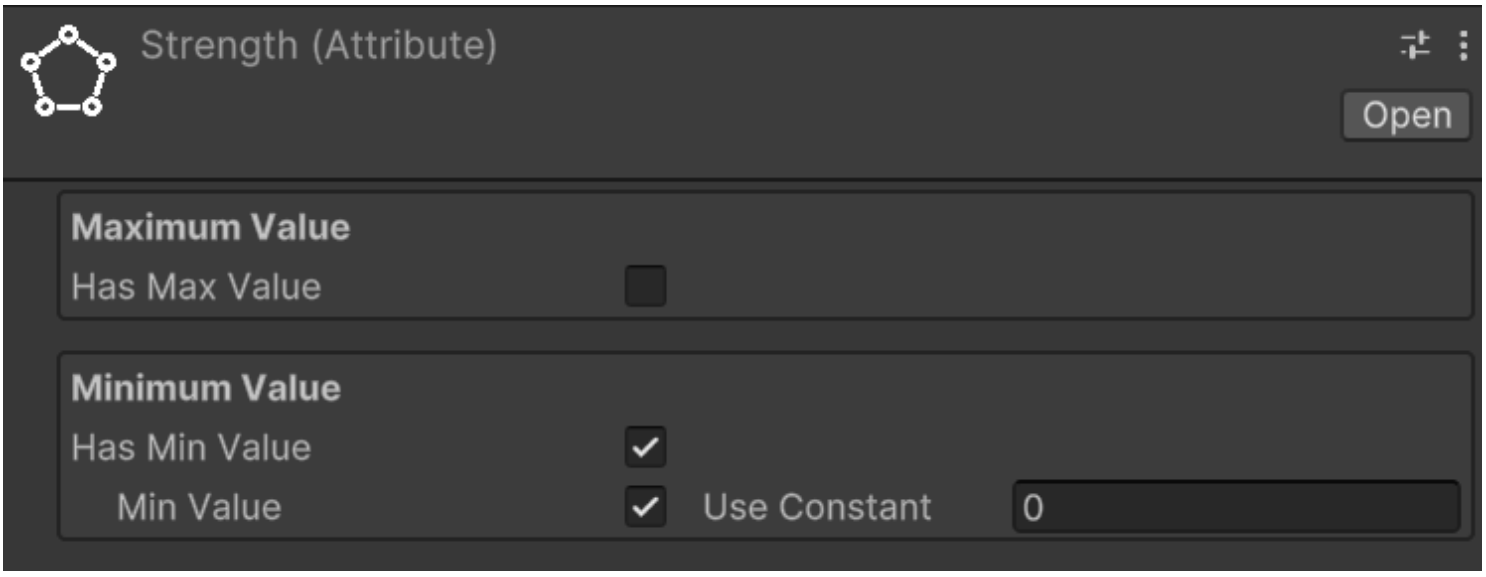
- Context menu: `Right click on the hierarchy > Create > Astra RPG Framework`
- Top bar: `Assets > Create > Astra RPG Framework`
- Hotkeys: By pressing the respective keyboard shortcut while a folder or an element of the hierarchy is currently selected

> **NOTE**
>
> For Mac users the `Ctrl` key corresponds to the `Cmd` key.

Create attributes

Keyboard shortcut: `Ctrl + Alt + A`
Relative path: `Attribute`

Once created a new attribute you can name it as you wish and you'll be able tweak some settings in the inspector. For example lets create a `Strength` attribute. Create an `Attributes` folder in your hierarchy, then press `A` and name the newly created attribute `Strength`.
In the inspector it should look like:



By checking `Has Max Value`, we will set a maximum value for the attribute. By default, there is no maximum value.

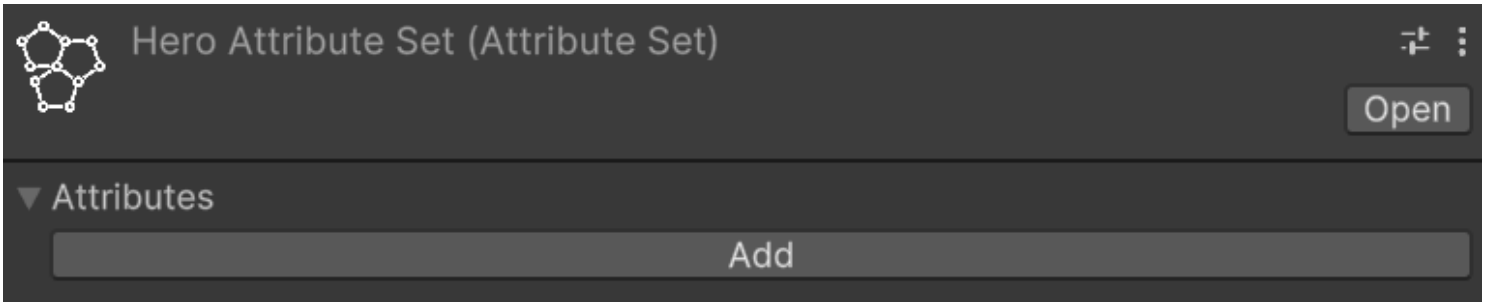
By checking `Has Min Value`, we will set a minimum value for the attribute. By default, the minimum value is zero.

Repeat the process for also the `Constitution`, `Intelligence`, and `Dexterity` attributes.

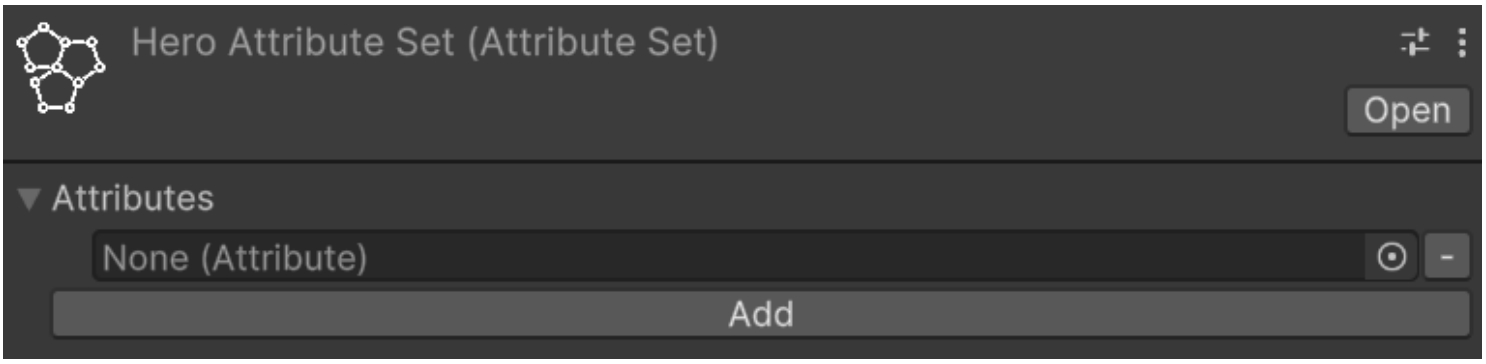
Create an attribute set

Relative path: `Attribute Set`

Now that we have some attributes let's create an `AttributeSet` named, for example, `Hero Attribute Set`. In the inspector it should look like this:



An attribute set without attributes isn't very useful, so let's add the previously created ones, one at a time. To do this, click on the `Add` button. Notice that an entry with `None (Attribute)` appears:

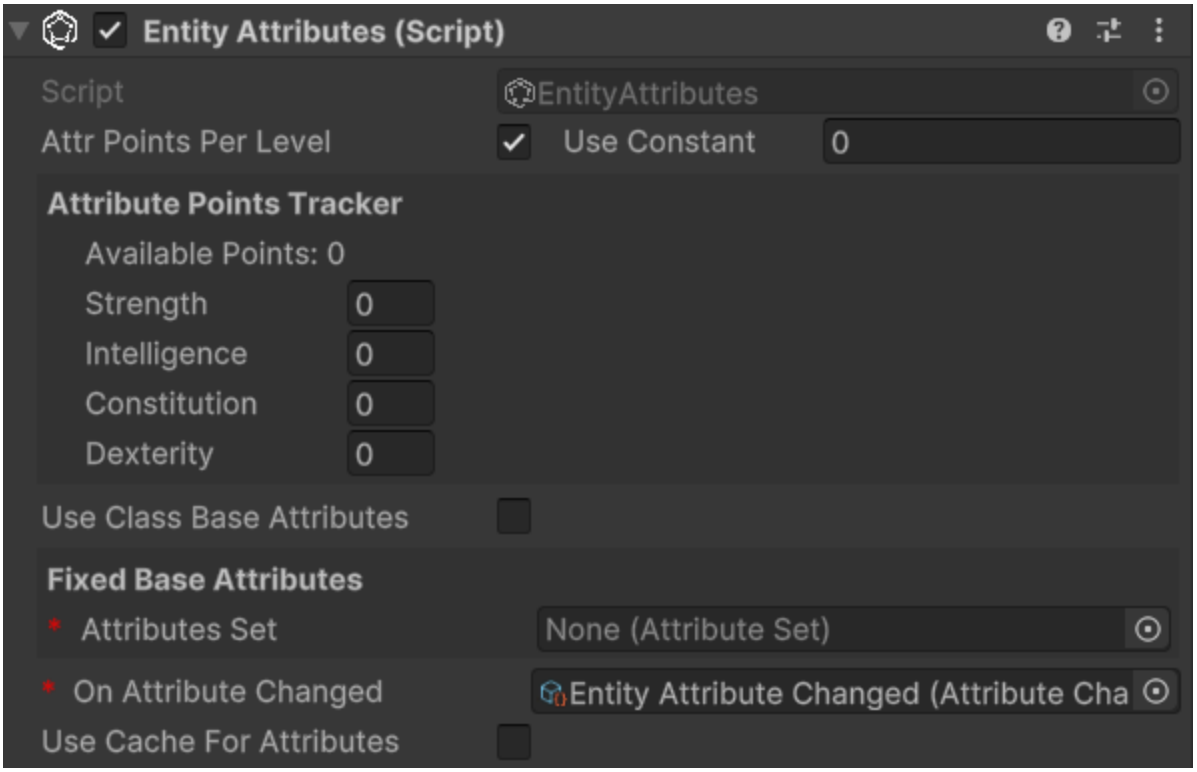


To assign an attribute to the entry, we can either drag & drop from the hierarchy or click on the small circle button on the right of the newly appeared entry. This mechanism is the same used for public variables or, more generally, for fields annotated with `SerializeField`, so it will be familiar to you.
Let's add `Strength` using whichever method you prefer. Repeat the process of adding an attribute to the set for `Constitution`, `Intelligence`, and `Dexterity` as well.

If you want to remove an attribute from the set, you can click on the small `-` button on the right of the attribute you want to remove.

Add `EntityAttributes` to an entity

The next step is to assign the attribute set we created to an entity. To do this, let's add the `EntityAttributes` component to our game object. The inspector will look like this:

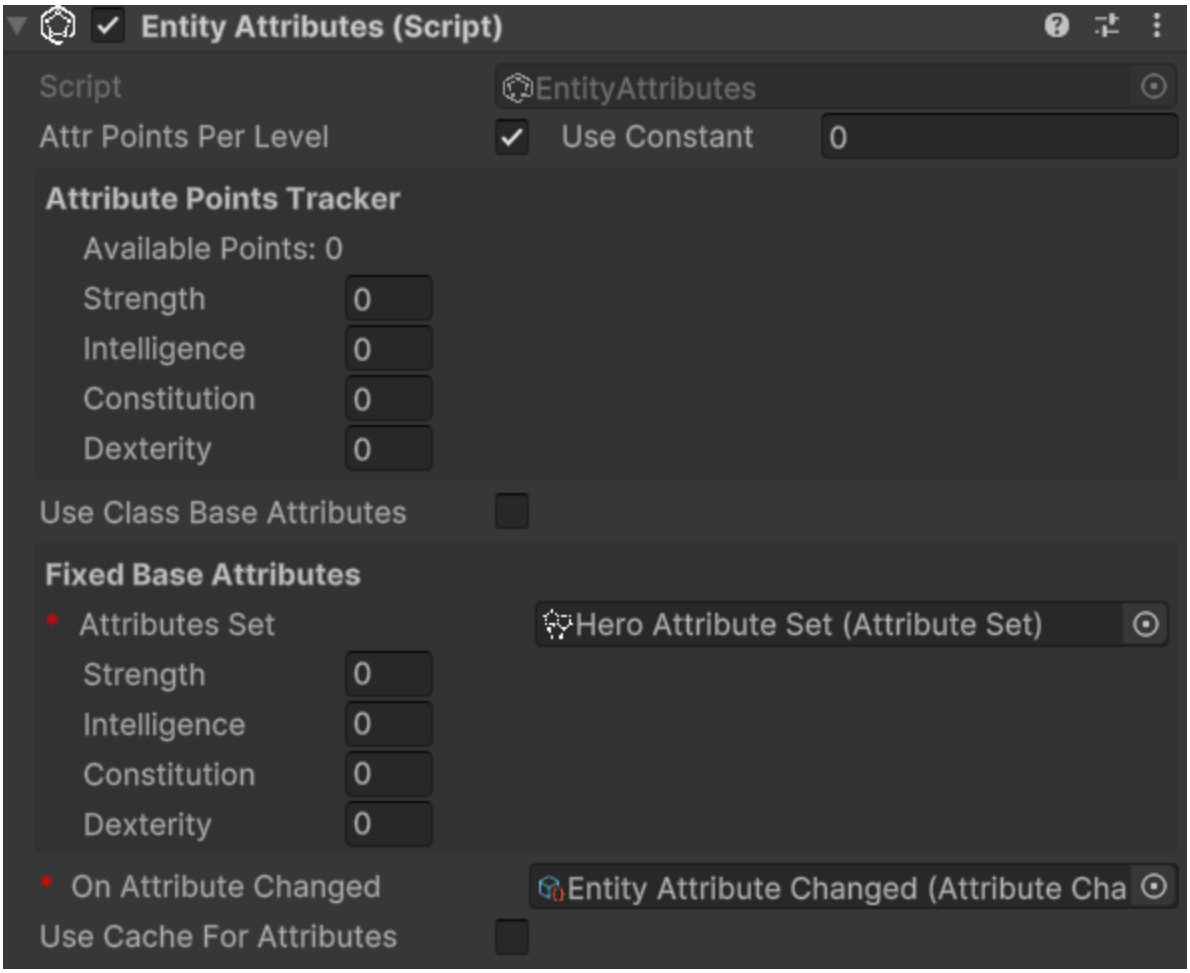


An entity has base points for attributes, which can be either fixed or derived from a class, a configurable amount of attribute points that can be arbitrarily assigned, and these points are granted at each level-up, along with flat and percentage modifiers for the attributes. Except for the modifiers, which can only be assigned via code, all other values can be configured from the inspector.

`Attr Points Per Level` defines how many arbitrarily spendable attribute points are provided at each level-up. They are assigned starting from level 2 on.

`Attribute Points Tracker` allows monitoring and assigning spendable points. `Available Points` defines how many unspent points are still available.
If you change the level of the entity you'll see that available points change accordingly. And as you spend them, `Available Points` will decrease.

Moreover, there is a checkbox labeled `Use Class Base Attributes`. For now, let's leave it unchecked since we haven't added a class yet. However, in this case, we need to manually assign an attribute set. Therefore, let's set the `Attribute Set` field found under `Fixed Base Attributes` with the `Hero Attribute Set`. By doing this, we now have access to additional fields in the inspector:



We can assign values to the attributes of `Fixed Base Attributes` as we see fit.

Understanding Attribute Modifier Types

The framework provides two distinct types of attribute modifiers that work together with spent attribute points to determine final attribute values. Understanding how each type works is essential for creating predictable character progression and balanced gameplay mechanics.

> **NOTE**
>
> General considerations for attribute modifiers
>
> - When adding modifiers through code, the `OnAttributeChanged` event will automatically be raised if the final value changes
> - If cache is being used: when adding modifiers through code, the attribute cache will automatically be invalidated to ensure the correct value is returned on the next access

Spent Attribute Points

Before diving into modifiers, it's important to understand that spent attribute points form the foundation of the attribute calculation system. These points are allocated by players during character progression and are applied immediately after the base value.

Characteristics:

- Player-controlled allocation of points earned through leveling
- Applied directly after base values in the calculation order
- Permanent increases (until points are redistributed)
- Each point provides a 1:1 increase to the attribute

Code example:

```
// Spend 3 points on Strength
entityAttributes.SpendOn(strengthAttribute, 3);

// Check available points before spending
int availablePoints = entityAttributes.AvailableAvailableAttributePoints;
if (availablePoints >= 2) {
    entityAttributes.SpendOn(constitutionAttribute, 2);
}
```

Flat Modifiers

Flat modifiers add or subtract a fixed amount to an attribute's value. They are applied after base values and spent points but before percentage modifiers.

Use cases:

- Equipment bonuses (e.g., +3 Strength from gauntlets)
- Temporary buffs (e.g., +5 Constitution from a fortitude potion)
- Race or class bonuses (e.g., Dwarves get +2 Constitution)
- Status effects that provide fixed bonuses or penalties
- Environmental effects (e.g., library affecting Intelligence)

Code example:

```
// Add a flat +4 bonus to Strength
entityAttributes.AddFlatModifier(strengthAttribute, 4);

// Add a flat -2 penalty to Dexterity (negative values work too)
entityAttributes.AddFlatModifier(dexterityAttribute, -2);
```

Calculation example:

- Base Strength: 12
- Spent points: +3
- Flat modifier: +4
- Result after flat modifiers: 12 + 3 + 4 = 19

Percentage Modifiers

Percentage modifiers apply a multiplicative increase or decrease to the current attribute value. They are the most powerful type of modifier and are applied last in the calculation chain, after all other values have been calculated.

Use cases:

- Powerful equipment bonuses (e.g., +20% to all attributes)
- Character traits or talents (e.g., "Natural Athlete: +15% Strength and Dexterity")
- Temporary powerful buffs or curses
- Class features
- Magical enchantments or artifacts

Code example:

```
// Add a 20% increase to Strength
entityAttributes.AddPercentageModifier(strengthAttribute, 20);

// Add a 10% decrease to Intelligence (negative percentage)
entityAttributes.AddPercentageModifier(intelligenceAttribute, -10)
```

Calculation example:

- Previous value: 19 (from previous steps)
- Percentage modifier: +20% = 19 × 0.20 = 3.8
- Final result: 19 + 3.8 = 22.8 (rounded to 23 for integer attributes)

Important notes:

- Multiple percentage modifiers are additive before being applied (e.g., +20% and +10% = +30% total)
- The percentage is calculated based on the value after base, spent points, and flat modifiers
- Percentage modifiers can be negative to create penalties

Complete Calculation Example

Let's see a complete example showing the full attribute calculation process:

Initial setup:

- Base Intelligence: 14
- Points spent on Intelligence: 4
- Equipment flat bonus: +2 (from a circlet)
- Trait percentage bonus: +25% (from "Scholar" trait)

Step-by-step calculation:

1. Start with base: 14
2. Add spent points: 14 + 4 = 18
3. Apply flat modifiers: 18 + 2 = 20
4. Apply percentage modifiers: 20 + (20 × 0.25) = 20 + 5 = 25

Final Intelligence value: 25

Comparison with Stat Modifiers

Unlike stats, attributes have a simpler modifier system:

Attributes have:

- Base values
- Spent attribute points (player-controlled)
- Flat modifiers
- Percentage modifiers

Stats additionally have:

- Stat-to-stat modifiers (attributes don't have attribute-to-attribute modifiers)
- More complex scaling relationships
- Stats can scale upon attributes

This simplicity makes attributes more predictable and easier for players to understand, while stats can have more complex interactions. Keeping them simple helps maintain clarity in gameplay

Retrieving Attribute Values from code

Concrete components such as `EntityAttributes` expose convenience methods like `Get` and `GetBase`, but for reusable or defensive code the preferred read-only contract is `IAttributeReader`.

```
IAttributeReader attributeReader = entity;

if (attributeReader.TryGet(strengthAttribute, out long currentStrength) &&
    attributeReader.TryGetBase(strengthAttribute, out long baseStrength))
```

```
{
    Debug.Log($"STR = {currentStrength} (base {baseStrength})");
}
```

`EntityCore` implements `IAttributeReader`, so many gameplay systems can read values through the entity itself without depending directly on `EntityAttributes`. See Advanced topics for the broader reader and rounding APIs.

Spending attribute points

If the entity's `Attr Points Per Level` is greater than zero and the level is greater than 1, we can spend attribute points on the attributes. To do this, we can use the `SpendOn` method:

```
// strengthAttribute is a reference to the Strength Attribute
entityAttributes.SpendOn(strengthAttribute, 2);
```

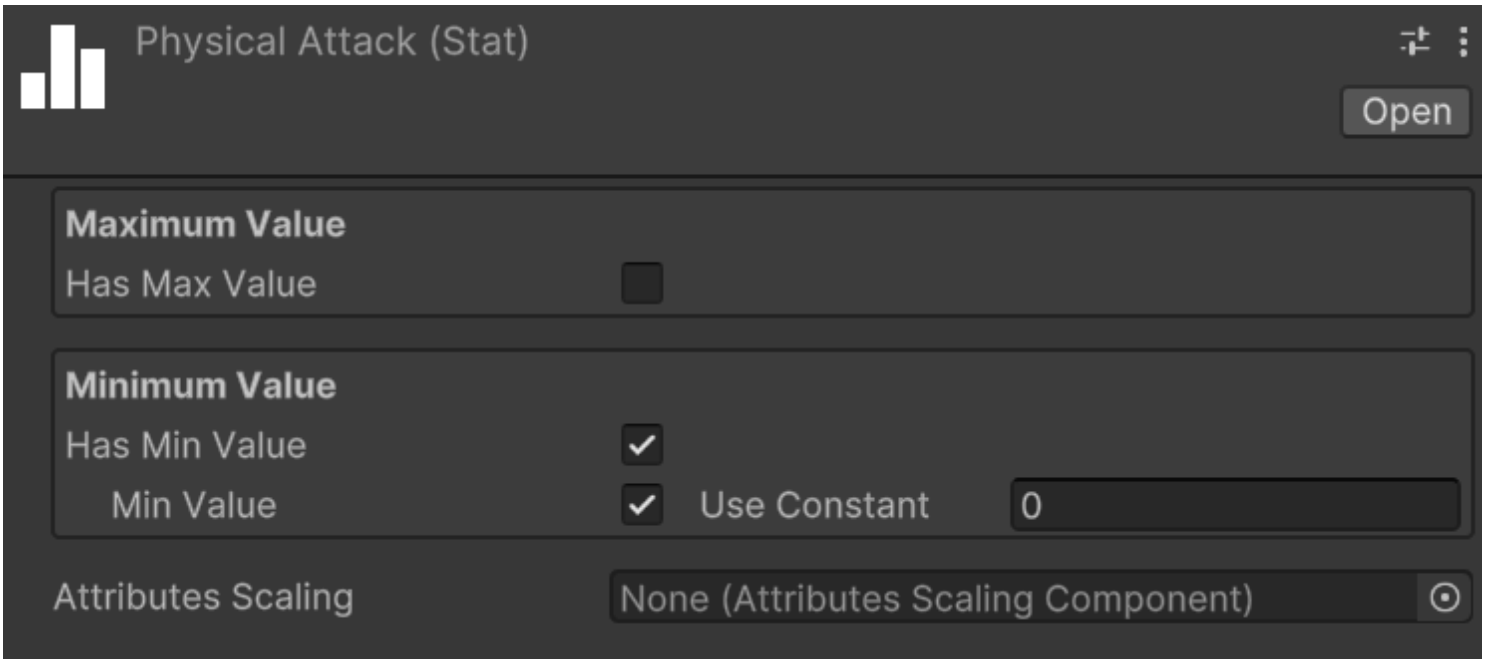
This will spend 2 points on the `Strength` attribute, increasing its value by 2. If there are not enough available points, a Debug.LogError will be raised.

> **NOTE**
> Debug.LogError messages are shown only in development builds. If you run a production build, you won't see them. This is useful to avoid cluttering the console with error messages that are not relevant in production.

Create stats

Keyboard shortcut: `Ctrl + Alt + S`
Relative path: `Stat`

As with attributes, you can create stats as you wish and assign them the names you prefer. Let's create the `Physical Attack` stat together. Create a new `Stats` folder, select it and press `S`. Name it `Physical Attack`. In the inspector, it should look like this:



As with attributes, you can assign both a maximum and a minimum value to a stat.

Repeat the process for the `Magical Power`, `Defense`, and `Critical Chance` stats.

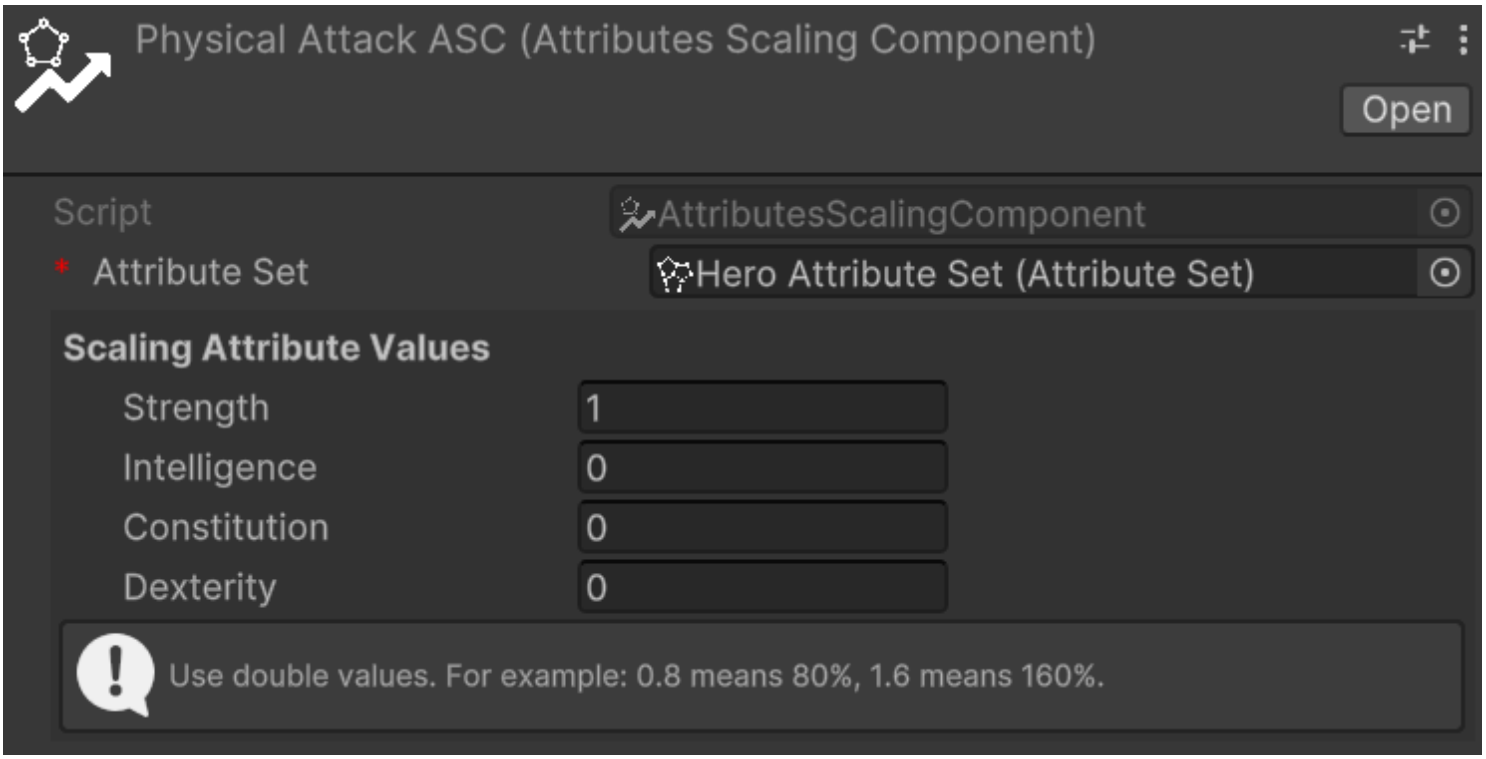
Unlike attributes, however, stats include `Attributes Scaling`.

Create an Attribute Scaling Component for Stats

Relative path: `Scaling -> Attribute Scaling Component`

Let's create a new `Attribute Scaling Component` to use with the strength stat we created earlier. Create a new folder named, for example, `Attribute Scalings for Stats`, and inside it, create an attribute scaling component called `Physical Attack Strength Scaling`.

Assign the previously created `Hero Attribute Set` to the `Set` field. You will see the attributes of the set appear. Here, you can assign scaling values using `double`. For example, set the scaling of `Strength` to `1.0`. This component defines a 100% scaling on the value of `Strength`.



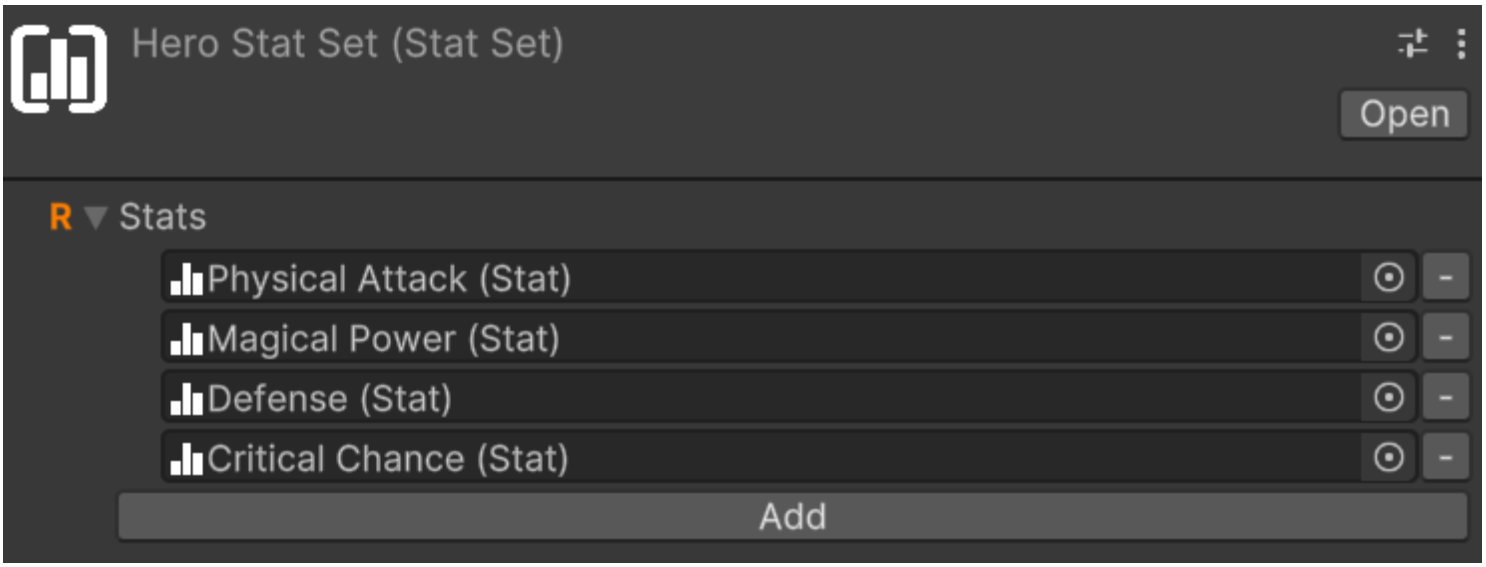
Now, assign this scaling component to the `Physical Attack` stat to ensure it scales with the `Strength` attribute.

Create a stat set

Relative path: `Stat Set`

Now that we have some stats, let's create a `StatSet` named, for example, `Hero Stat Set`.

A stat set without stats isn't very useful, so let's add the previously created ones, one at a time. To do this, click on the `Add` button. Notice that an entry with `None (Stat)` appears. To assign a stat to the entry, we can either drag & drop from the hierarchy or click on the small circle button on the right of the newly appeared entry. This mechanism is the same used for public variables or, more generally, for fields annotated with `SerializeField`, so it will be familiar to you.
Let's add `Physical Attack` using whichever method you prefer.
Repeat the process of adding a stat to the set for `Magical Power`, `Defense`, and `Critical Chance` as well.
The stat set should look like:



If you want to remove a stat from the set, you can click on the small `-` button on the right of the stat you want to remove.

Modular stat sets

Stat sets can include other stat sets, allowing for modular and reusable configurations. This is particularly useful if we have various kind of entities that share some stats but not all of them. For example, let's consider three entities: a deer, a ballista turret, and our hero character. The deer can take damage, move around, and cannot attack. The turret instead can take damage, deal damage, but cannot move. The hero can do all three things.

A first approach could be to create three distinct stat sets for each entity, but this would lead to a lot of redundancy since many stats would be repeated across the three sets. What if we decide to add a new stat that all three entities should have? We would have to remember to add it to all three sets, which is error-prone and inefficient. Alternatively, we could use a single all-embracing stat set that includes all the stats needed by all entities. However, this would lead to unnecessary complexity for entities that don't need all those stats, making it harder to manage and understand.

A better approach is to create modular stat sets that can be combined as needed. We can create three stat sets:

- `Damageable Stat Set`: includes stats like `Armor`, `Magical Defense`, `Dmg Reduction`, and so on
- `Damage Dealer Stat Set`: includes stats like `Physical Attack`, `Magical Power`, `Critical Chance`, and so on
- `Movable Stat Set`: includes stats like `Movement Speed`, `Jump Height`, etc.

Then, we can create three additional stat sets for our entities:

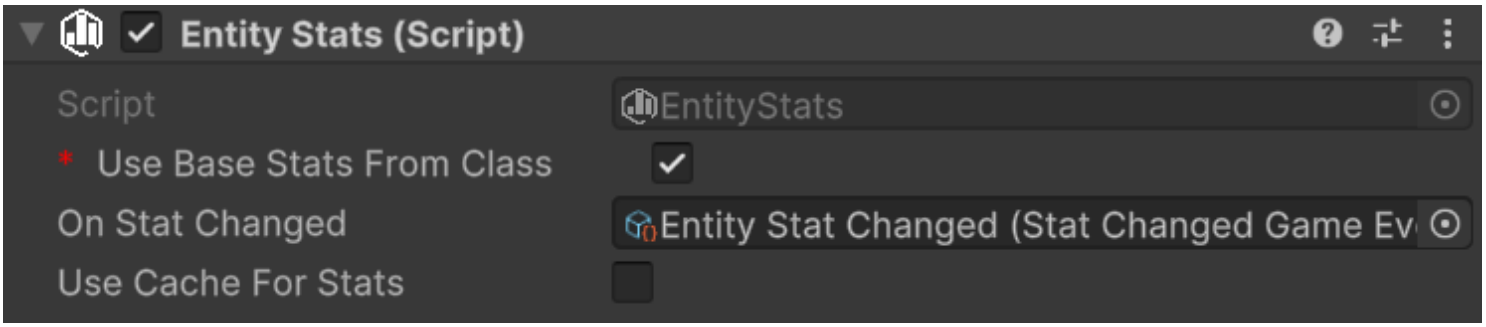
- `Prey Stat Set` for the deer, which includes only the `Damageable Stat Set` and the `Movable Stat Set`

- `Turret Stat Set` for the ballista turret, which includes only the `Damageable Stat Set` and the `Damage Dealer Stat Set`
- `Hero Stat Set` for our hero character, which includes all three stat sets: `Damageable Stat Set`, `Damage Dealer Stat Set`, and `Movable Stat Set`

This modular approach allows us to reuse stat configurations across different entities, reducing redundancy and making it easier to manage and update stats.

Add `EntityStats` to an Entity

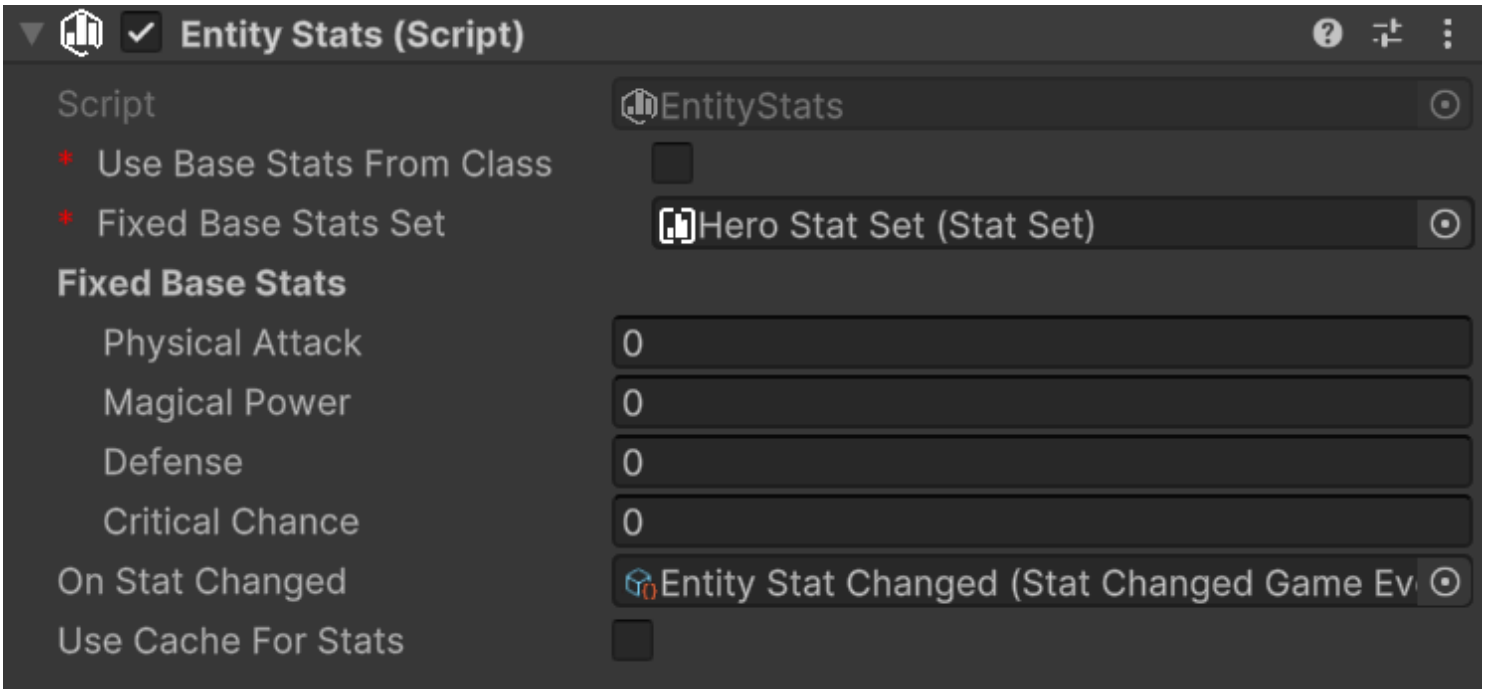
The next step is to assign the stat set we created to an entity. To do this, let's add the `EntityStats` component to our game object. The inspector will look like this:



An entity has base stats that can be either fixed or derived from a class. Additionally, stats can be modified through flat modifiers, stat-to-stat modifiers, and percentage modifiers.

`Use Class Base Stats` checkbox determines whether the base stats should come from the entity's class (if one is available) or from fixed values defined in the inspector. For now, let's leave it unchecked since we haven't added a class yet.

With `Use Class Base Stats` unchecked, we need to manually assign a stat set. Set the `Stat Set` field under `Fixed Base Stats` with our `Hero Stat Set`. This will reveal additional fields in the inspector where we can set the base values for each stat:



`On Stat Changed` event gets raised whenever any stat value changes due to modifiers. You can use this to update UI elements or trigger other game logic.

`Use Cache` enables caching of final stat values. This is useful for performance when you have many entities or complex stat calculations.

Understanding Stat Modifier Types

The framework provides three distinct types of stat modifiers, each serving different purposes and applied in a specific order during final value calculation. Understanding how each type works is crucial for creating balanced and predictable stat systems.

> **NOTE**
>
> **General considerations for stat modifiers**
>
> - If cache is being used: when adding modifiers through code, the attribute cache will automatically be invalidated to ensure the correct value is returned on the next access
> - When adding modifiers through code, the `OnStatChanged` event will automatically be raised if the final value changes.

Flat Modifiers

Flat modifiers add or subtract a fixed amount to a stat's value. They are the simplest type of modifier and are applied directly after the base value.

Use cases:

- Equipment bonuses (e.g., +5 Physical Attack from a sword)
- Temporary buffs (e.g., +10 Defense from a shield spell)
- Status effects that provide fixed bonuses or penalties

Code example:

```
// Add a flat +15 bonus to Physical Attack
entityStats.AddFlatModifier(physicalAttackStat, 15);

// Add a flat -5 penalty to Defense (negative values work too)
entityStats.AddFlatModifier(defenseStat, -5);
```

Calculation example:

- Base Physical Attack: 50
- Flat modifier: +15
- Result after flat modifiers: 50 + 15 = 65

Stat-to-Stat Modifiers

Stat-to-stat modifiers allow one stat to contribute a percentage of its value to another stat. This creates interesting dependencies between different stats and allows for more complex character builds.

These modifiers are applied right after the flat modifiers.

Use cases:

- Cross-stat synergies (e.g., 25% of Armor is added to Physical Attack).

> ⚠ **WARNING**
>
> When using stat-to-stat modifiers, **only the base value and flat modifiers of the source stat are used** in the calculation. Stat-to-stat modifiers and percentage modifiers applied to the source stat are ignored. This ensures predictable and non-circular calculations.

Code example:

```
// 25% of armor is added to physical attack
entityStats.AddStatToStatModifer(physicalAttackStat, armorAttribute, 25);

// Negative modifier: -10% of armor is subtracted from physical attack
entityStats.AddStatToStatModifer(physicalAttackStat, armorAttribute, -10);
```

Calculation example:

- Base Physical Attack: 50
- Flat modifier: +15 (from previous step)
- Current value: 65
- Armor value: 40
- Stat-to-stat modifier: 50% of Armor = 40 × 0.5 = 20
- Result after stat-to-stat modifiers: 65 + 20 = 85

Important notes:

- The source stat's base value + its flat modifiers is used for calculation
- Multiple stat-to-stat modifiers from different sources are additive, and order of calculation is commutative
- You can have the same source stat contribute to multiple target stats
- Circular dependencies can be defined, as the base values + flat modifiers are used for calculations

Percentage Modifiers

Percentage modifiers apply a multiplicative increase or decrease to the current stat value. They are applied last in the calculation chain. Because of this, they can have a significant impact on the final value.

Use cases:

- Powerful equipment bonuses (e.g., +25% damage increase)
- Character traits or talents (e.g., "Warrior's Might: +20% Physical Attack")
- Temporary powerful buffs or debuffs

Code example:

```
// Add a 25% increase to Physical Attack
entityStats.AddPercentageModifier(physicalAttack, 25);

// Add a 15% decrease to movement speed (negative percentage)
entityStats.AddPercentageModifier(movementSpeed, -15);
```

Calculation example:

- Previous value: 85 (from previous steps)
- Percentage modifier: +25% = 85 × 0.25 = 21.25
- Final result: 85 + 21.25 = 106.25 (rounded to 106 for integer stats)

Important notes:

- Multiple percentage modifiers are additive before being applied (e.g., +25% and +15% = +40% total)
- The percentage is calculated based on the value after flat and stat-to-stat modifiers
- Percentage modifiers can be negative to create penalties

Complete Calculation Example

Let's see a complete example with all three modifier types:

Initial setup:

- Base Physical Attack: 100
- Armor value: 60 (base + flat modifiers)

Applied modifiers:

1. Flat modifier: +20 (from weapon)
2. Stat-to-stat modifier: 75% of Armor = 60 × 0.75 = 45
3. Percentage modifier: +30% (from various sources)

Step-by-step calculation:

1. Start with base: 100
2. Apply flat modifiers: 100 + 20 = 120
3. Apply stat-to-stat modifiers: 120 + 45 = 165
4. Apply percentage modifiers: 165 + (165 × 0.30) = 165 + 49.5 = 214.5 → 214

Final Physical Attack value: 214

> **NOTE**
> The framework does not provide a built-in tool for removing applied modifiers. It is up to you to define your own abstraction for buffs, debuffs, or other temporary effects that add and remove modifiers as needed. In the future, the Astra RPG Modifiers extension for this framework will be released, which will include such abstractions thought to integrate seamlessly with the existing systems. Check the status of the extension for more details at https://electricdrill.github.io/

Retrieving Stat Values from code

Concrete components such as `EntityStats` expose convenience methods like `Get` and `GetBase`, but for reusable or defensive code the preferred read-only contract is `IStatReader`.

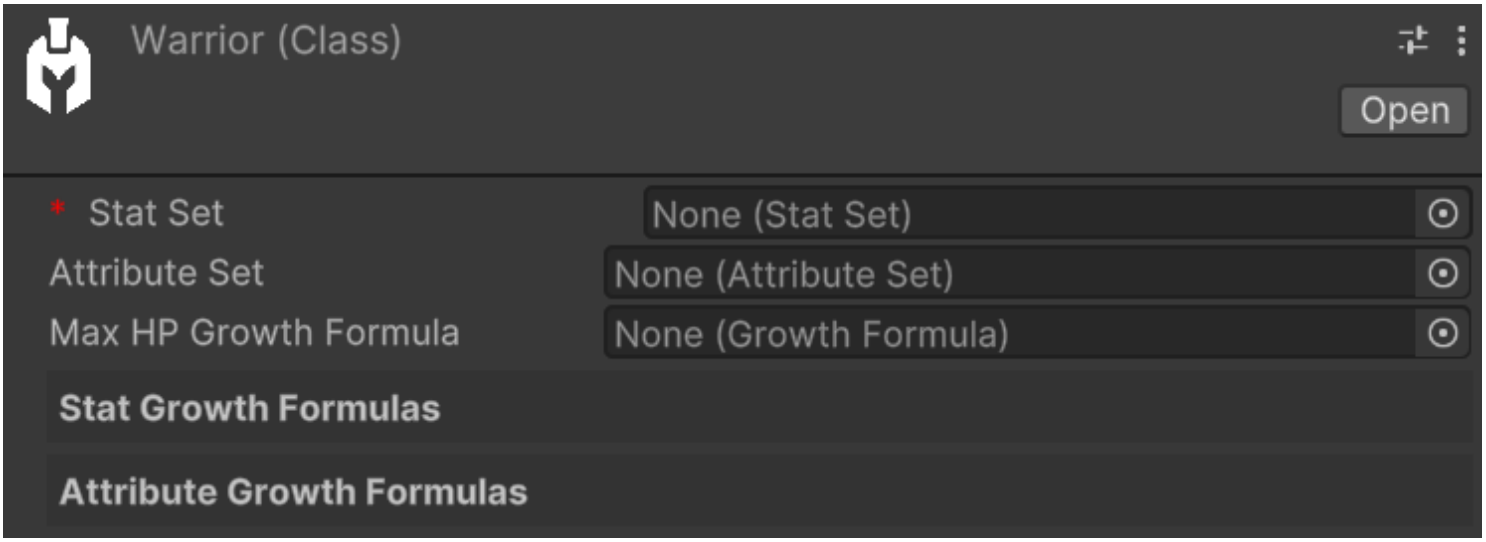
```
IStatReader statReader = entity;

if (statReader.TryGet(phyAtkStat, out long physicalAttack) &&
    statReader.TryGetBase(phyAtkStat, out long basePhysicalAttack))
{
    Debug.Log($"Physical Attack = {physicalAttack} (base {basePhysicalAttack})");
}
```

`EntityCore` implements `IStatReader`, so systems that already work with the entity can query stats without reaching into a specific `EntityStats` component. See Advanced topics for more on readers, `TryGetBase`, and related APIs.

Create a class

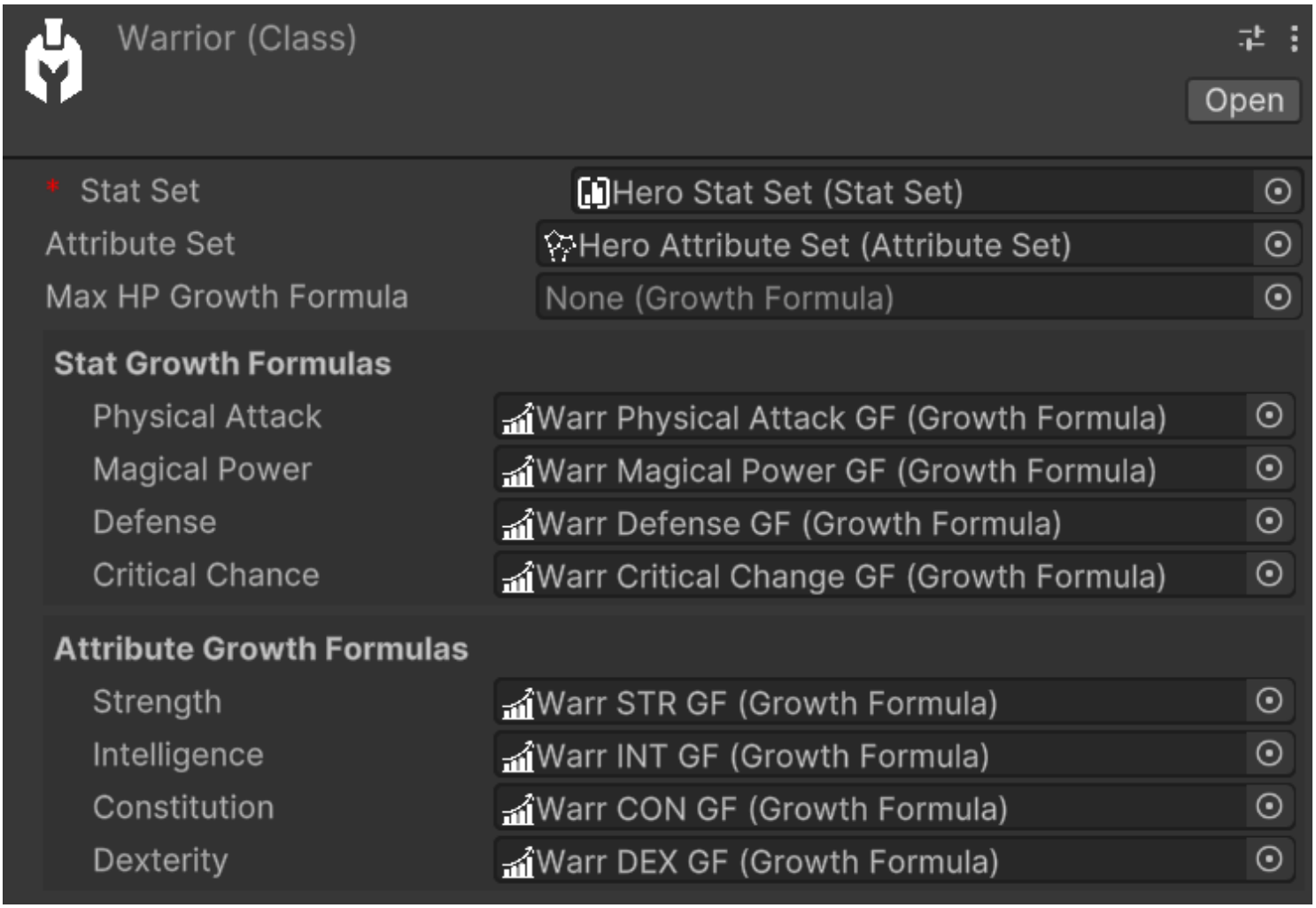
Relative path: `Class`

Let's create an instance of `Class` called `Warrior`. It should appear like this:



The only mandatory field is `Stat Set`. If we don't make use of attributes and Max HP, we can leave the `Attribute Set` and `Max HP Growth Formula` fields empty.

In our case, let's assign our `Hero Stat Set` to `Stat Set` and `Hero Attribute Set` to `Attribute Set`. This way, the `Warrior` will have access to all stats and attributes from the assigned `Stat Set` and `Attribute Set`. As we fill these two fields, we'll see that the `Stat Growth Formulas` and `Attribute Growth Formulas` sections will automatically populate with the stats and attributes from the assigned `Stat Set` and `Attribute Set`. Let's proceed to create all the growth formulas for the warrior's stats and attributes. Follow the steps outlined in the Growth Formulas section to create the growth formulas for the warrior's stats and attributes.
Once all growth formulas are assigned, the `Warrior` should look like this:

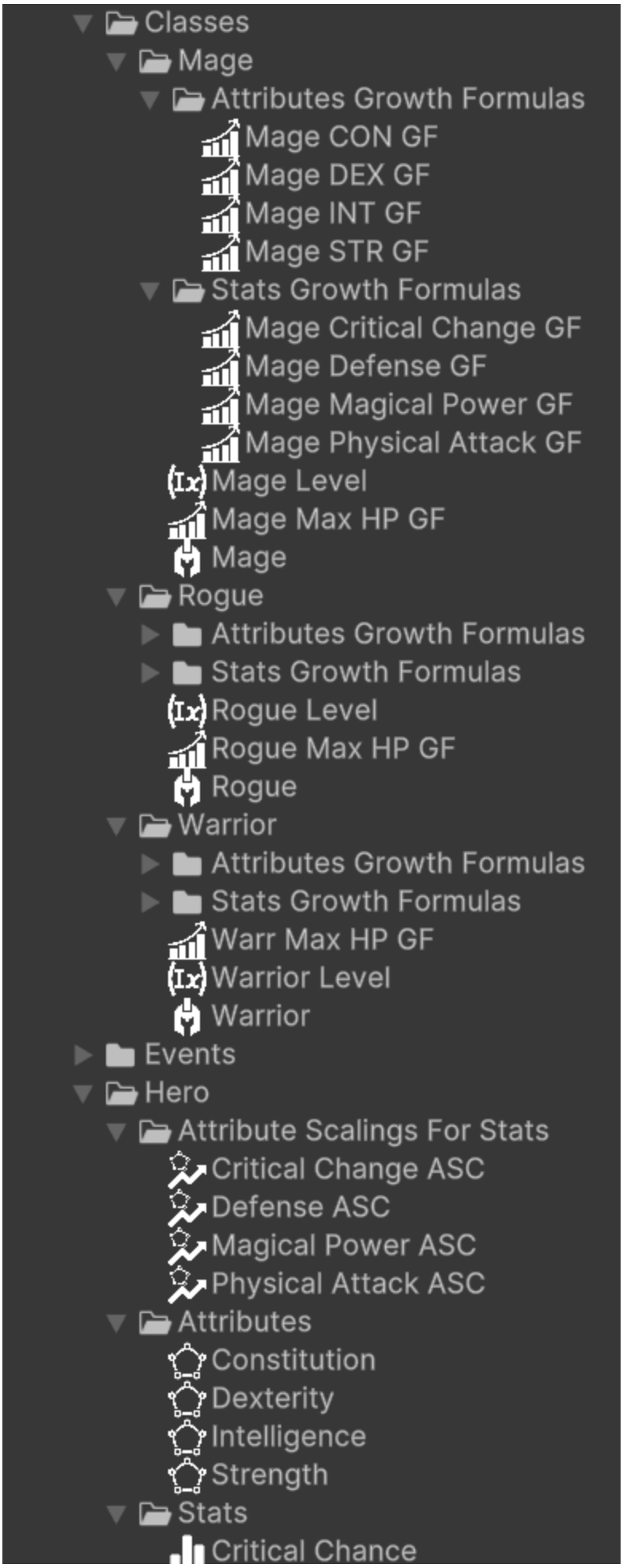


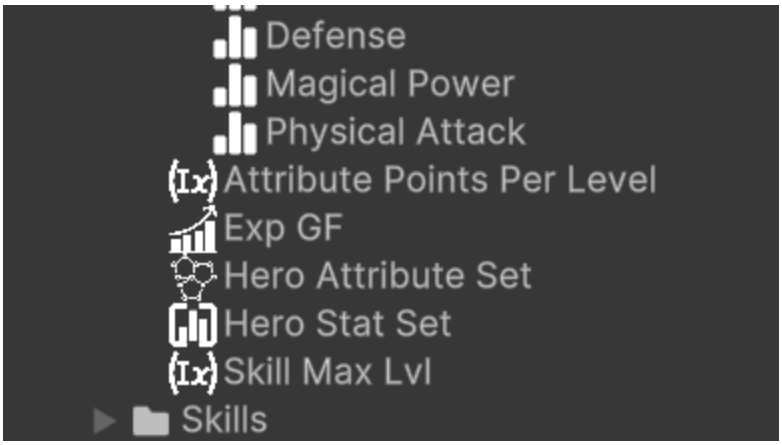
`Max HP Growth Formula` allows specifying how the Max HP value grows as levels change. In our example, we'll leave it empty. The presence of this field for hit points might be surprising since this module of the framework isn't focused on health management. Indeed, damage and health are managed by the *Astra RPG Health* module, which will be released in the coming months. However, this field is positioned here since the scaling of base max hp still depends on the class.

Keeping the hierarchy clean

By now you should have a lot of assets in your hierarchy. To keep it clean, you can create a folder named `Classes` and move the `Warrior` class inside it. You can do the same for the `Attributes` and `Stats` growth formulas inside the `Warrior` folder. This way, you can keep the hierarchy organized and easily find the assets.
Similarly, the `Hero Stat Set` and `Hero Attribute Set` could be placed in a `Hero` folder, that is common to all the classes. This way, you can have a single set of stats and attributes for all the classes that will be created in the future.

This is how your hierarchy could look like:

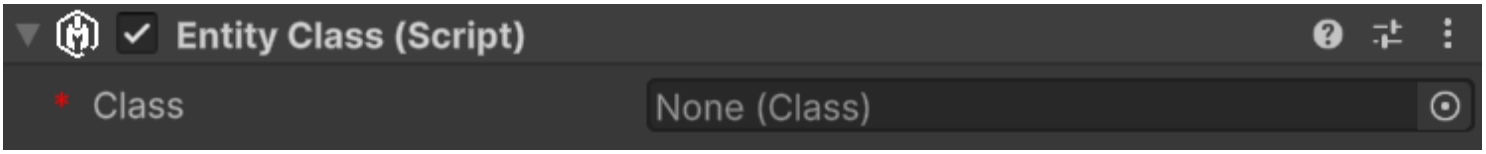

Classes
Mage
Attributes Growth Formulas
Mage CON GF
Mage DEX GF
Mage INT GF
Mage STR GF
Stats Growth Formulas
Mage Critical Change GF
Mage Defense GF
Mage Magical Power GF
Mage Physical Attack GF
Mage Level
Mage Max HP GF
Mage
Rogue
Attributes Growth Formulas
Stats Growth Formulas
Rogue Level
Rogue Max HP GF
Rogue
Warrior
Attributes Growth Formulas
Stats Growth Formulas
Warr Max HP GF
Warrior Level
Warrior
Events
Hero
Attribute Scalings For Stats
Critical Change ASC
Defense ASC
Magical Power ASC
Physical Attack ASC
Attributes
Constitution
Dexterity
Intelligence
Strength
Stats
Critical Chance



Obviously this is just a possible organization of the assets. Feel free to organize it as you prefer.

Add `EntityClass` to an entity

To assign a class to an entity, we need to add the `EntityClass` component to it. The inspector will look like this:



All we have to do now is just assign the `Warrior` class we created earlier to the `Class` field.

Switching to class-based attributes and stats

We can now check the `Use Class Base Attributes` and `Use Class Base Stats` checkboxes. By doing this, the entity will use the base attributes and stats defined by the class. The `Fixed Base Attributes` and `Fixed Base Stats` fields will be disabled, and the values will be automatically retrieved from the class growth formulas.

Retrieving class-based values from code

With a class we can access:

- attributes values: `GetAttributeAt(Attribute attribute, int level)`
- stats values: `GetStatAt(Stat stat, int level)`
- Max HP values: `GetMaxHpAt(int level)` For example:

```
// hero is a reference to the EntityCore component of the hero
EntityClass warriorClass = hero.GetComponent<EntityClass>();
// hero.Level is of EntityLevel type, but it is implicitly be converted to an int
int level = hero.Level;
int strengthAtLevel5 = warriorClass.GetAttributeAt(strengthAttribute, level);
```

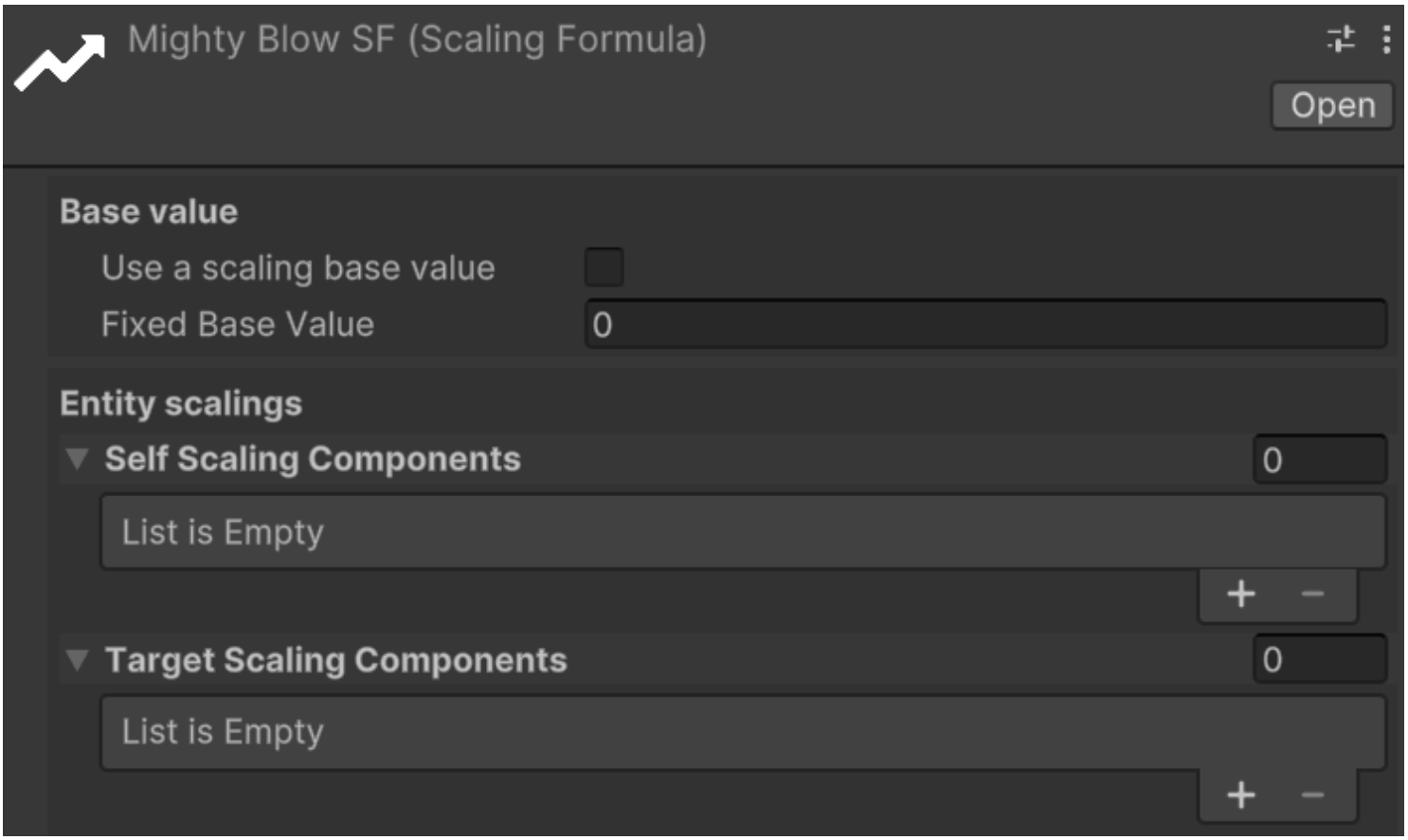
```
int physicalAttackAtLevel5 = warriorClass.GetStatAt(phyAtkStat, level);
int maxHpAtLevel5 = warriorClass.GetMaxHpAt(level);
```

Create Scaling Formulas

Keyboard shortcut: `Alt + Shift + S`
Relative path: `Scaling -> Scaling Formula`

We already saw how to create an `Attribute Scaling Component` for stats. On top of such usage, scaling components, and more in general scaling formulas, can be used for much more situations. For example, they can be used to define the damage of an ability, to define the bonus granted by a piece of equipment, or to define the damage of a weapon. In general, they can be used to define any kind of scaling that can be expressed as a function of one or more variables.

For example, let's create a `Scaling Formula` called `Mighty Blow SF`. It should look like this in the inspector:

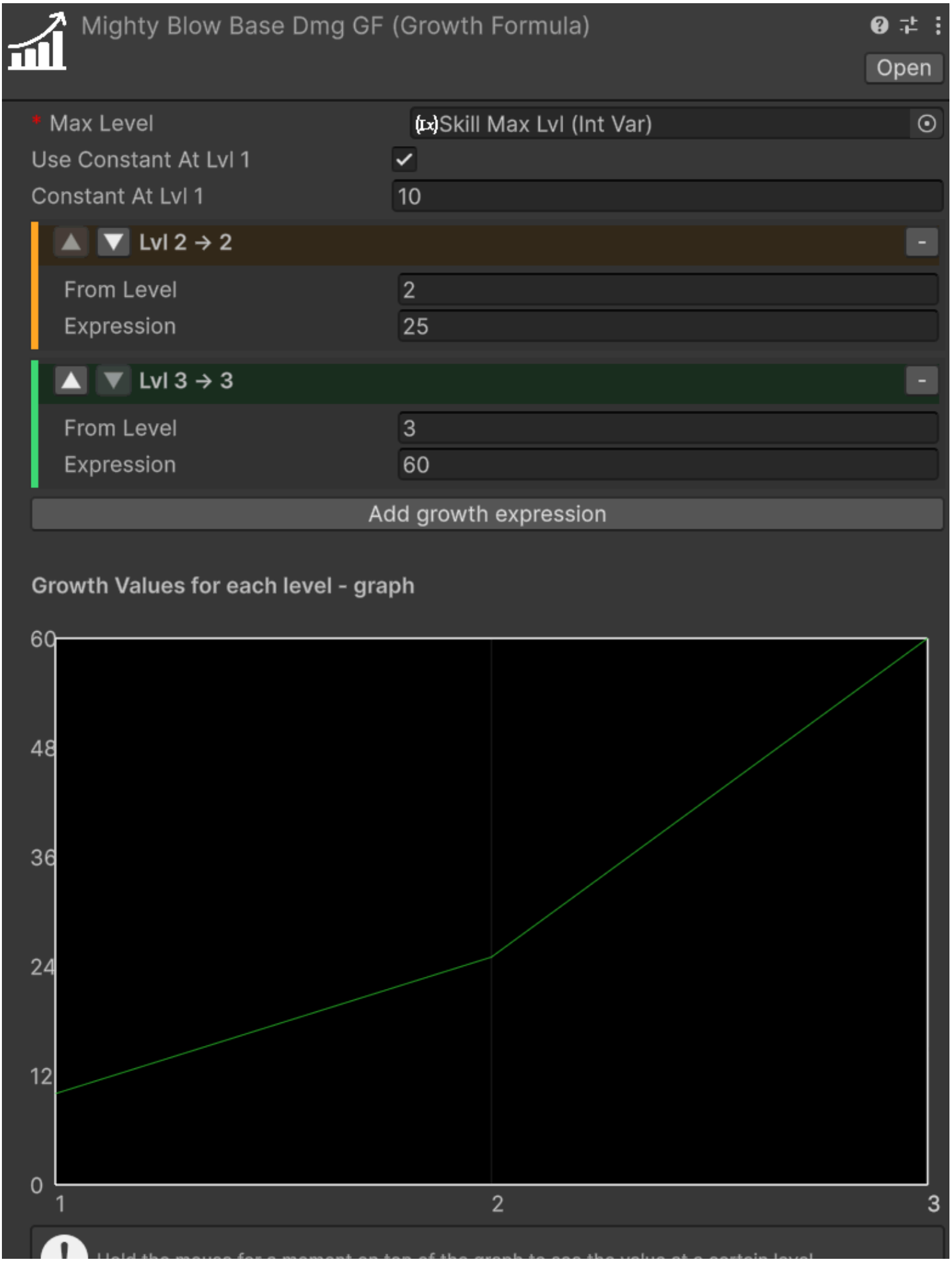


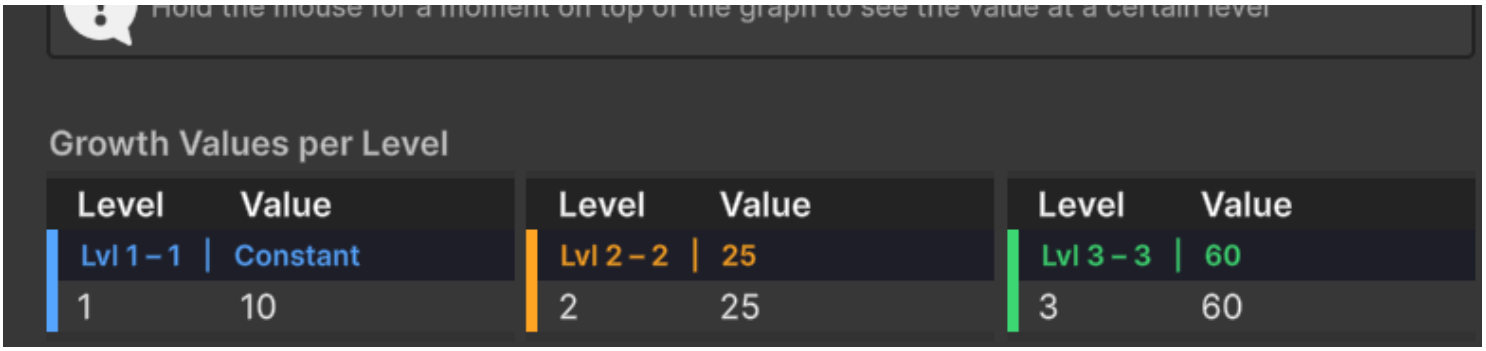
`Base Value` determines the starting point for the scaling formula. It can either be a fixed constant value or a value that scales with levels (e.g., the level of the Mighty Blow skill). If the latter is chosen, a `Growth Formula` must be provided to define how the base value changes as levels increase.

This scaling formula will be used to define the damage of a skill called `Mighty Blow`. The scaling formula will be defined as follows:

- Base damage: 10 at lvl 1, 25 at level 2, 60 at lvl 3
- Damage scaling: 1.5 * Physical Attack + 0.5 * Constitution

Since we want a base value that varies as level grows, let's check the `Use a scaling base value` checkbox and create a `Growth Formula` named `Mighty Blow Base Dmg GF`. The `Mighty Blow Base Damage GF` should look like this:


Mighty Blow Base Dmg GF (Growth Formula)
Open
* Max Level
Skill Max Lvl (Int Var)
Use Constant At Lvl 1
Constant At Lvl 1
10
Lvl 2 → 2
From Level
2
Expression
25
Lvl 3 → 3
From Level
3
Expression
60
Add growth expression
Growth Values for each level - graph
60
48
36
24
12
0
1
2
3



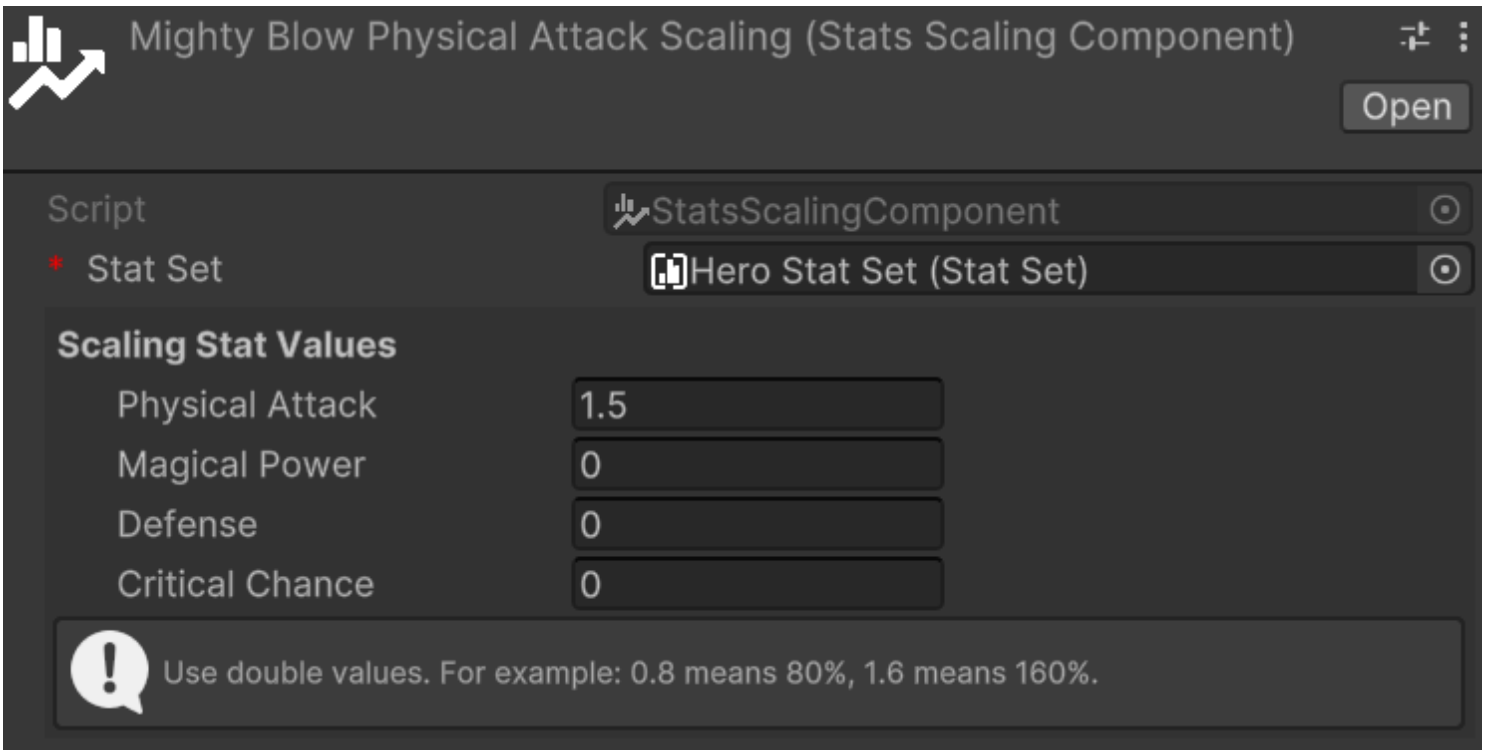
Notice that a new Skill Max Lvl has been created and assigned to `Max Level`. This is necessary as the skill max level is not related to the max level of our hero.

We can now assign this growth formula to the `Base Value` field of the `Mighty Blow SF` scaling formula.

Under `Entity Scalings` we have `Self Scaling Components` and `Target Scaling Components`. The former are used to define the scaling of the entity itself, while the latter are used to define the scaling of the target of the ability. In our case, we will only use `Self Scaling Components`, so we can leave `Target Scaling Components` empty.

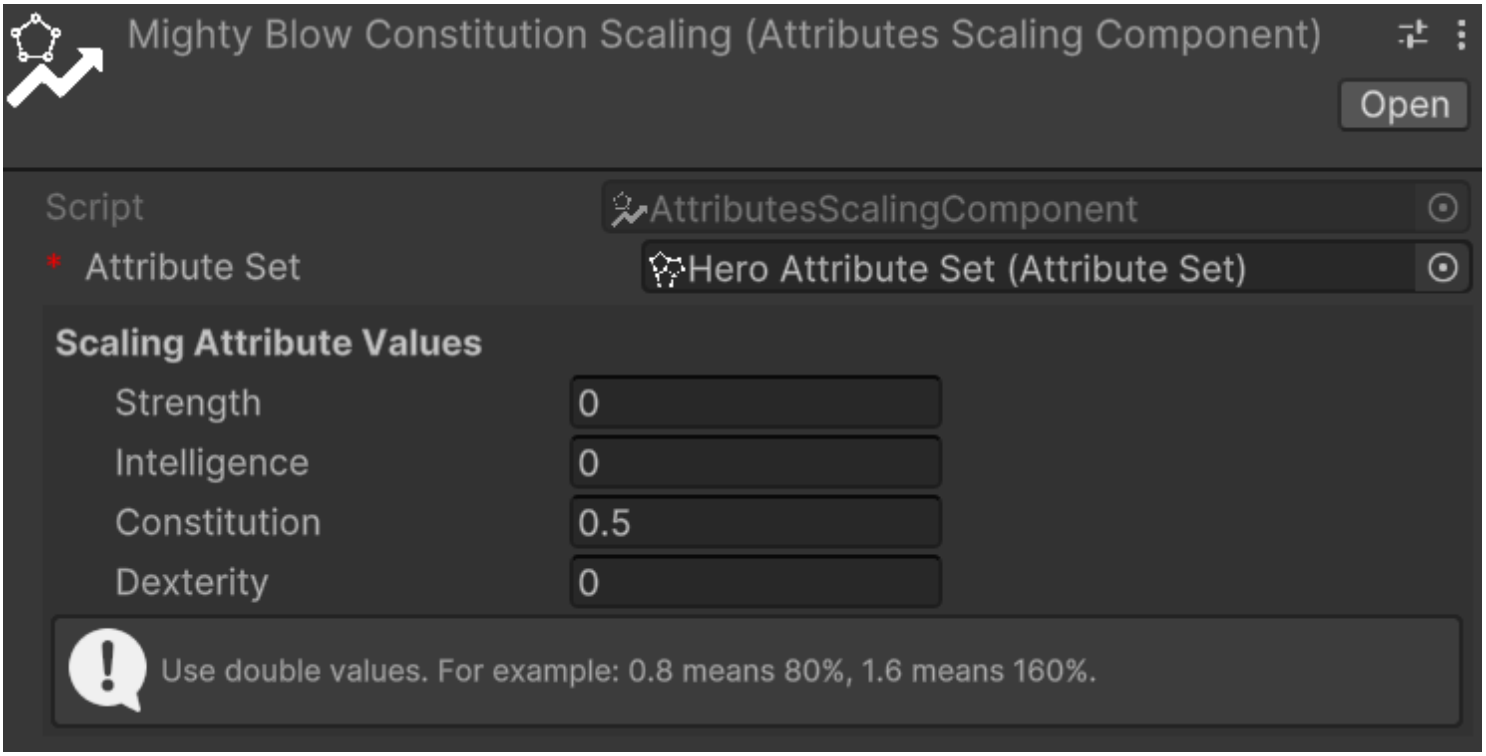
We can now proceed to create the scaling components for the `Physical Attack` stat and the `Constitution` attribute.

Let's create a new `Stat Scaling Component` called `Mighty Blow Physical Attack Scaling`. Assign the `Hero Stat Set` to it and set the scaling of the `Physical Attack` stat to `1.5`. The scaling component should look like this:

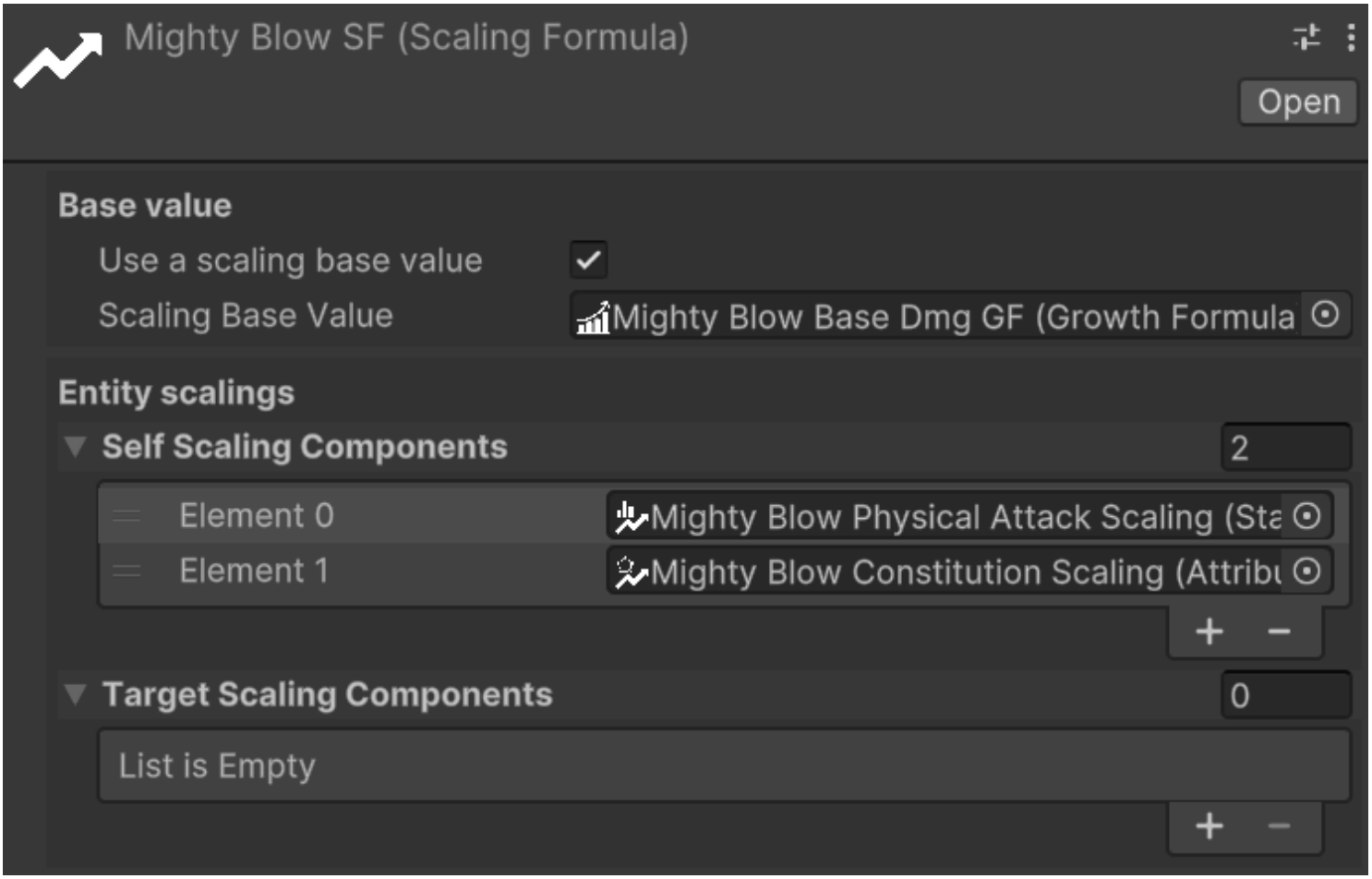


Next, we will create a similar `Attribute Scaling Component` for the `Constitution` attribute called `Mighty Blow Constitution Scaling`. Assign the `Hero Attribute Set` to it and set the scaling of the `Constitution`

to `0.5`. The scaling component should look like this:



Finally, let's press on the `+` of `Self Scaling Components` and assign the two scaling components we just created. The `Mighty Blow SF` should look like this:



Using scaling formulas in code

Because `ScalingFormula` is a shared `ScriptableObject` asset, multiple entities can reference the same instance. To support per-entity runtime modifications without affecting other entities, use `ScalingFormulaInstance` — a lightweight wrapper you create once per entity at initialisation:

```
var instance = new ScalingFormulaInstance(mightyBlowSF);
```

`ScalingFormulaInstance` exposes two lists: `TmpSelfScalingComponents` and `TmpTargetScalingComponents`. These let you add scaling components at runtime without touching the original asset. For example, the character could get a temporary buff that makes the `Mighty Blow` skill scale also with the `Intelligence` attribute. In this case, we can add an `AttributeScalingComponent` for `Intelligence` to the instance's `TmpSelfScalingComponents`:

```
instance.TmpSelfScalingComponents.Add(intelligenceScalingComponent);
```

If the buff wears off, remove the component:

```
instance.TmpSelfScalingComponents.Remove(intelligenceScalingComponent);
```

There is also `ResetTmpScalings()` to clear all temporary components at once, useful for example when the player completes a room and all temporary buffs should be cleared:

```
instance.ResetTmpScalings();
```

Both `ScalingFormula` and `ScalingFormulaInstance` implement the `IScalingFormula` interface, so you can use either interchangeably wherever only value calculation is needed.

The four `CalculateValue` methods are available on both:

- `long CalculateValue(EntityCore self)`: Calculates the value of the scaling formula by summing the value returned by each self scaling component (calculated on the entity itself values), there must not be any target scaling components.
- `long CalculateValue(EntityCore self, int level)`: If the scaling formula has a base value that varies with levels, this method calculates the value of the scaling formula for the entity itself, and adds the base value at a specific level. Again, there must not be any target scaling components.
- `long CalculateValue(EntityCore self, EntityCore target)`: Calculates the value of the scaling formula by summing the value returned by each self scaling component (calculated on the entity itself values) and each target scaling component (calculated on the target entity values).

- `long CalculateValue(EntityCore self, EntityCore target, int level)`: Calculates the value of the scaling formula by summing the value returned by each self scaling component (calculated on the entity itself values) and each target scaling component (calculated on the target entity values), and adds the base value at a specific level.

Game Events

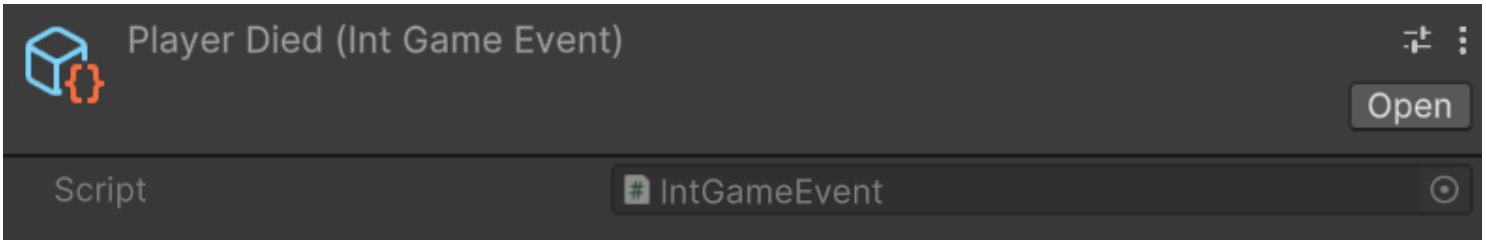
Relative path: `Events -> Game Event` *Relative path for custom game events:* `Events -> Generated -> *CustomEventName*`

At the beginning of this page we briefly mentioned the concept of game events as scriptable objects, and we have introduced the generic `GameEvent`. Such event is great for notifying actions that happen in the game, but it has limited flexibility as it is not carrying along any context information. For example, if we want to notify that a character has leveled up, we might want to pass along the entity that leveled up and the new level reached. In other cases it could be useful to pass along with the event a reference to the entity that triggered the event, so we would like to be able to pass a reference of type `EntityCore` as context parameter. And so on.
The framework comes along with some pre-defined game events:

- `IntGameEvent`: an event that carries along an `int` as context parameter
- `EntityCoreGameEvent`: an event that carries along an `EntityCore` as context parameter
- `EntityLeveledUpGameEvent`: an event that carries along an `EntityCore` and an `int` as context parameters, where the `EntityCore` is the entity that leveled up and the `int` is the new level reached
- `StatChangedGameEvent`: an event that carries along a `StatChangeInfo` context parameter, which contains:
  - A reference to the `EntityStats` component of the entity that has changed
  - The `Stat` that has changed
  - The previous value of the stat
  - The new value of the stat
- `AttributeChangedGameEvent`: an event that carries along an `AttributeChangeInfo` context parameter, which contains:
  - A reference to the `EntityAttributes` component of the entity that has changed
  - The `Attribute` that has changed
  - The previous value of the attribute
  - The new value of the attribute

Along with the game events, the frameworks provides the `*GameEventListener` counterparts, which are components that can be attached to a game object to listen for the events and execute a method when the event is raised. For example, `IntGameEventListener` listens for a specific `IntGameEvent` and executes a method that takes an `int` as parameter when the event is raised.

Let's suppose we need a game event for notifying, each time the player dies, how many times the player has died so far. In the inspector, create a new `IntGameEvent` called `PlayerDied`. It should look like this:



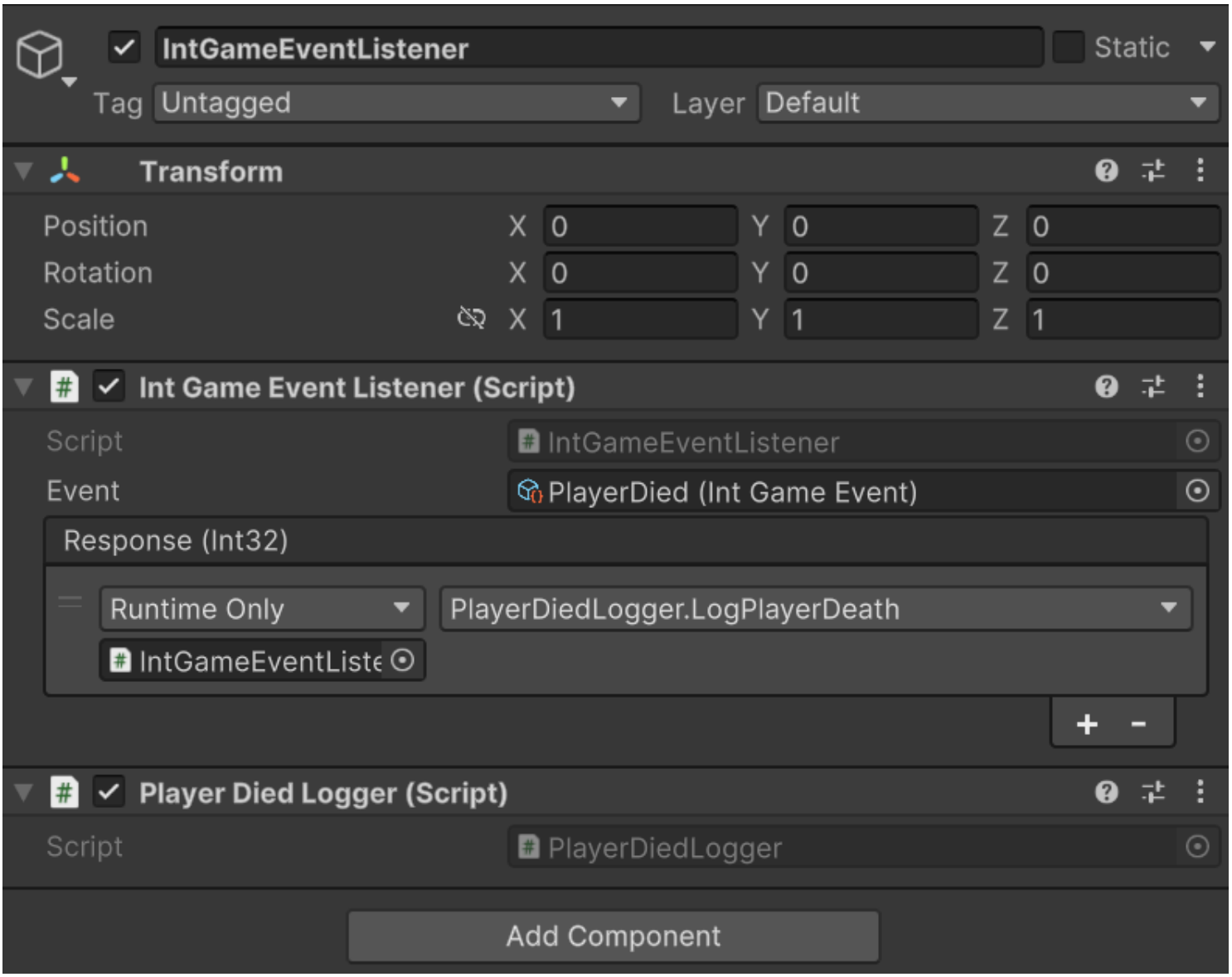
There are no fields to fill in the inspector. The integer to be passed as context parameter will be passed in the code that is responsible for raising the event.
The method to raise the event is `Raise(int value)`, which will raise the event and pass along the integer as context parameter. Therefore in a dedicated script you can call it like this:

```
public void OnPlayerDeath() {
    playerDeathCount++;
    // Assuming playerDiedEvent is a reference to the PlayerDied event
    playerDiedEvent.Raise(playerDeathCount);
}
```

Now let's create a listener for this event. Let's say that we want to show a message in the console when the event is raised. To do this, create a new game object in the hierarchy and add a script that defines a public method for logging an event, that takes an `int` as input parameter. It should be like:

```
public void LogPlayerDeath(int deathCount) {
    Debug.Log($"Player has died {deathCount} times.");
}
```

Now add the `IntGameEventListener` component to the listener game object and, in the inspector, assign the `PlayerDied` event to the `Event` field. Now drag the `Logger` script created before into the `Response`'s object selector. You should be able now to select the `LogPlayerDeath` method from the dropdown menu. The result should look like this:



This is a powerful mechanism that decouples the event producers from the event consumers, allowing for a more modular and maintainable codebase. And most of the setup is inspector-based. You just need to define the raising and the listening methods from code.

Global framework events and reactive filtering

The framework dispatches its built-in core events through the active Astra RPG Framework configuration. `EntityCore`, `EntityLevel`, `EntityStats`, and `EntityAttributes` send their shared `Spawned`, `Level Up`, `Level Down`, `Stat Changed`, and `Attribute Changed` notifications through the configuration asset selected in `Edit -> Project Settings -> Astra RPG Framework`. The Project Settings entry selects the active configuration asset; the shared event references themselves are assigned inside that configuration asset. If no Project Settings config is assigned, the runtime falls back to a `Resources/Astra Rpg Framework Config` asset.

This shared-event model is convenient, but it also means listeners and reactive triggers usually subscribe to common event sources rather than to holder-specific event instances. In other words, the holder attached to a reactive subscription is used for ownership and cleanup, not as an automatic payload filter.

When a global stat or attribute event is raised, every subscriber on that event source can receive the payload.

In practice, pair shared event sources with payload-aware filtering. The most inspector-friendly option is usually a `ConditionalGameAction<TContext>` that checks the entity, tags, or value-change details you care about before running its inner action. See Conditions for the full condition model and Game Tags for the tag-based workflow.

Game Event Generators

Relative path: `Events -> Game Event Generator`

> **WARNING**
> Game event generators are an experimental feature and may change in future releases.
> To avoid potential issues, it is recommended to back up or version control your project before renaming or moving the generated events to different folders.

Sometimes the pre-defined game events are not enough to cover all the use cases. In this case, you can create a custom game event generator. A game event generator is a scriptable object that lets you define a custom game events with up to four context parameters. You can choose the type of each parameter. Parameters can either be primitive types (like `int`, `float`, `string`, etc.) or more complex types (like `EntityCore`, `Stat`, `StatChangeInfo`, ..., or your own data types).

Game Event Generator setup

Let's create a custom game event generator to manage all the events related to the experience and leveling up of entities. Rename the newly created events generator `EntityLevelingEvents`. In the inspector, it should look like this:

With `Menu Base Path` we can change the path of the context menu where the generated events will be available for creation. By default, it is set to `Astra RPG Framework/Events/Generated`, but we can change it to `Astra RPG Framework/Events/Generated/Experience` for the sake of organization.

With `Base Save Location` we can change the path where the source code files for the generated events will be saved. By default it is set to `Assets`, but for this example let's set it to `Assets/Events`.

Adding new events

Now let's create an event that will be raised when an entity grants experience to another entity. To do this, click on the `Add new event` button and fill in the fields as follows:

- `Event Name`: `EntityGrantedExp`
- `Documentation`: an entity granted experience to another entity
- `Parameters`: press the `+` button and add the following three parameters:
  - `Parameter Type` to `Mono Script`, `Mono Script` type to `EntityCore` (drag it from the AstraRpgFramework folder located in the Packages folder)
  - `Parameter Type` to `Mono Script`, `Mono Script` type to `EntityCore` (drag it from the AstraRpgFramework folder located in the Packages folder)
  - `Parameter Type` to `Native`, `Native type` to `long`

The first parameter represents the entity that granted the experience, the second parameter represents the entity that received the experience, and the third parameter represents the amount of experience granted.

Now let's press the `Generate Game Events` button to create the new event. We can now navigate to `Assets/Events/GeneratedEvents/EntityLevelingEvents` to find the following two folders:

- `GameEventListeners`: contains the source code for all the game event listeners generated by our `EntityLevelingEvents` game event generator.
- `GameEvents`: contains the source code for all the game events generated by our `EntityLevelingEvents` game event generator.

Inside both folders, you'll find a subfolder named `3`. The Game Event Generators organize the generated events in subfolders based on the number of parameters they have. In this case, we have a game event with three parameters, so it is placed in the `3` subfolder.

However, more interesting is the fact that if we now use the context menu and navigate to `Astra RPG Framework/Events/Generated/Experience`, we can find the `EntityGrantedExp` event. From here, you can follow the same steps as for the pre-defined game events to create a listener for this event, and to wire it up to the appropriate game logic.

If you need to create more experience related events, you can repeat the process of adding new events to the `EntityLevelingEvents` game event generator.

After having generated the source code for the game events, you can also click on the `Remove Event` button under each Game Event to remove it from the generator and delete the generated source code files. It might be necessary to refresh (right click on an asset folder and select `Refresh`) the asset folder containing the generated events to see the changes reflected in the Unity editor.

> **WARNING**
>
> Pay attention that renaming or moving the generated events to different folders will cause the game event generator to lose track of them. If you change the `Base Save Location` and then re-generate the events, the game event generator will first delete the old files and then create new source code files in the new location. This means that any references to the old files will be lost, so you will need to reassign them in the inspector or in the code. This can be highly disruptive if you have many references to the old files, so it is recommended to back up or version control your project before renaming or moving the generated events to different folders.
> To safely move files you should move them from the Unity editor and then update the `Base Save Location` field in the game event generator to match the new location of the generated events. This way, the game event generator will be able to find the generated events in the new location and will not attempt to delete/recreate them.
> Same applies if you rename the generated game event type scripts. To rename them, rename their C# scripts first, and then, and then update the `Event Name` field in the game event generator to match the new name.

Game Tags

🏷️ *Version 2.0.0+*

`GameTag`s let you classify entities, assets, and reusable framework objects with lightweight `ScriptableObject` identifiers. Each tag can also carry visual metadata, so tagged objects are immediately recognizable in the Inspector through colored pill-style labels.

What a `GameTag` contains

A `GameTag` asset stores:

- A primary color
- An optional icon (typically an emoji or short symbol)
- An optional two-color gradient
- A gradient direction

Assigned tags are stored through `GameTagSet`, which is the serializable container used by taggable objects. Built-in framework types that expose tags directly through `ITaggable` include:

- `EntityCore`
- `Stat` and `StatSet`
- `Attribute` and `AttributeSet`
- `Class`
- `GrowthFormula`
- `ScalingComponent` and `ScalingFormula`
- `IntVar` and `LongVar`
- `GameAction<TContext>` assets

This makes tags useful both for gameplay meaning and for editor organization.

Querying tags from code

At runtime, the most direct way to work with tags is through objects that implement `ITaggable`. The `Tags` property exposes a `GameTagSet`, which provides the three core queries:

- `Contains`
- `ContainsAny`
- `ContainsAll`

For a single tag:

```
if (entityCore.Tags.Contains(bossTag))
{
```

```
    Debug.Log("This entity is tagged as a boss.");
}
```

To check whether an object contains at least one tag from another set:

```
if (entityCore.Tags.ContainsAny(hostileOrUndeadTags))
{
    Debug.Log("This entity matches at least one tag in the query set.");
}
```

To check whether an object contains every tag from another set:

```
if (entityCore.Tags.ContainsAll(requiredLootTags))
{
    Debug.Log("This object satisfies the full tag requirement.");
}
```

These helpers make `GameTagSet` useful both for gameplay checks and for higher-level filtering logic.

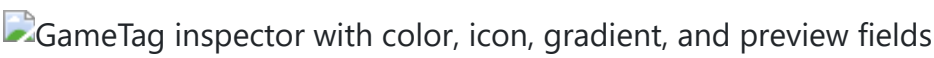
Creating a `GameTag`

Relative path: `Game Tag`

The custom inspector exposes a **Visual** section where you can configure the tag's appearance:

- **Color** for the main pill color
- **Icon** for an optional emoji or short symbol
- **Emojis...** to open the emoji picker
- **Use Gradient** to switch from a flat fill to a two-color gradient
- **End Color** and **Direction** when gradient mode is enabled

The inspector also shows a live preview of the resulting pill.


GameTag inspector with color, icon, gradient, and preview fields

Where tags appear

When the inspected target implements `ITaggable`, the framework automatically injects a tag bar directly below the default inspector header. No per-type setup is required for supported objects.

This injected header bar is the intended authoring workflow for tag editing. In tag-aware custom inspectors, the underlying serialized data may be hidden on purpose so interaction stays guided through the pill-based header UI instead of a lower-level field editor.

> **NOTE**
>
> If an object does not implement `ITaggable` but still contains one or more `GameTagSet` fields, the header pills are shown in display-only mode. In that case, the header still helps you scan tags quickly, but the interactive editing workflow remains on the serialized field itself.

Editing tags from the inspector header

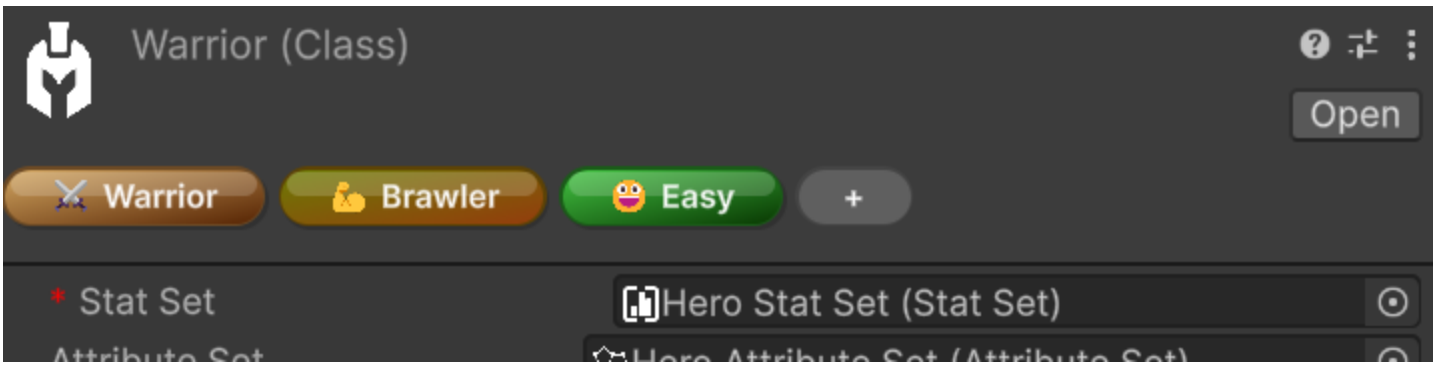
Single-object editing

When a single taggable object is selected, the header pills are fully interactive:

- Click a visible pill to ping the `GameTag` asset in the Project window
- Hover a pill to reveal the `-` remove affordance
- Click the `+` pill to open the tag selection popup
- Drag pills to reorder them

The add popup supports several quality-of-life features:

- Search by tag name
- `All` and `None` quick selection controls
- Multi-selection followed by **Add**
- Double-click on a tag to add it immediately



GameTag add popup with search and multi-selection

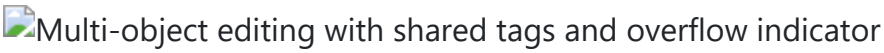
> **TIP**
>
> Double-click is the fastest way to add a single tag. If you want to add several tags in one pass, select them in the popup first and then confirm with **Add**.

Multi-object editing

When multiple compatible `ITaggable` objects are selected, the header still supports batch editing, but the displayed pills follow shared-state rules:

- Visible pills represent the intersection of tags present on **all** selected objects
- A grey `+N` overflow pill indicates tags that exist on only some of the selected objects
- The `+` pill adds the chosen tags to all selected objects in one batch operation
- Tags already present on every selected object are excluded from the add popup
- Tags present on only some selected objects remain selectable, so you can complete the assignment across the whole selection
- Removing a visible shared tag opens a confirmation popup listing the affected objects before the batch removal is applied

In multi-object mode, reordering and click-to-ping are intentionally not the primary workflow, because a heterogeneous selection does not have a single stable tag order.

 Multi-object editing with shared tags and overflow indicator

Pill appearance preferences

The visual pill rendering also has per-user editor preferences under:

`Edit > Preferences > Astra RPG Framework > Game Tag Pills`

These preferences let each developer customize how tag pills are presented locally in the Unity Editor. Available options include:

- **Gloss Highlight** for a glossier pill style
- **Drop Shadow** for extra separation from the inspector background
- **Icon-Only Mode** for compact visualization when the tag has an icon
- **Expand on Hover** to reveal the full label while hovering compact pills
- **Animate Expand** to make the compact expansion smoother
- **Animation Speed** to control how fast the hover expansion animates

> **NOTE**
>
> These preferences only affect how pills are rendered in the local Unity Editor. They do not modify the `GameTag` asset data itself. **Icon-Only Mode** only affects tags that actually have an icon configured, and **Expand on Hover** / **Animate Expand** apply only when icon-only mode is enabled.

Practical uses

`GameTag`s work well for:

- Grouping class-related assets (for example, Warrior-related formulas, abilities, and supporting assets)
- Marking gameplay categories such as factions, rarity, or behavior flags
- Organizing reusable assets like `GameAction`s, formulas, and value assets
- Making important editor objects easier to scan visually
- Supporting tag-aware runtime logic in systems that evaluate tag presence

Using tags consistently also helps keep larger projects navigable over time. Future updates may introduce custom windows and other tooling that use `GameTag`s to scan Astra assets more easily across a project — for example, to surface assets related to a Warrior class setup or to inspect collections of mob scaling formulas. Even before those tools arrive, using tags to create order in your project is already a strong long-term investment.

Conditions

🏷️ *Version 2.0.0+*

> **WARNING**
>
> The `Conditions` system is currently experimental and may change at any time.
>
> Be careful when adopting it for production-critical features. Future package updates may require refactoring condition trees, revisiting authoring workflows, or adjusting integration code that depends on the current behavior.
>
> The feature is expected to move toward a more stable shape once Astra Modifiers is released, where `Condition`s are used extensively. Until then, it is safer to treat the current API and authoring flow as provisional.

`Condition`s are reusable predicates evaluated against an `EvaluationContext`. They let you express editor-authored rules such as "run this action only when the holder is below a threshold", "only pass when the payload entity matches the holder", or "only continue when the target has a required tag set".

Unlike `GameTag`s or `GameAction`s, conditions are usually not standalone assets. They are authored inline in `[SerializeReference]` fields, where you choose a concrete condition type from a grouped picker and configure it directly in the Inspector.

The condition model

Every concrete condition derives from `Condition` and implements `Evaluate(EvaluationContext ctx)`.

The evaluation context carries the runtime references that built-in conditions inspect:

Context value	Typical meaning
`Holder`	The entity that owns the evaluated effect or action
`Performer`	The entity that applied or cast the effect, when available
`EventPayload`	The current trigger payload or event data

`EvaluationContext` also exposes `TryGetPayload<T>()` when a condition needs typed access to the current payload.

Many built-in conditions resolve one of these context slots before applying their actual check. For example, an entity condition may compare the holder against a payload entity, while a tag condition may

inspect the resolved entity for one or more required tags.

Context contracts and payload roles

Many of the less obvious conditions are really about **which contracts the current payload implements**. The system does not rely on a single event-context base class. Instead, conditions look for small focused interfaces that describe what the payload can provide.

Common context contracts

Contract	Meaning
`IHasEntity`	Exposes the payload's primary entity
`IHasTarget`	Exposes the target entity of the payload
`IHasPerformer`	Exposes the entity that performed or caused the action
`IHasVictim`	Exposes the victim entity when that role exists
`IHasValueChange<T>`	Exposes `PreviousValue`, `NewValue`, and `AbsAmount` for numeric-change payloads

`IHasEntity` is the most important bridge contract in the framework. It represents the primary entity carried by a payload, and it is also the interface that lets many `GameAction<IHasEntity>` workflows accept both entity components and event payloads in a consistent way.

How `ConditionTarget` resolves entities

Entity-based and tag-based conditions select an entity slot through `ConditionTarget`:

Target slot	Resolved from
`Holder`	`EvaluationContext.Holder`
`Performer`	`EvaluationContext.Performer`
`PayloadEntity`	`IHasEntity` on the payload, or the `EntityCore` of a payload `Component`
`PayloadPerformer`	`IHasPerformer` on the payload
`PayloadTarget`	`IHasTarget` on the payload
`PayloadVictim`	`IHasVictim` on the payload

This is why some conditions appear to "just work" with certain event payloads and not with others: the payload must actually expose the requested role.

Common payloads used by built-in conditions

Several built-in event payloads already expose the contracts that the condition system expects:

Payload type	Contracts	Typical use
`StatChangeInfo`	`IHasEntity`, `IHasTarget`, `IHasValueChange<long>`	Stat-based conditions and long value-change conditions
`AttributeChangeInfo`	`IHasEntity`, `IHasTarget`, `IHasValueChange<long>`	Attribute-based conditions and long value-change conditions
`EntityLevelChangedContext`	`IHasEntity`, `IHasTarget`, `IHasValueChange<int>`	Entity level conditions and int value-change conditions

For example, `StatChanged`, `AttributeChanged`, `EntityLevelUp`, and `EntityLevelDown` payloads already expose the entity and value-change contracts needed by the related condition families.

Built-in condition families

The type picker groups the built-in condition library into several families:

`Composites`

Use these to combine or transform other conditions:

- `AllConditions`: logical AND; every child must pass, and an empty list evaluates to true
- `AnyCondition`: logical OR; at least one child must pass, and an empty list evaluates to false
- `NoneCondition`: logical NOR; no child may pass, and an empty list evaluates to true
- `NotCondition`: inverts a single inner condition; if the inner value is null, it evaluates to true
- `AtLeastNCondition`: passes when at least `N` child conditions pass
- `AtMostNCondition`: passes when at most `N` child conditions pass
- `ExactlyNCondition`: passes when exactly `N` child conditions pass

These are the main building blocks for condition trees that need more than one rule. `AtLeastNCondition`, `AtMostNCondition`, and `ExactlyNCondition` are especially useful when you want a count-based rule instead of a strict AND/OR shape.

`Leaf/Constant`

Simple fixed predicates:

- `AlwaysTrueCondition`: always passes
- `AlwaysFalseCondition`: never passes

These are useful as placeholders or for testing larger composite trees.

Leaf/Attr. & Stat

Conditions in this family evaluate attributes, stats, or stat/attribute-related payloads:

- `AttributeThresholdCondition`: resolves an entity slot, reads one attribute from that entity, and compares it against a threshold
- `StatThresholdCondition`: resolves an entity slot, reads one stat from that entity, and compares it against a threshold
- `ChangedAttributeCondition`: checks whether the current payload is an `AttributeChangeInfo` for a specific `Attribute`
- `ChangedStatCondition`: checks whether the current payload is a `StatChangeInfo` for a specific `Stat`

Threshold-based conditions operate on the current value stored on the resolved entity. By contrast, `ChangedAttributeCondition` and `ChangedStatCondition` do not compare numbers at all: they only verify which attribute or stat the current change payload refers to.

`AttributeThresholdCondition` can also work in percentage mode, where the current value is interpreted as a percentage of the attribute's configured range.

Leaf/Entity

These conditions reason about entity roles and entity relationships in the evaluation context:

- `EntityReferenceCondition`: compares two selected entity slots such as holder, payload entity, performer, target, or victim
- `EntityLevelCondition`: resolves one entity slot and compares that entity's **current** level against a threshold
- `EntityLevelThresholdTransitionCondition`: reads an `int` value-change payload and checks whether a level comparison became satisfied or became unsatisfied across the change
- `HolderLevelCondition`: a shorter holder-only version of a level comparison
- `IsPerformerCondition`: passes when `Performer` and `Holder` are the same entity

This family is especially useful when you need to distinguish self-targeting, compare holder and payload entity, or react only at specific level boundaries.

Some practical reading tips:

- `EntityReferenceCondition` is the most general entity-role matcher. It is ideal when the meaning of the rule depends on relationships such as "holder equals payload entity" or "payload target is

different from performer"

- `EntityLevelCondition` looks at the entity's current live level, not at a before/after transition
- `EntityLevelThresholdTransitionCondition` is for level-change payloads and asks whether a rule changed state across the event (for example, whether `level >= 10` just became true)
- `HolderLevelCondition` is simpler than `EntityLevelCondition`, but also less flexible because it cannot look at payload roles
- `IsPerformerCondition` is mainly useful in self-applied or self-triggered workflows

> **NOTE**
>
> `PayloadEntity` is often the most useful slot when working with built-in event payloads, because types such as `StatChangeInfo`, `AttributeChangeInfo`, and `EntityLevelChangedContext` already implement `IHasEntity`.

Leaf/Tag

The tag-focused family lets conditions inspect tags on resolved entities or on active modifier data:

- `EntityHasTagCondition`: resolves one entity slot and checks whether that entity's tags match a configured `GameTagSet`
- `HasActiveModifierTagCondition`: checks whether the holder currently has at least one active modifier whose tags match the configured set

`EntityHasTagCondition` supports both **Any Of** and **All Of** matching against a configured `GameTagSet`. `HasActiveModifierTagCondition` relies on `IActiveModifierTagProvider`, so it is mainly relevant when the holder is participating in modifier-based workflows. For the tagging workflow itself, see Game Tags.

Leaf/ValueChange

These conditions are designed for payloads that carry value-delta information through `IHasValueChange<T>`.

All of them read one of these projections:

- `PreviousValue`
- `NewValue`
- `AbsAmount`

The built-in numeric payloads are split by type:

- `EntityLevelChangedContext` provides `IHasValueChange<int>`
- `StatChangeInfo` and `AttributeChangeInfo` provide `IHasValueChange<long>`

> **NOTE**
>
> This is why the payloads used by built-in events such as entity level up, entity level down, stat changed, and attribute changed work naturally with the `Leaf/ValueChange` family: they already implement `IHasValueChange<T>`.

The available conditions are:

- `IntValueChangeDirectionCondition`: checks whether an `int` payload increased, decreased, or stayed the same
- `IntValueChangeModuloCondition`: applies a modulo rule to a projected `int` value
- `IntValueChangeThresholdCondition`: compares a projected `int` value against a threshold
- `IntValueChangeThresholdTransitionCondition`: checks whether an `int` threshold comparison became satisfied or unsatisfied across the change
- `LongValueChangeDirectionCondition`: checks whether a `long` payload increased, decreased, or stayed the same
- `LongValueChangeModuloCondition`: applies a modulo rule to a projected `long` value
- `LongValueChangeThresholdCondition`: compares a projected `long` value against a threshold
- `LongValueChangeThresholdTransitionCondition`: checks whether a `long` threshold comparison became satisfied or unsatisfied across the change

They are useful when the interesting rule is not the absolute final value, but the way a value changed.

This distinction matters:

- **Direction** conditions care only about increase, decrease, or equality
- **Threshold** conditions evaluate a single projected value such as `NewValue >= 10`
- **Threshold transition** conditions evaluate whether the comparison changed state between previous and new values
- **Modulo** conditions are useful for cyclical checks on the projected value, such as verifying that a `NewValue` level lands on 5, 10, 15, ... or that another projected value repeats on a fixed remainder pattern

For example, if a level changes from 9 to 10:

- `IntValueChangeThresholdCondition` with `NewValue >= 10` passes because the new value is 10
- `IntValueChangeThresholdTransitionCondition` with `>= 10` and **Become Satisfied** passes because the comparison was false before and true after

`Leaf/Random`

Randomized gating is provided through:

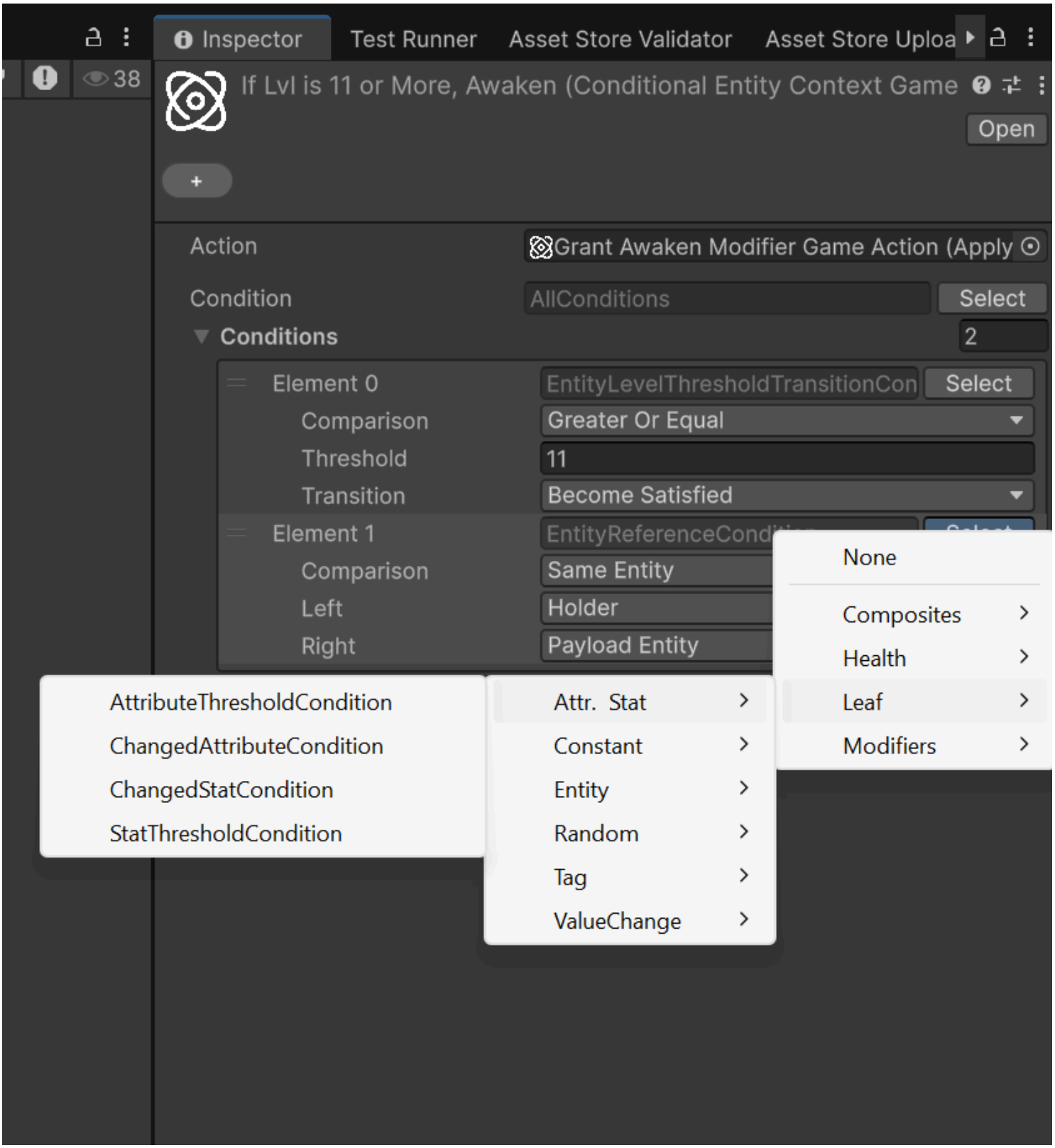
- `RandomChanceCondition`: passes with the configured percentage chance

This condition evaluates to true based on a configured percentage chance.

Authoring conditions in the Inspector

Because conditions are managed-reference values, the typical workflow is:

1. Locate a field that accepts a `Condition`
2. Open the type picker for that field
3. Choose a concrete condition from the grouped menu
4. Configure its fields, such as target entity slot, comparison mode, threshold, or required tag set
5. Wrap the root in composites if the rule needs multiple checks



NOTE

Conditions are authored inline, so their serialized data lives inside the object that owns the field. You do not create a standalone `Condition` asset first and reference it later.

Conditional game actions

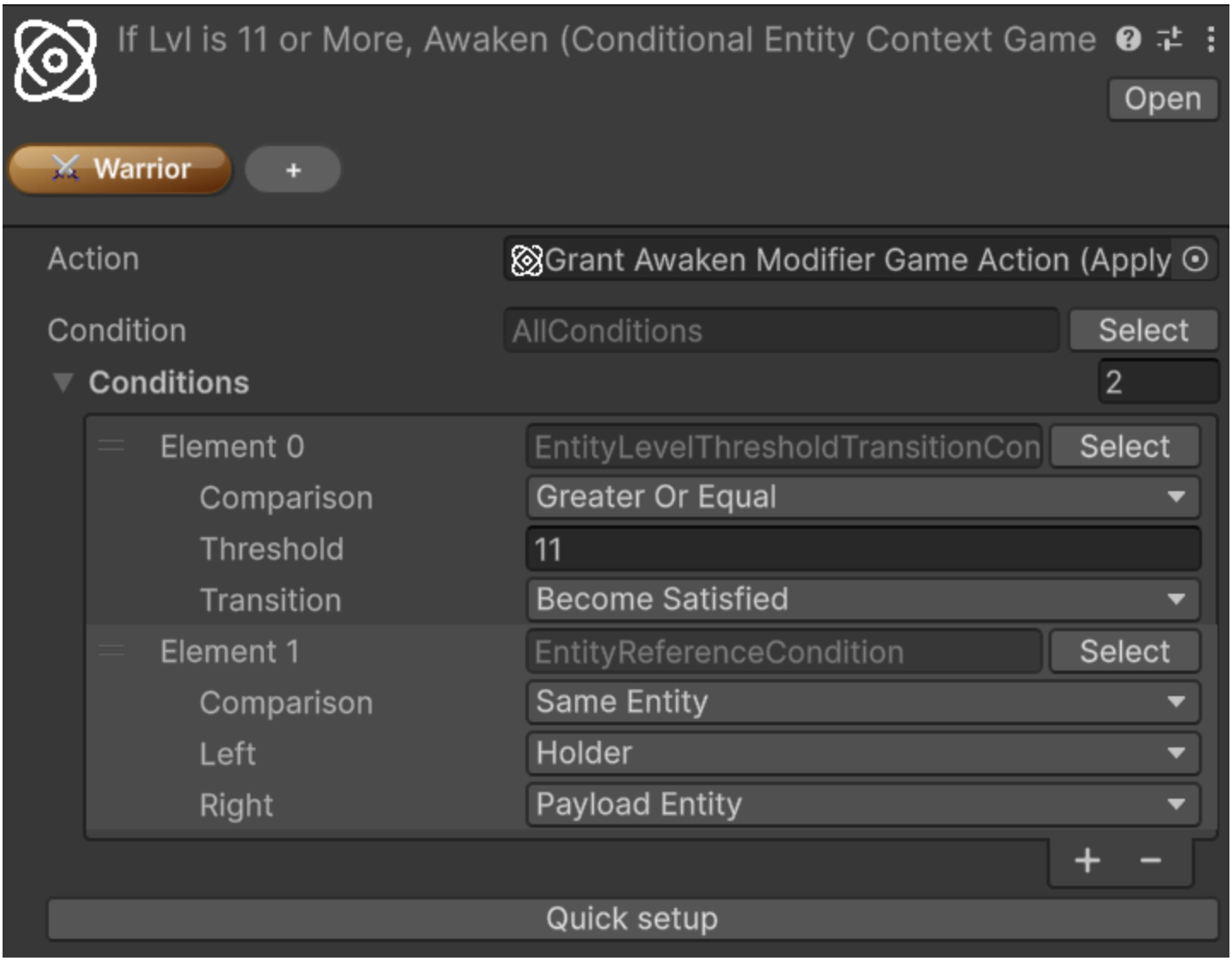
The most direct built-in use of the system is `ConditionalGameAction<TContext>`, which wraps another `GameAction<TContext>` with an optional guard condition.

- `Astra RPG Framework/Game Actions/Context: Entity/Conditional`
- `Astra RPG Framework/Game Actions/Context: Component/Conditional`

If the configured condition evaluates to `false`, the nested action is skipped. If the condition field is left empty, the action behaves as always allowed.

The custom inspector for conditional actions also exposes a **Quick setup** row below the condition field. This is meant to speed up structural edits to the root condition tree:

- Wrap the current root in `AllConditions`
- Wrap the current root in `AnyCondition`
- Wrap the current root in `NotCondition`
- Unwrap the current root when the current shape supports it



For the broader `GameAction` workflow, see Game Actions.

Practical uses

`Condition`s are especially useful for:

- Gating a `GameAction` so it only runs for certain entities or payload states
- Comparing holder, performer, and payload entity roles without writing custom glue code
- Reacting only when an attribute or stat crosses a threshold
- Filtering tag-aware behavior through `GameTagSet` matching
- Adding probabilistic behavior through a reusable chance-based condition

The main strength of the system is composition. Instead of hardcoding special-case checks into every consumer, you can assemble reusable condition trees directly in the Inspector and keep the rule logic close to the data that depends on it.

Advanced topics

Presentation layer and localization

🏷️ *Version 1.2.0+*

The framework focuses on the business logic of RPG systems and does not impose any specific presentation layer or localization strategy. You are free to implement your own UI and localization solutions that best fit your project's needs. However, the framework provides basic support for the presentation layer by allowing you to define display names for attributes, statistics, and classes. These display names can be used in your UI to present information to players in a user-friendly manner. In fact, ScriptableObjects names are convenient and quick to use during development, but they are not ideal for presentation to players. By defining display names, you can separate the internal representation of your RPG systems from the user-facing presentation layer.

`IDisplaySONameProvider`

The framework includes the `IDisplaySONameProvider` interface, which can be implemented by UI components to display readable names for attributes, statistics, and classes:

```
public interface IDisplaySONameProvider<in T> where T : ScriptableObject
{
    string GetDisplayName(T asset);
}
```

The framework provides a default implementation of this interface, `DefaultDisplaySONameProvider`, which retrieves the ScriptableObject's name and returns it as the display name. You can use this default implementation or create your own custom implementation to suit your localization needs.

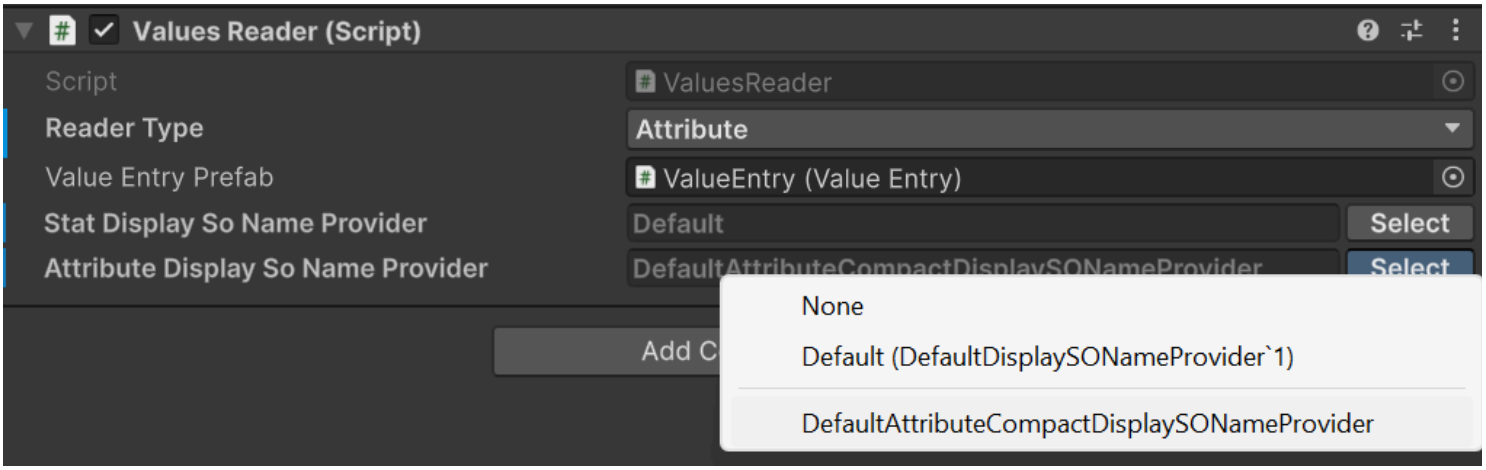
The attributes are a particular as in many RPGs they can either be presented with the extended name (e.g., "Strength") or with an abbreviation (e.g., "STR"). To accommodate this, the framework provides the `DefaultAttributeCompactDisplaySONameProvider` implementation, which returns the first three letters, capitalized, as the display name for a certain attribute.

Using custom display name providers

To use a custom display name provider, you can instantiate your implementation in your own scripts, or rely on the `TypeSelectable` attribute provided by the framework to select the implementation from the Unity inspector. For example:

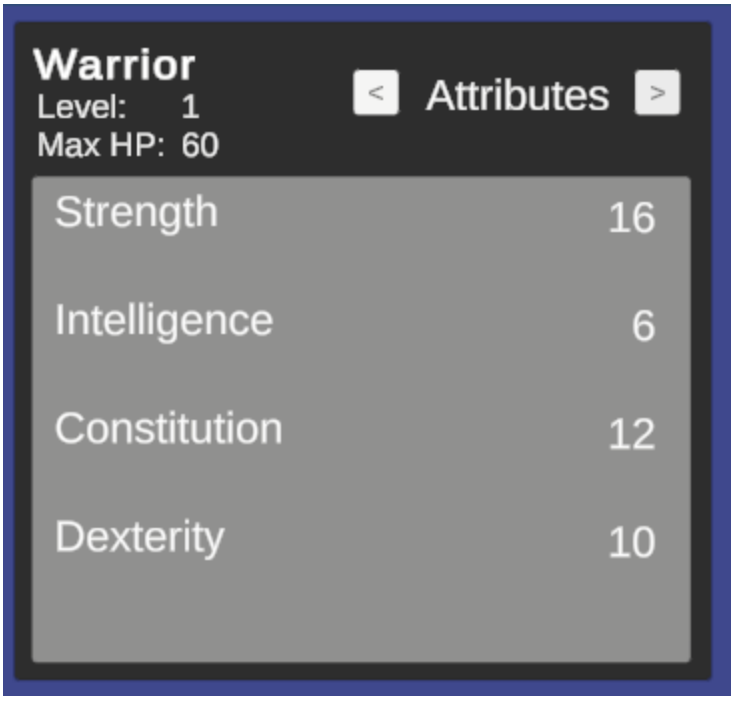
```
[SerializeReference, TypeSelectable(typeof(DefaultDisplaySONameProvider<Attribute>))]
private IDisplaySONameProvider<Attribute> _attributeDisplayNameProvider;
```

`TypeSelectable` is an experimental feature of the framework that allows you to select a concrete implementation of an interface from the Unity inspector.
The parameter passed to `TypeSelectable` is the default implementation that will be selected when the inspector is first displayed. You can then choose a different implementation from a dropdown list in the inspector. If you take a look at the demo scene, if you select the `AttributesContainer` GameObject inside `WarriorCanvas -> HeroPanel` in the hierarchy, you will see a `Attribute Display So Name Provider` field in the inspector under the `Values Reader` component. This field uses the `TypeSelectable` attribute to allow you to choose between the default and compact display name providers for attributes.

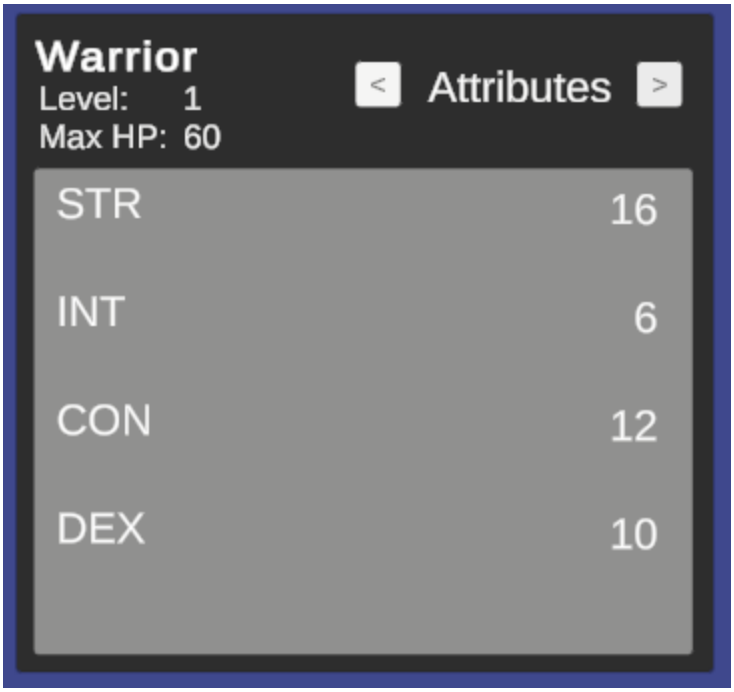


The implementation passed as parameter to `TypeSelectable` will be instantiated automatically by the framework during script compilation. Moreover, the framework will show it in the inspector, in the dropdown list, as the Default option under `None`.

During Play Mode, you can switch between the different implementations to see how the display names change in the UI of the sample scene. With the default implementation, the attributes will be displayed with their full names:



With the compact implementation, the attributes will be displayed with their abbreviations:



Reader APIs and safe value access

🏷️ *Version 2.0.0+*

When you only need to read values, prefer the dedicated reader interfaces over reaching into concrete components. `IAttributeReader` and `IStatReader` expose safe `TryGet` / `TryGetBase` APIs that let you query both final and pre-modifier values without assuming the requested asset is present.

```
IAttributeReader attributeReader = entityCore;
IStatReader statReader = entityCore;

if (attributeReader.TryGet(strengthAttribute, out long strength) &&
    statReader.TryGet(physicalAttackStat, out long physicalAttack))
{
    Debug.Log($"STR {strength}, PATK {physicalAttack}");
}
```

`EntityCore` implements both reader interfaces, which makes it a convenient read facade for gameplay systems that already work with the entity as their entry point.

If you also need the unmodified values, use `TryGetBase`:

```
if (statReader.TryGetBase(physicalAttackStat, out long basePhysicalAttack))
{
    Debug.Log($"Base Physical Attack: {basePhysicalAttack}");
}
```

`Get` and `GetBase` can still be convenient on concrete containers when you already know the asset exists, but `TryGet` and `TryGetBase` are the safer default for reusable code paths. This is especially relevant in v2.0.0 because `StatSetInstance` is no longer an `IStatReader`; code that needs a generic stat reader should depend on `EntityCore`, `EntityStats`, or another explicit `IStatReader` provider instead.

Rounding double-based calculations

🏷️ *Version 2.0.0+*

The framework now exposes `RoundingMode` to make integer conversion intent explicit when a calculation produces a `double`. The built-in modes are:

- `Round`: rounds to the nearest integer, with midpoint values rounding away from zero
- `Floor`: rounds down
- `Ceil`: rounds up

```
double scaledValue = 214.5;
int rounded = RoundingMode.Round.ApplyToInt(scaledValue); // 215
int floored = RoundingMode.Floor.ApplyToInt(scaledValue); // 214
```

Use `RoundingMode` when authoring custom calculators or other extension points that need deterministic integer conversion semantics instead of ad-hoc casts.

Owner-aware `GameAction` execution

🏷️ *Version 2.0.0+*

`GameActionBase` is the non-generic root used by inspector-facing authoring surfaces that need to reference actions with different concrete context types. Concrete assets still execute through `GameAction<TContext>`, which now also supports owner-aware execution through `ExecuteWithOwnerAsync`.

This matters most for event-driven workflows. `GameEventListener`s can dispatch owner-aware actions using the listener component as runtime owner, and wrapper actions such as `Composite`, `Delayed`, `Conditional`, and projection actions preserve that owner while forwarding execution. This allows conditions that depend on `Holder` resolution to behave consistently across nested action chains.

For the practical inspector workflow, see Workflows. For the condition-side resolution model, see Conditions.

Limitations

Current Limitations

- Currently, multi-classing is not supported. Each entity can only have one class at a time
- Stat modifiers are applied in the following order:
  1. Flat modifiers
  2. Stat-to-stat modifiers
  3. Percentage modifiers
- Stat-to-stat modifiers use the base value and the flat modifiers for the source stat for the calculations. This ensures that the modifiers are applied consistently and predictably, and that circular dependencies can be defined without issues.
- Attribute modifiers do not have a attribute-to-attribute modifier system. This was a design choice to keep the system simpler.
- Attribute modifiers are applied in the following order:
  1. Flat modifiers
  2. Percentage modifiers
- Only one event for stats changed, attributes changed, and entity spawned can be assigned to entities. However, multiple event listeners can be assigned for the same event type, allowing for multiple responses
- Custom game event generators support up to four context parameters. To work around this limitation, users can combine multiple pieces of data into a single user-defined class—a technique known as "parameter packing." This approach is extensible through inheritance and helps maintain a simple, user-friendly API by avoiding telescoping parameters.

Addressed Limitations

Since Version	Original Limitation	Notes
🏷️ *v2.0.0+*	~~`OnStatChangedEvent` is risen only when modifiers are applied, not on level-ups/base stat changes~~	Stat-changed events now fire consistently during class progression formulas, level-ups, level-downs, and when dependent attributes change, following internal refactoring of `EntityStats` and `EntityAttributes`.
🏷️ *v2.0.0+*	~~`OnAttributeChangedEvent` is risen only when modifiers are applied and spendable points are assigned, not on level-ups/base attribute changes~~	Attribute-changed events now fire consistently during class progression formulas, level-ups, and level-downs, following internal refactoring of `EntityAttributes`.

Requirements

- Unity 6 or later

Package Contents

The package ships more than the sample scene. Besides examples, it includes the runtime and editor subsystems that Astra RPG Framework uses internally and exposes for your own project workflows.

For a guided tour of the sample content, see Samples.

Main package subsystems

Subsystem	What it contains	Learn more
Entities, stats, attributes, classes, scaling	The core progression model, including `EntityCore`, value containers, class data, growth formulas, and scaling assets.	Workflows
Game Events	Predefined event types, listeners, and the generator-based workflow for creating additional typed events.	Workflows
Global framework configuration	The shared config used to dispatch built-in framework events such as spawned, level up/down, stat changed, and attribute changed.	Workflows
Game Actions	Reusable asynchronous actions, projections, wrappers, and owner-aware event-driven execution flows.	Workflows, Advanced topics
Game Tags	Tag assets, tag sets, and inspector header pills used to organize Astra assets and drive tag-based logic.	Game Tags
Conditions and reactive filtering	Polymorphic condition trees, conditional actions, and the trigger/filtering model used by reactive systems.	Conditions

Editor tooling included in the package

The package also contains editor-side tooling under `Editor/` that powers several authoring workflows documented elsewhere in this site:

- the Project Settings page for the global Astra RPG Framework configuration
- inspector support for `SerializeReference`-based authoring flows
- the injected header pills used by taggable assets and components
- helpers used by condition and game-action inspectors

These editor tools are part of the package experience and are one of the reasons the framework can expose rich inspector-driven workflows without requiring custom tooling in your own project.

Samples

Importing the Samples

After having imported the package in your project, to import the samples into your project, head first to the package manager window. For latest versions of Unity, you can find it under `Window -> Package Management -> Package Manager`, otherwise, it should be under `Window -> Package Manager`.

From the package manager window, select the `In This Project` tab from the menu on the left, then select `AstraRPGFramework` from the list of packages. You'll notice that there are four tabs on the right side of the window: `Details`, `Version History`, `Dependencies`, and `Samples`. Select the `Samples` tab, then click on the `Import` button to import the desired samples in the project.

The import will copy the samples of the package into your project's `Assets` folder, under `Assets/AstraRPGFramework`.

The samples folder contains 2 sub-folders:

- **utils**: contains some utility objects that can be used in your game.
- **examples**: contains resources that are used in the sample scene to showcase the package's features

Utils

The `Utils` folder contains 2 sub-folders:

- `EventInstances`: containing `Entity Attribute Changed`, `Entity Spawned`, `Entity Stat Changed`, and `On Player Level Up` Game Events. These are instances of the Game Events defined in the package's source code.
- `Player`: containing `Max Level`, `Player Hp`, `Player Max Hp`, and `Player Level` Int and Long variables. These are useful to use Scriptable Object variables in your game, as showcased in the Workflows documentation.

Examples

There are several objects in this folder. Let's start by the Sample Scene. Here we have two relevant game objects: `Heroes` and `HeroesHUD`. These wrappers contain the entity game objects and the GUI elements that are used to display the heroes' stats and attributes respectively. There are three heroes in the `Heroes` GO: `Warrior`, `Mage`, and `Rogue`. Each hero has its own game object, with the following components:

- `EntityCore`
- `EntityClass`
- `EntityStats`
- `EntityAttributes`

The objects in the `HeroesHUD` are not as relevant. Feel free to explore them, but they are not the focus of this documentation as they are just used to display the heroes' stats and attributes in the UI.

Along with the Sample Scene, there are a few sprites and UI scripts (in the `Sprites` and `Scripts` folders respectively), some prefabs in the `Prefabs` folder, a few events in the `Events` folder, and some more relevant content in the `Instances -> Classes` folder. Here, we have all the instances of the frameworks's scriptable objects for the Sample Scene.
For each class we have the following:

- Attributes growth formulas
- Stats growth formulas
- An Int var for the hero's level
- A Max HP growth formula
- The class itself All these objects are used in the Sample Scene to showcase the package's features.

Using the Sample Scene

The scene uses `Strength`, `Intelligence`, `Dexterity`, and `Constitution` as attributes, and `Physical Attack`, `Magical Attack`, `Critical Chance`, and `Defense` as stats. Each stat scales with the respective attribute:

- `Physical Attack` scales with `Strength`
- `Magical Attack` scales with `Intelligence`
- `Defense` scales with `Constitution`
- `Critical Chance` scales with `Dexterity`

If you now enter play mode in the Sample Scene, you'll see three heroes: the Warrior (left), Mage (center), and Rogue (right), each with their respective HUD displayed above them. You can use the arrow buttons (`<` and `>`) to toggle between viewing the heroes' stats and attributes.

The Sample Scene is designed to demonstrate how stats and attributes respond to changes made in the inspector.

For example, select the `Warrior` in the hierarchy and change its Level from 1 to 2 using the Entity Core component. You'll observe that the Warrior's `Physical Attack` increases from 28 to 29, and `Defense` rises from 26 to 27.

Increasing the level also grants the Warrior a spendable attribute point (visible in the `EntityAttributes` component). If you allocate this point to `Strength`, the `Physical Attack` will increase from 29 to 30.
If you revert the level back to 1, `Physical Attack` returns to 28 and the spent attribute point is removed.

Next, let's adjust how `Physical Attack` scales with the `Strength` attribute. By default, each point of `Strength` adds 1 point to `Physical Attack`. Suppose you want each point of `Strength` to contribute 3 points instead.
To do this, locate the `Physical Attack` stat instance in `Examples -> Instances -> Hero -> Stats`. Select

the `Physical Attack` stat, then double-click the `Physical Attack ASC` object in the inspector. This Attribute Scaling Component controls how `Physical Attack` scales with `Strength`.
You'll notice the scaling value for `Strength` is set to `1`. Change this to `3`. Instantly, the Warrior's `Physical Attack` at level 1 jumps from 28 to 60, since the base value is 12 and the Warrior has 16 points of `Strength`: 12 + (16 × 3) = 60.
If you increase the Warrior's level to 2, `Physical Attack` becomes 61. Assigning the new attribute point to `Strength` further increases `Physical Attack` from 61 to 64.

After exiting play mode, inspect the modified objects. You'll see that the Warrior's level is reset to 1, `Physical Attack` returns to 28, and the `Physical Attack ASC` (ASC stands for Attribute Scaling Component) scaling for `Strength` is back to 1.
This happens because changes made to `ScriptableObjects` during play mode are not saved to disk when you exit play mode. This is intentional Unity behavior, not a package bug.
Persisting such changes would be risky. For example, if a debuff reduces the scaling of `Physical Attack` from `Strength` to zero during play mode and this is saved, you would permanently lose the original scaling—even after restarting without the debuff.
`ScriptableObjects` in this package are intended to define your game's baseline mechanics. Any modifications during play mode are temporary and not meant to be saved. Fine-tuning these values is crucial in RPGs, and once you achieve the desired balance, you want to ensure those values remain intact.

You can also explore the other heroes and all their components in the scene. Play around with their levels, stats scalings, attributes and their spendable points, and see how they affect the heroes' stats and attributes in real-time.

Installation instructions

Importing Astra RPG Framework and its samples

1. From the package manager, import Astra RPG Framework. You can find the package in the "My Assets" section.
2. After importing, head to the "In Project" section, always in the Package Manager, and click on "AstraRPGFramework".
3. Click on the "Samples" tab and import the samples you desire. Find out more about the samples in the Samples documentation.
4. If you imported the "Example scene and instances" samples you need to import also TextMeshPro Essentials. Click on "Window > TextMeshPro > Import TMP Essential Resources".

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog.

[2.0.0] - 2026-04-19

> **WARNING**
> This update includes breaking changes and behavior changes that can affect existing projects. Refer to the Migrating to v2.0.0 section of the migration guide before updating.

Added

Runtime Features

- Added the `Conditions` system, including composite conditions, entity/tag/value-change/stat/attribute/random leaf conditions, and `ConditionalGameAction`.
- Added the `Game Tags` system with `GameTag`, `GameTagSet`, `ITaggable`, and tag-based conditions.
- Added framework-level configuration assets for built-in shared events through `AstraRpgFrameworkConfigSO`, `AstraRpgFrameworkGlobalSettingsSO`, and `AstraRpgFrameworkConfigProvider`.
- Added reactive trigger infrastructure with `IReactiveTrigger`, typed `GameEventTrigger<TContext>` implementations, and `EventSource` support for shared subscription coalescing.
- Added `IHasEntity` as the common bridge contract for entity-aware payloads and game actions, plus projection actions for entity-context workflows.
- Added owner-aware execution paths for `GameAction`s and introduced `GameActionBase` to support inspector-facing polymorphic action references.
- Added `IStatReader` and `IAttributeReader`, then extended the API with `TryGetBase`.
- Added `RoundingMode` and its helper extensions for explicit integer conversion from double-based calculations.

Editor Features

- Added dedicated authoring helpers for conditions, including quick setup utilities, managed-reference body drawers for condition fields, and condition tooltips.
- Added the Game Tag header-pill workflow, including popup search, multi-select add/remove, double-click quick add, intersection handling, overflow display, drag reordering, click-to-ping, and visual customization support.
- Added custom managed-reference drawers for condition-related attribute and stat fields.

Changed

Runtime Features

- Built-in `Spawned`, `Level Up`, `Level Down`, `Stat Changed`, and `Attribute Changed` events are dispatched through the active Astra RPG Framework configuration asset.
- `GameAction` workflows now lean on `IHasEntity` as the primary context for event-driven authoring, with owner propagation preserved across wrapper and projection actions.
- `EntityCore` now implements both `IStatReader` and `IAttributeReader`, making the entity itself a convenient read facade for gameplay code.
- Corrected the `EntityStats.AddStatToStatModifer(...)` typo to `AddStatToStatModifier(...)`.
- Stat and attribute changed notifications now cover broader effective changes, including dependent recalculations and bulk level transitions. See Addressed Limitations for details.
- `EntityLevel` now exposes more robust global-event integration and extra per-entity event registration helpers.

Editor Features

- Improved the fixed-base attribute handling in `EntityAttributesEditor` and `EntityStatsEditor` for Play Mode usage.

Removed

Runtime Features

- Removed the deprecated legacy `EntityLeveledUpGameEvent` and `EntityLeveledDownGameEvent` types together with the remaining deprecated legacy hooks in `EntityLevel`. These had already been deprecated in `v1.4.0`; if your project still uses them, complete the migration described in Migrating to v1.4.0 before upgrading.
- Removed the deprecated `AttributePointsTracker` compatibility layer from `EntityAttributes`. The transition to `EntityPointsTracker` and `AttributePortfolio` started in `v1.3.0`.

Editor Features

- Removed the deprecated `DerivedTypePicker` editor utility. The replacement path to `TypeSelectionMenu` was introduced in `v1.2.0`.

Fixed

Runtime Features

- Fixed level-down handling so fixed-base attribute snapshots are preserved correctly while raising change events.
- Fixed a null-reference path in `GameEventListener` validation when the UnityEvent response is missing.

[1.5.1] - 2026-04-04

Changed

Editor Features

- Reordered the EntityAttributes fields in the custom inspector to prevent the bug mentioned below

Fixed

Editor Features

- Fixed a bug in the EntityAttributes custom inspector that was not allowing to set either the fixed base Attribute Set nor the "Use Class Attributes From Class"

[1.5.0] - 2026-03-23

Added

Runtime Features

- Added `IScalingFormula` interface exposing the 4 main methods for scaling formulas. `ScalingFormula` and `ScalingFormulaInstance` now implement this interface.
- Added `ScalingFormulaInstance`. This class shall be used in place of `ScalingFormula` when you need to have a scaling formula that may use temporary scaling components. Temporary scaling components on the SO `ScalingFormula` have been deprecated an will be removed (see below). This ensures that the temporary scaling components do not modify the original `ScriptableObject` instance of the scaling formula, which could cause unintended side effects across different entities using the same scaling formula.
- Added several public read-only properties to `ScalingFormula`:
  - `UseScalingBaseValue`
  - `HasSelfComponents`
  - `HasTargetComponents`
  - `SelfScalingComponents`
  - `TargetScalingComponents`
- Added a public `MaxLevel` get property to `EntityLevel`
- All game event listeners log now an error if play mode is entered while no game event SO instance is assigned to them

Editor Features

- Added `Percentage` custom property drawer

Changed

Editor Features

- Improved the `GrowthFormula`'s custom inspector in several ways:
  - Repositioned remove buttons (Now just a "-" sign) to the top right of each growth expression (previously at the bottom)
  - When the `Use Constant Value At Level 1` is checked/unchecked, the first growth expression starting level is automatically set to 2 or 1 respectively, without the need for user input.
  - When adding the first growth expression, if the `Use Constant Value At Level 1` is checked, the starting level of the growth expression is automatically set to 2. If it's unchecked, it's set to 1.
  - When adding a growth expression, the starting level of the new expression is automatically set to one level higher than the previous last expression. The user can of course change it after, but this provides a more intuitive default behavior.
  - When a certain growth expression level is changed, the validation is not performed right away any more, but only after the user finishes editing the field (presses Enter or changes focus). This prevents weird behavior of the inspector while editing the growth expressions, such as automatic changes of the level of other expressions while editing a certain expression's level, which could be confusing.
  - Expressions can now be reordered with up and down buttons.
  - The growth expressions now have dedicated colored headers.
  - The table of the level to value mappings has been redesigned and now clearly shows the various growth expression ranges by using the same colors as the growth expressions.
  - The graph has been redesigned and now light vertical lines are shown to better identify the ranges.
  - The on-hover popup shown over the graph has been redesigned. It now also shows the growth expression and its range to which the hovered level belongs.

Samples

- Reviewed the style of the sample scene

Deprecated

Runtime Features

- Deprecated `TmpSelfScalingComponents` and `TmpTargetScalingComponents` in `ScalingFormula`

Removed

Runtime Features

- `ToggleHarvestedExpSourceGameAction` no longer prints a warning when no `ExpSource` component is attached to the entity on which is being executed

[1.4.2] - 2026-02-13

Changed

Samples

- The sample scene is now using custom materials and a custom shader graph to support multiple render pipelines.

Fixed

- Fixed a "Stat cannot be read" error being thrown upon project opening/scene loading.

[1.4.1] - 2026-02-09

Fixed

- Fixed an issue with the display of Included Stat Sets.

[1.4.0] - 2026-02-04

> **WARNING**
> This update involves breaking changes. Refer to the Migrating to v1.4.0 section of the migration guide for detailed steps to update your project.

Added

Runtime Features

- Added `GameAction` to encapsulate game logic as `ScriptableObject`s. Has a generic context type parameter to specify the type of object the action operates on.
- Added `IExecutable` interface. Implemented by `GameAction`.
- Added the following implementations of `GameAction`:
  - `ToggleRenderersGameAction`: Toggles the enabled state of all `Renderer` components on a GameObject and its children.
  - `ToggleHarvestedExpSourceGameAction`: Toggles the harvested state of an `ExpResource` component.
  - `ToggleCollidersGameAction`: Toggles the enabled state of all `Collider` components on a GameObject and its children.
  - `ToggleActiveGameObjectGameAction`: Toggles the active state of a GameObject.
  - `IncreaseCounterGameAction`: Increases a LongVar by a specified amount. Amount can also be negative to decrease the counter.
  - `DoNothingGameAction`: A no-op action that does nothing when executed. Reserved mostly for Astra RPG Health for neutral responses to certain events.
  - `DestroyGameAction`: Destroys the target GameObject.
  - `DelayedGameAction`: Executes another GameAction after a specified delay.

    - `CompositeGameAction`: Executes a list of GameActions in sequence.
- Added the Create Menu entries for all the above GameActions, with `EntityEngine.Component` as context type parameter.
- Added `GameActionRunner` MonoBehaviour. Used to execute GameActions in a fire-and-forget manner on a specific owner `GameObject`. Useful, for example, if the action is expected to complete and the GameObject owning the GameAction may be destroyed at any moment, or if you want to centralize the execution of GameActions on a specific GameObject (such as a manager).
- Added `IHasSource` interface to establish a contract for objects that have a source EntityCore. Used by Game Event contexts that have a source entity doing something. Currently reserved for future use in Astra RPG Health.
- Added `IHasTarget` interface to establish a contract for objects that have a target EntityCore. Used by Game Event contexts that have a target entity receiving something. Currently used by the `EntityLevelChangedContext` for the new Entity Level Up and Entity Level Down Game Events (see changes for details).
- `IHasValueChange<out T>` interface to establish a contract for objects that represent a change in value of type T. Used by Game Event contexts that represent a change in value:
    - `EntityLevelChangedContext` (int value change)
    - `AttributeChangeInfo` (long value change)
    - `StatChangeInfo` (long value change)
    - `IHasVictim` interface to establish a contract for objects that have a victim EntityCore. Reseved for future use in Astra RPG Health.
- Added custom icon for `ExpSource` MonoBehaviour.

Editor Features

- Added a `BrokenEventFinder` utility to help identify broken responses in Unity Events (and therefore in Game Events). Can be accessed from the menu: `Tools/Astra RPG Framework/Find Broken UnityEvents`.

Samples

- Added custom font for the Sample Scene.

Changed

Runtime Features

- Simplified `ExpSource` implementation. Now `Harvested` property is no longer set to true upon getting `Exp` property. Now the external code is responsible for setting the `Harvested` property to true when appropriate. This change allows for more flexible usage of the `ExpSource` component, as it can now be used in scenarios where the experience points are not immediately harvested upon retrieval. This change was made to better accommodate future features in Astra RPG Health.

Samples

- Updated the input system in the Sample Scene so that no manual action from the user is required when importing the project in Unity v6.2+.
- Renamed `SampleScene` to `Astra RPG Framework Sample Scene`.
- Updated `CommonApiCheatSheet` to reflect new EntityLevelUp and EntityLevelDown game events (see Deprecated section).

Fixed

Samples

- Fixed the input handling in the Sample Scene for Unity 6.2+

Deprecated

- Mark `EntityLeveledDown` and `EntityLeveledUp` events as obsolete, suggesting new alternatives. Use the new `EntityLevelUpGameEvent` and `EntityLevelDownGameEvent`, and the respective listeners, instead. **Notice that the new events don't have the `ed` suffix after `EntityLevel`**. See the Migration Guide for detailed migration steps.

Removed

- Removed previously deprecated `_includedStatSets` field in `StatSet`

[1.3.1] - 2026-01-10

Fixed

- Fixed an editor-scripts related issue that was causing build problems.

[1.3.0] - 2025-12-16

Added

Runtime Features

- Added `PRV[N]` and `AT[N]` keywords for growth formulas' expressions.
  - `PRV[N]`: Represents the n-previous Growth Formula value. For example, `PRV[3]` at level 8 would return the Growth Formula value at level 5.
  - `AT[N]`: Represents the Growth Formula value at a specific level N. For example, `AT[10]` would return the Growth Formula value at level 10.
- `BoundedValue.Clamp` now supports also clamping of double values. For now is used internally by the framework and is reserved for future use is Astra RPG Health.

Editor Features

- Added support for `enum` formatting in `InspectorTypography`.

- Added utility to provide compact representation of large numbers in the editor (e.g., 1.5K, 4M). Reserved for future use in Astra RPG Health.

Changed

Runtime Features

- Changed the internal `AttributePointsTracker` used by `EntityAttributes` to track spendable attribute points. `EntityPointsTracker` replaces the legacy tracker, and relies on a new `AttributePortfolio` to manage the allocation of attribute points to attributes. This new system is more readable, more robust, and more maintainable. The old system is left for backward compatibility. By updating the framework to this version, existing projects will automatically migrate to the new system.

Fixed

Runtime Features

- Fixed Round Robin Strategy for attribute points removal on level down or reset to level 1.

[1.2.0] - 2025-11-25

Added

Runtime Features

- Added `OnLevelDownEvent` Game Event. This event is triggered whenever an entity levels down, allowing you to respond to level down events in your game.
- Added XP deduction feature. Now you can deduct XP from an entity using the `EntityCore.Level.RemoveExp(long amount)` method. This will also handle level downs if the deducted XP causes the entity to drop below the current level's XP threshold. Triggers the `OnLevelDownEvent` once for each level the entity drops.
- Added reset to level 1 feature. You can now reset an entity's level and XP to level 1 using the `EntityCore.Level.ResetToLevelOne()` method. This will set the entity's level to 1 and XP to 0. Triggers the `OnLevelDownEvent` once for each level the entity drops.
- Added basic support for statistics, attributes, and classes names presentation. This includes:
  - `IDisplaySONameProvider<in T> where T : ScriptableObject` interface for providing custom display names for ScriptableObjects.
  - `DefaultDisplaySONameProvider<T>` class that implements the above interface and returns the ScriptableObject's name as the display name.
  - `DefaultAttributeCompactDisplaySONameProvider` class that provides compact display names for attributes (e.g., "Str" for "Strength"). This one simply takes the first three letters of the attribute's name, capitalize them, and return that as the display name.
    You can implement your own display name providers by implementing the

`IDisplaySONameProvider<T>` interface to provide custom display names for your statistics, attributes, and classes or to use **localization systems**.

Editor Features

- Added *Read Only* (`RO`) property in InspectorTypography. Marks a field in the inspector as read only, in the sense that the component automatically manages it, and the user should not manually edit any associate LongVar/IntVar associated values. Currently reserved for future use.
- Added `IValidatable` interface. Used by the framework to propagate validation events to depending objects. Used to ensure that when a MonoBehaviour/ScriptableObject is validated in the inspector, all depending objects are also validated to ensure data consistency.
- Added `TypeSelectionMenu`. Provides a reusable editor context menu to pick a concrete type derived from a given base type. Configurable display (optional "None" or "Default" entries, optional name trimming, exclusion of test types, extra filters). Returns the chosen type via a callback so callers can create or clear instances accordingly.
- Added `[TypeSelectable]` attribute for use on `[SerializeReference]` fields; supports an optional DefaultType to present a "Default" choice. Shows also "None" to clear the reference.
- Added `SerializeReferenceDrawer` custom property drawer. New editor PropertyDrawer showing current concrete type and a Select button for managed references. Opens centralized `TypeSelectionMenu`, supports choosing None, Default or concrete types and creates/assigns instances.
- Added `SerializeReferenceInitializer`. Runs in the editor to ensure `[SerializeReference]` fields marked with `[TypeSelectable]` that declare a default type are populated when scenes/prefabs are opened or on demand (during script compilation). Automatically instantiates and assigns default instances where fields are null, provides a menu command to initialize all applicable fields, and marks modified assets/scenes as dirty.

Samples

- Added `ExpUtils` MonoBehaviour. Adds utility methods that can be invoked from the Unity Editor (in play mode) to add or remove XP from an entity for testing purposes.

Changed

Runtime

- `SoSetScalingComponentBase` now logs a warning if no entity set is assigned during the lookup of values to use for scaling calculations.
- `AttributeScalingComponent` and `StatScalingComponent` now return `0` as the scaled value if no entity set is assigned, instead of rasing an error. This behavior allows for more flexible setups with entities with different sets of attributes/stats. The warning logged by the base class will help identify potential misconfigurations.

- Marked `OnEntityLevelUp` and `OnEntityLevelDown` events in `EntityCore` as required (red *). Such methods are becoming crucial for the correct functioning of the various components of the framework. Therefore, it's important to ensure that these events are properly assigned.
  Entities that are missing these events will raise an error during their `Start()` method.

Editor

- Improved hot-reloading support for StatSet and AttributeSet.
- Attributes of `AttributeSet` are not "Re-play" (yellow R) any more. This means that it supports changes (addition/removal) of attributes in the AttributeSet without exiting play-mode.
- Improved `InspectorTypography` class. Now it supports also property drawn within GUI rects.

Samples

- `ValuesReader` and `ValueEntry` now use `[SerializeReference]` and `[TypeSelectable]` attributes for allowing easier selection of the presentation strategy for each value entry (`IDisplaySONameProvider` implementation).

Deprecated

- Deprecated `DerivedTypePicker` in favor of the new `TypeSelectionMenu` system. Use the new system for better type selection in the editor. `DerivedTypePicker` will be removed in future releases.

Fixed

- Added missing invalidation of the cached values for the statistics in `EntityStats.OnEnable()` method.
- Fixed `SetTotalCurrentExp` that was not raising level up and level down events when appropriate.
- Fixed the assignment of total available attribute points. Previously, this was handled only in OnValidate, which caused issues in builds (where editor-only code is stripped). Initialization is now also performed in `EntityCore.Start()`. Additionally, total available attribute points are now updated within `AddPoints()`, ensuring they are correctly recalculated whenever points are added—not only during `Init()`.

[1.1.2] - 2025-11-12

Changed

- Updated the "Example scene and instances" samples. Now by default the Hero prefab uses a IntVar for the attribute spendable points per level. This makes more consistent the amount of spendable attribute points acquired by each character on level up
- Updated offline docs with Installation Guide section

[1.1.1] - 2025-11-06

Changed

- Updated the links in `package.json` to match the new docs URL

[1.1.0] - 2025-11-06

Changed

Updated branding from "SOAP RPG Framework" to "Astra RPG Framework"

IMPORTANT: Refer to the migration guide for updating existing projects that used the previous version.

- UPM package name changed to "com.electricdrill.astra-rpg-framework".
- UPM package display name changed to "Astra RPG Framework".
- Namespaces updated to reflect new branding: "ElectricDrill.AstraRPGFramework".
- Menu paths in Unity Editor updated to "Astra RPG Framework".

Deprecated

- `_includedStatSets` field in `StatSet` will become an internal property in future releases.

[1.0.0] - 2025-10-30

Added

- Initial release of SOAP RPG Framework.
- ScriptableObject-based toolkit for core RPG mechanics.
- MonoBehaviour-based components for easy integration.

ScriptableObject Features

- Statistics & Attributes: Define core stats like Strength and Intelligence, as well as derived attributes like Health or Mana.
- StatSets & AttributeSets: Group related stats and attributes together for organization. Sets can be nested for complex structures.
- Experience & Leveling System: Custom experience curves and character progression.
- Classes: Define character classes with unique progressions and abilities.
- Progression System: Control how stats and attributes grow as characters level up.
- Scaling Formulas: Custom formulas to calculate final values of stats and attributes.
- Scaling Components: Modular scaling formulas reusable across stats and attributes.
- Game Events: Design and trigger custom game events.
- Growth Formulas: Determine how values progress based on a character's level.
- Game Event Generators: Define custom game event types.
- Variables: Use Long and Int variables as ScriptableObjects for persistent data storage.

MonoBehaviour Features

- EntityCore: Manage a character's core data, including experience and level.
- EntityStats & EntityAttributes: Assign statistics and attributes to any entity. Components can retrieve values from the assigned EntityClass based on level.
- EntityClass: Assign a class to an entity, linking it to specific progression paths.

API & Modifiers

- Comprehensive API for advanced customization (see API Docs).
- Dynamic application of modifiers to stats and attributes:
  - Flat & Percentage Modifiers for attributes.
  - Flat, Percentage, & Stat-to-Stat Modifiers for statistics.

Migration Guide

Migrating to v2.0.0

v2.0.0 introduces new major subsystems such as Conditions, Game Tags, reactive triggers, and global framework configuration. Most of these additions are additive, but this release also includes a few breaking API changes and some behavior changes that existing projects should review carefully.

> **CAUTION**
>
> Before upgrading to `v2.0.0`, you must already have completed the migrations for older deprecated features that are permanently removed in this release:
>
> - **Legacy `EntityLeveledUpGameEvent` / `EntityLeveledDownGameEvent` and their listeners** — see the `v1.4.0` section of Changelog and Migrating to v1.4.0.
> - **`AttributePointsTracker`** — see the `v1.3.0` section of Changelog, where `EntityPointsTracker` replaced the legacy tracker. If your project comes from a pre-`v1.3.0` setup, open and save the affected prefabs, scenes, and assets on a `v1.3.x+` version before moving to `v2.0.0`.
> - **`DerivedTypePicker`** — see the `v1.2.0` section of Changelog, where it was deprecated in favor of `TypeSelectionMenu`.

1. Back up your project

Before updating, make sure your project is backed up or committed to version control. This release changes both runtime APIs and event wiring expectations, so it is worth having a safe rollback point.

2. Update the package from the Package Manager

Open Unity and navigate to `Window -> Package Management -> Package Manager` (`Window -> Package Manager` for older versions of the editor). Locate Astra RPG Framework in the "In This Project" list and update it to `2.0.0` (or the latest available). Let Unity finish recompiling before starting the migration work below.

Required migration steps

Update the framework configuration and shared event assets

The framework now dispatches its core global events — entity spawned, level up, level down, stat changed, and attribute changed — through a central `AstraRpgFrameworkConfigSO` asset. Three of those events (spawned, level up, level down) are **required**; without them the corresponding broadcasts are never fired.

> **IMPORTANT**
>
> **Do not use the sample events for your production project.**
> If you already have `EntityCoreGameEvent`, `EntityLevelUpGameEvent`, `EntityLevelDownGameEvent`, `StatChangedGameEvent`, and `AttributeChangedGameEvent` assets in your project that your systems already subscribe to, you must wire those into the config — not the generic instances that come with the samples. Reusing your existing event assets ensures that all existing listeners and gameplay logic continue to work without any changes.

Steps:

1. **Create your own `AstraRpgFrameworkConfigSO`** — right-click in the Project window and choose **Create → Astra RPG Framework → Config**, then save it somewhere in your project (e.g., `Assets/Config/`).
   For a full description of each field and the loading strategy, see Package Configuration.
2. **Open the newly created asset** and assign your existing event assets to the five fields:
   - *Global Entity Spawned Event* * — the `EntityCoreGameEvent` your systems already listen to
   - *Global Entity Level Up Event* * — the `EntityLevelUpGameEvent` your systems already listen to
   - *Global Entity Level Down Event* * — the `EntityLevelDownGameEvent` your systems already listen to
   - *Global Stat Changed Event* — the `StatChangedGameEvent` your systems already listen to (optional but recommended)
   - *Global Attribute Changed Event* — the `AttributeChangedGameEvent` your systems already listen to (optional but recommended)
3. **Register the config in Project Settings** — open `Edit → Project Settings → Astra RPG Framework` and assign your new config asset as the **Active Config Profile**.

> **CAUTION**
>
> If you skip step 3 and leave Project Settings empty, the framework will attempt the convention-based fallback (a `Resources` asset named `Astra Rpg Framework Config`). If neither is found, global events are never dispatched and any system that relies on them will silently fail at runtime. Always verify the Project Settings page shows **"✓ Using Explicit Configuration"** before entering Play Mode.

Rename the fixed typo in `EntityStats`

The `EntityStats` method name `AddStatToStatModifer(...)` was corrected to `AddStatToStatModifier(...)`.

1. Search your codebase for `AddStatToStatModifer`.
2. Replace each occurrence with `AddStatToStatModifier`.
3. Recompile and confirm no stale references remain.

Behavior changes to review

These changes may not produce compile errors, but they can change how existing gameplay logic behaves after the upgrade.

Stat and attribute changed events now cover more cases

Stat and attribute changed notifications are now raised for broader effective changes, not only for the most direct mutation paths. This includes dependent recalculations and bulk transitions such as some level-up/level-down flows.

Review any listeners, UI refresh logic, analytics counters, or reactive systems that assume those events only fire for direct edits. If needed, add filtering logic based on the payload contents before executing gameplay responses.

Disabled `EntityAttributes` components now resolve their current set differently

`EntityAttributes.GetCurrentAttributeSetAndHandleChanges()` now returns `null` when the component is inactive or disabled.

If you have custom code that queries attribute sets while the component is disabled, add a null check or defer that query until the component is enabled again.

Recommended cleanup and adoption

These steps are not strictly required to get your project compiling again, but they help align your code and content with the current package direction.

Migrating to v1.4.0

v1.4.0 introduces the new GameAction system and moves Game Events to a single-context parameter (Parameter Object pattern). This change improves extensibility and simplifies integration with GameAction because both systems now use a single context and are compatible with UnityEvent.

Key benefits

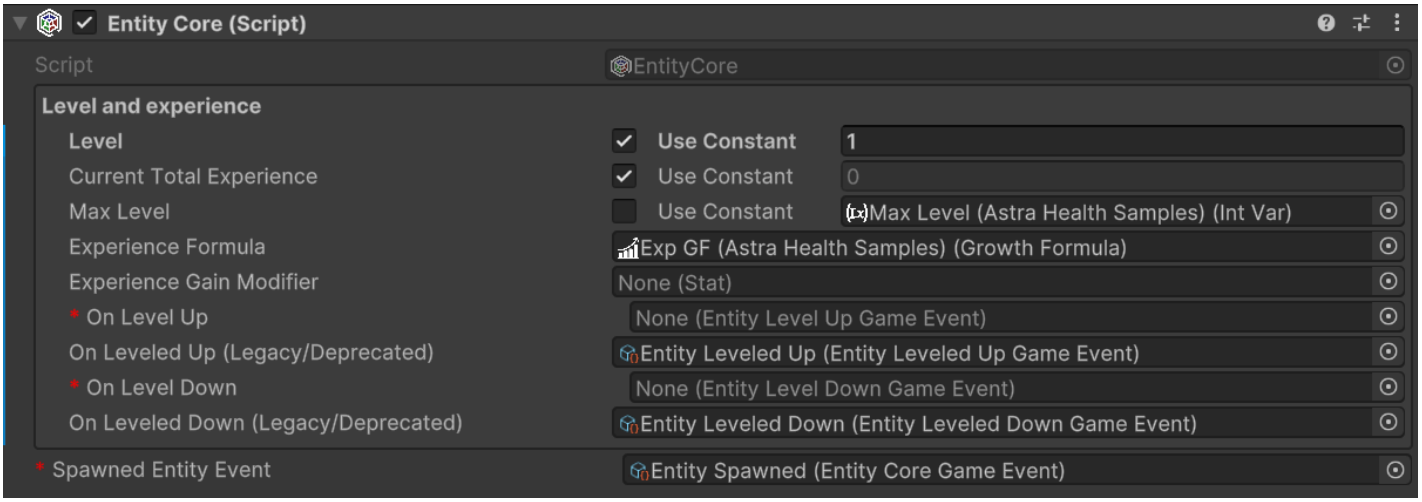
- Easier, non-breaking extension of event data via a single context object and polymorphism.
- Seamless interoperability with GameAction and UnityEvent-based listeners.

Breaking change

With the deprecation of the legacy `EntityLeveledUpGameEvent`, `EntityLeveledDownGameEvent` and their dedicated listeners, `EntityStats` and `EntityAttributes` now subscribe the new EntityCore's single-parameter events internally. All your `EntityCore` instances must be updated to use the new `EntityLevelUpGameEvent` and `EntityLevelDownGameEvent` instances. (Notice that the name of the deprecated events contained "Level**ed**" while the new events contain "Level".)

Migration steps - breaking changes

1. **Back up your project before making changes.**
2. Either create the new `EntityLevelUpGameEvent` and `EntityLevelDownGameEvent` instances from the context menu (`Create > Astra RPG Framework > Game Events > Generated > Entity Level Up` and `Entity Level Down`), or import the new event instances from the updated v1.4.0 `Utils` samples (`Utils > EventInstances > 1param > Entity Level Up Game Event` and `Entity Level Down Game Event`). If you re-import samples, prefer removing duplicate assets from the newly imported samples. Keep only the new events instances.
3. For each entity (prefab or scene object) open the `EntityCore` inspector and assign to the new `On Level Up` and `On Level Down` event fields to the new single-parameter event instances just created. The fields for the new events are marked as required (red asterisk), while the deprecated events fields are optional and marked with `(Legacy/Deprecated)`. The inspector, before assigning the new events, should look like this:



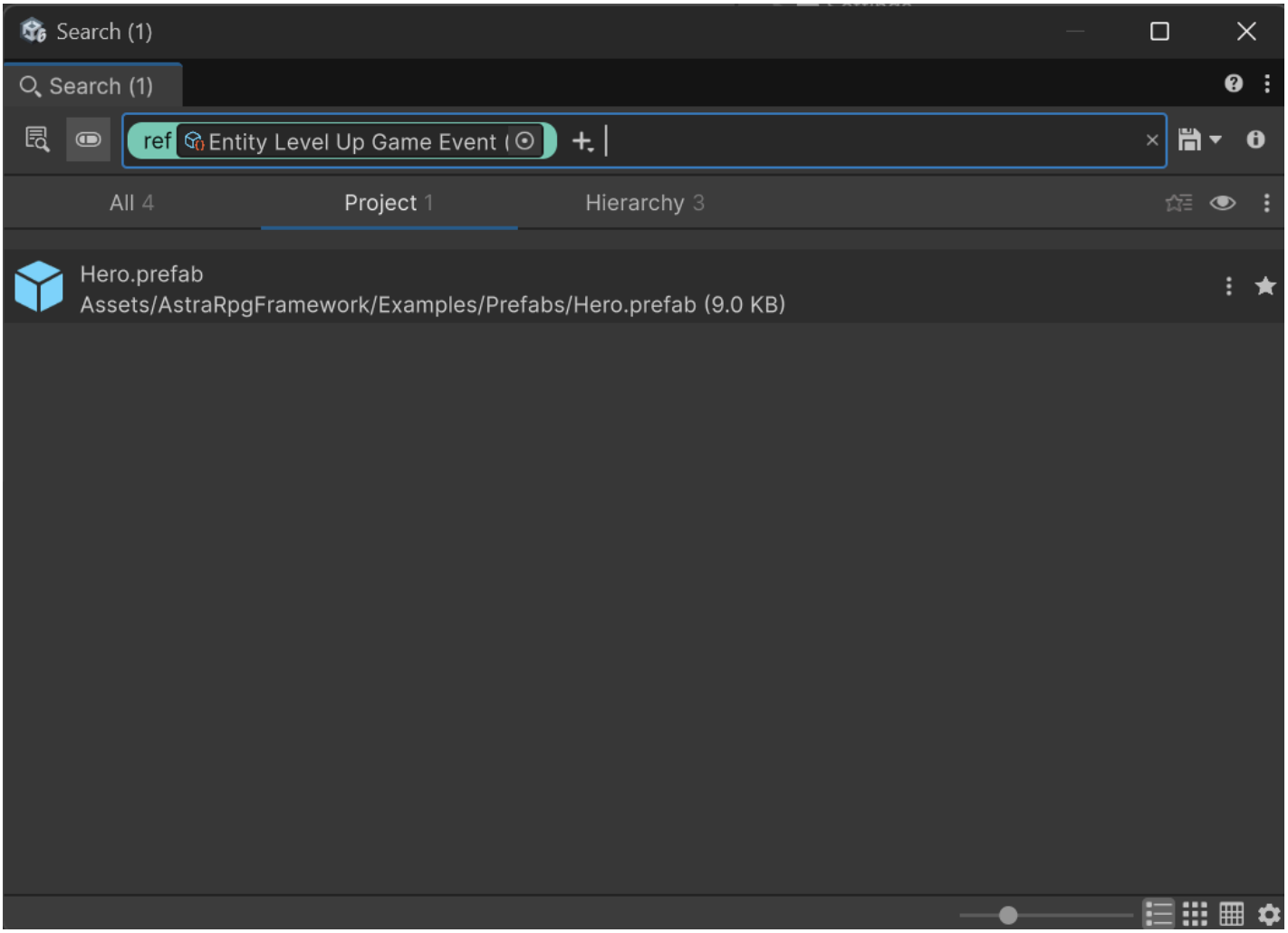
Migration steps - changing deprecated events

> **CAUTION**
>
> `EntityLeveledUpGameEvent`, `EntityLeveledDownGameEvent`, and their remaining legacy hooks were removed in `v2.0.0`. If you are upgrading directly to `v2.0.0`, complete the migration below before installing that version.

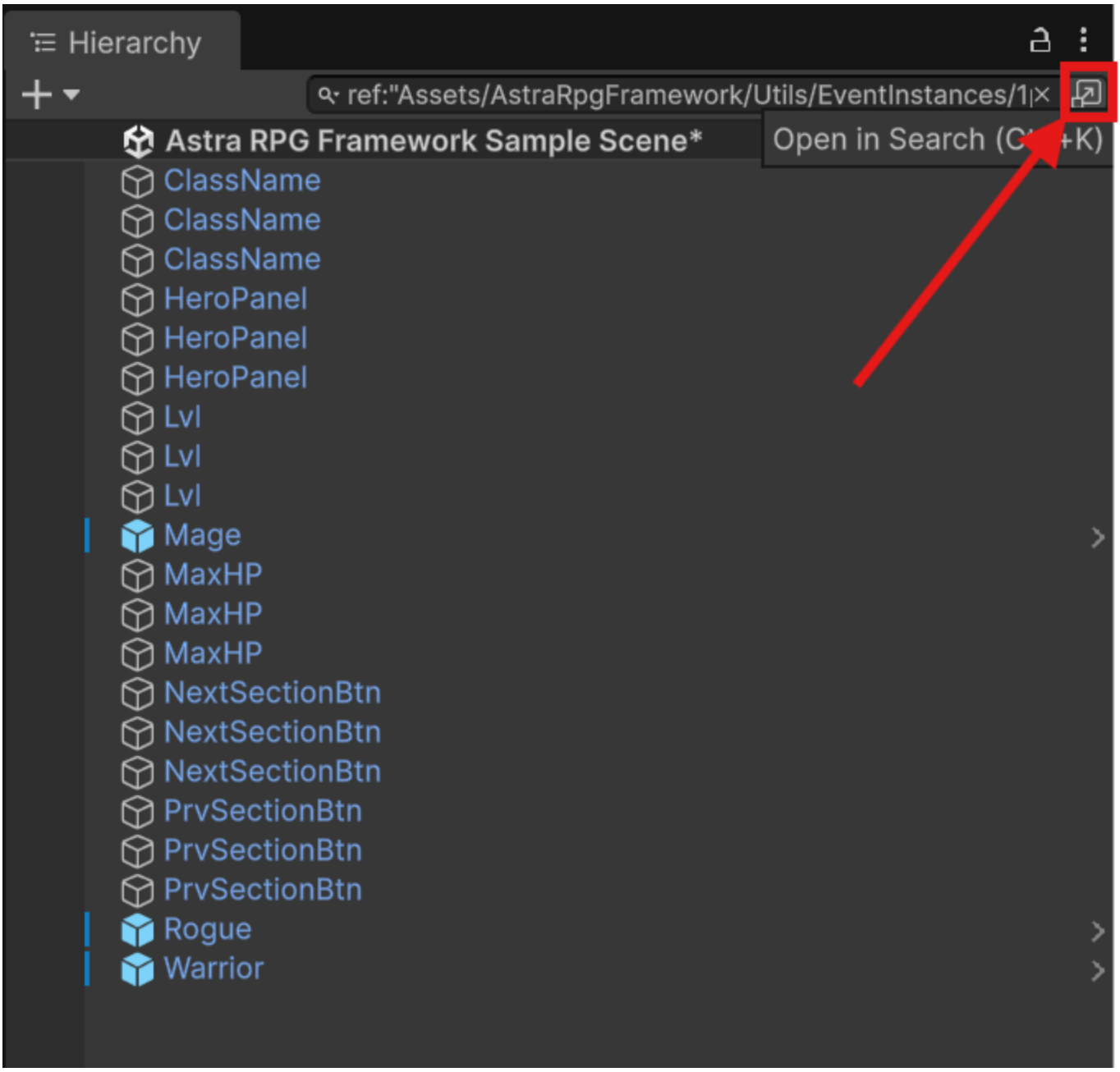
These legacy events and listeners were kept for backward compatibility after `v1.4.0`, but they are no longer available in `v2.0.0`. Projects that still reference them must be migrated to `EntityLevelUpGameEvent`, `EntityLevelDownGameEvent`, and the corresponding modern listeners before the `v2.0.0` update.

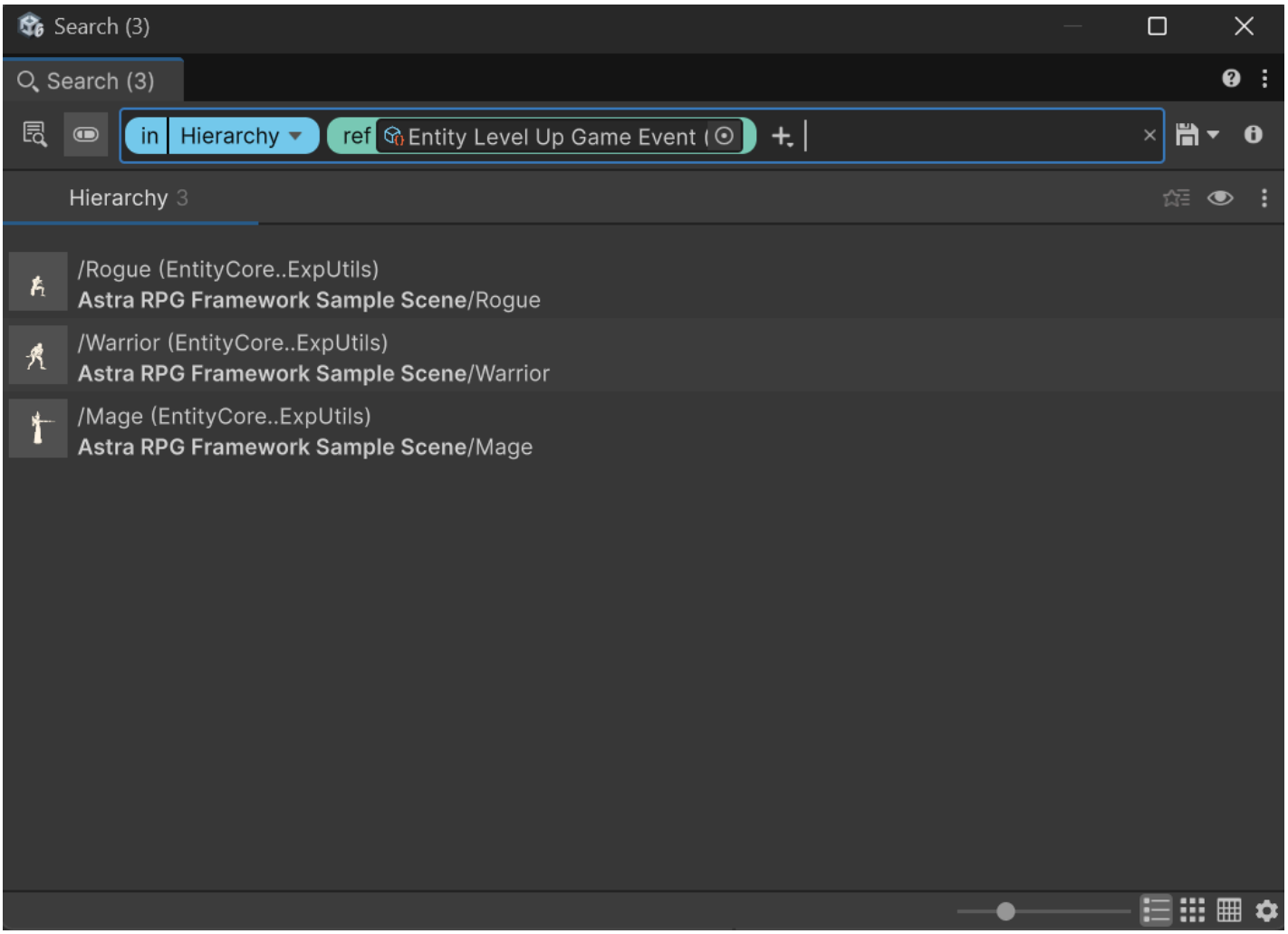
To update them, I suggest to search your project and scenes for usages of the deprecated `EntityLeveledUpGameEvent` and `EntityLeveledDownGameEvent`. To do this, I would suggest you to use the Unity Editor search functionality:

1. Right-click on the scriptable object instance of the deprecated event you want to search for (e.g., `Entity Leveled Up Game Event`).
2. Select `Find References In Project`. A window like this will open:



3. Inspect the found references to identify where you used the deprecated event and update them to use the new `EntityLevelUpGameEvent` or `EntityLevelDownGameEvent` and/or the respective listeners instead. You'll catch also the GameEvent listeners as they reference the event instance.
4. Right-click again on the scriptable object instance of the deprecated event and select `Find References In Scene` to find any usage in the currently opened scene. You will see the hierarchy window changed, but no meaningful references will be shown. However, you can click on the top-right button to open a window similar to the previous one, showing all references in the scene:


Hierarchy
ref:"Assets/AstraRpgFramework/Utils/EventInstances/1
Open in Search (C
+K)
Astra RPG Framework Sample Scene*
ClassName
ClassName
ClassName
HeroPanel
HeroPanel
HeroPanel
Lvl
Lvl
Lvl
Mage
MaxHP
MaxHP
MaxHP
NextSectionBtn
NextSectionBtn
NextSectionBtn
PrvSectionBtn
PrvSectionBtn
PrvSectionBtn
Rogue
Warrior



5. Inspect the found references in the scene and update them to use the new `EntityLevelUpGameEvent` or `EntityLevelDownGameEvent` and/or the respective listeners instead.
6. Repeat steps 4-5 for each scene in your project.
7. Repeat steps 1-6 for the other deprecated event.

Migrating to v1.1.0

The rebranding of SOAP RPG Framework to Astra RPG Framework involves several changes that need to be addressed when updating existing projects. This guide outlines the necessary steps to ensure a smooth transition.

1. Backup Your Project

Before making any changes, it's crucial to back up your project. Whether you are using version control or simply creating a copy of your project folder, having a backup will allow you to revert to the previous state if anything goes wrong during the migration process.

2. Update the Package from the Package Manager

Open Unity and navigate to the Package Manager (`Window -> Package Manager`). In the "In This Project" section, locate the SOAP RPG Framework package. Update it to version 1.1.0, which is now listed as Astra

RPG Framework.

3. Update Namespaces in Your Code

If your project contains scripts that reference the old SOAP RPG Framework namespaces, you'll need to update these references to the new Astra RPG Framework namespaces. For doing this, you can use your IDE's find-and-replace-all functionality (`Ctrl + Shift + R` in VSCode):

1. Open the find-and-replace dialog in your IDE.
2. Type `ElectricDrill.SoapRpgFramework` and assert that the "Match Case" option is enabled.
3. Inspect the found occurrences to ensure they are correct.
4. Replace all occurrences that makes sense to you with `ElectricDrill.AstraRPGFramework`.
5. Save all modified files.
6. Return to Unity and recompile the project. If this is not automatic, you can force recompilation with `Ctrl + R`.

If your project compiles correctly, you're done! If you get errors related to duplicate `.asmdef` files, follow the next steps.

4. Remove duplicate `.asmdef` files in the package

1. In the hierarchy of your project, navigate to the `Packages` folder and locate the `Astra RPG Framework` package.
2. Open the `Runtime` folder.
3. Ensure that there is only one `.asmdef` file named `com.electricdrill.astra-rpg-framework.Runtime`. If you find also a file named `com.electricdrill.soap-rpg-framework.Runtime`, delete it.
4. Open now the `Editor` folder of the package.
5. Ensure that there is only one `.asmdef` file named `com.electricdrill.astra-rpg-framework.Editor`. If you find also a file named `com.electricdrill.soap-rpg-framework.Editor`, delete it.
6. Click on the `com.electricdrill.astra-rpg-framework.Editor` file to select it. In the Inspector window, in the `Assembly Definition References` section, add the reference to `com.electricdrill.astra-rpg-framework.Runtime` if it's not already present, and delete any reference to `com.electricdrill.soap-rpg-framework.Runtime` if present.
7. Click on apply to save the changes.

> **NOTE**
>
> Unity sometimes has unpredictable behaviors when updating `.asmdef` files in and their references. After applying the changes mentioned above, try also to close and reopen Unity to ensure that the changes are correctly applied. If you still encounter issues, fix the new problems and re-start Unity again. At that point, the changes should be correctly applied.

5. Recompile the Project

If Unity didn't automatically recompile the project after these changes, you can force recompilation.

Your project should now be successfully migrated to Astra RPG Framework v1.1.0.
Happy developing!

Troubleshooting

I re-imported Samples that I already had from v1.0.0 and now I have errors

If you used the ScriptableObject based samples of the `Utils` folder in your project, you can keep using them as they are fully compatible with the new version. In fact, **you should not re-import them as that would create duplicates in your project**.

> **WARNING**
> Do not delete any ScriptableObject based asset that you are using in your project! Delete the duplicates from the newly imported samples instead. This will prevent data loss in your project.

Moreover, it was noticed that re-importing samples in a project that was using v1.0.0 of the package, resulted in old namespaces being used in the samples scripts. This is likely a Unity bug as does not happen on a fresh project. If this happens, please rename the old `ElectricDrill.SoapRpgFramework` namespace in the scripts of the samples to `ElectricDrill.AstraRPGFramework` manually, analogously to what is described in step #3.

Still Need Help?

For any issue during the migration, feel free to reach me out by sending me an email at electricdrill.info@gmail.com